

[NOM DE L'ÉTABLISSEMENT]

[Faculté / Département / Filière]

RAPPORT DE STAGE

Conception, développement et expérimentation de Droplet : une application de communication décentralisée fonctionnant sur un réseau maillé sans connexion Internet

Présenté par : [Nom et prénom de l'étudiant]

Matricule : [Matricule]

Entreprise d'accueil : [Nom de l'entreprise]

Maître de stage : [Nom du maître de stage]

Encadreur académique : [Nom de l'encadreur académique]

Année académique : [20XX / 20XX]

Période du stage : [du .. au ..]

À COMPLÉTER PAR VOUS — *Insérez le logo de votre établissement en haut de cette page, et remplacez tous les champs entre crochets.*

PAGE D'APPROBATION / VALIDATION

Ce rapport a été rédigé dans le cadre du stage effectué au sein de [Nom de l'entreprise], et présenté en vue de l'obtention du diplôme de [intitulé du diplôme].

Signature de l'encadreur académique : _____

Signature du maître de stage : _____

Visa de l'entreprise : _____

DÉDICACE

À COMPLÉTER PAR VOUS — Rédigez ici quelques lignes personnelles. Personne ne peut écrire cette page à votre place — elle doit vous ressembler.

REMERCIEMENTS

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et de ce rapport.

À COMPLÉTER PAR VOUS — *Complétez en citant nommément : votre famille, votre établissement, vos enseignants, votre encadreur académique, l'entreprise d'accueil, votre maître de stage. Citez des noms précis — un remerciement générique se voit immédiatement.*

RÉSUMÉ

Dans un contexte de dépendance croissante aux infrastructures Internet et télécom pour les communications quotidiennes, les coupures réseau, les zones mal couvertes et les situations d'urgence révèlent la fragilité de ces systèmes centralisés. Ce rapport présente le travail réalisé dans le cadre d'un stage au sein de [Nom de l'entreprise], portant sur la conception, le développement et l'expérimentation de Droplet, une application mobile de communication décentralisée fonctionnant sur un réseau maillé (Mesh), sans Internet, sans opérateur et sans serveur central.

Le problème traité est celui de la fiabilité et de la confidentialité des communications lorsque l'infrastructure classique est indisponible. La démarche adoptée combine analyse des besoins, conception UML, choix technologiques (Flutter, Dart, Riverpod, SQLite/Drift, BLE, Wi-Fi Direct, WebRTC), développement du relais Mesh multi-hop et implémentation d'un chiffrement de bout en bout (X25519, AES-256-GCM).

Le prototype obtenu représente 45 185 lignes de code Dart réparties en 93 fichiers, 26 écrans applicatifs, 17 services techniques et 8 tables de persistance. L'évaluation a été menée selon deux axes complémentaires : une suite de 63 tests automatisés, et un banc d'essai à deux paires mesurant le taux de livraison réel sur un lien radio simulé. Ce banc a mis en évidence un défaut majeur de fiabilité — des messages comptabilisés comme livrés alors qu'aucun octet n'avait quitté l'appareil — qui a été corrigé et dont la non-régression est désormais garantie par un test dédié.

Ce travail ouvre des perspectives d'amélioration concernant la fragmentation applicative des messages longs, la signalisation des transferts en attente de Wi-Fi, l'optimisation énergétique et la validation sur appareils multiples.

Mots-clés

Droplet, réseau Mesh, messagerie hors ligne, Flutter, communication P2P, chiffrement de bout en bout, BLE, WebRTC, fiabilité de livraison.

À COMPLÉTER PAR VOUS — Ajoutez une à deux phrases décrivant VOTRE contribution personnelle précise, à la fin du deuxième paragraphe.

ABSTRACT

As reliance on Internet and telecom infrastructure for everyday communication keeps growing, network outages, poorly covered areas and emergency situations expose the fragility of these centralized systems. This report presents the work carried out during an internship at [Company name], focused on the design, development and experimentation of Droplet, a mobile application for decentralized communication operating over a mesh network, without Internet, without a carrier and without a central server.

The problem addressed is the reliability and confidentiality of communications when conventional infrastructure is unavailable. The approach combines requirements analysis, UML design, technology choices (Flutter, Dart, Riverpod, SQLite/Drift, BLE, Wi-Fi Direct, WebRTC), development of a multi-hop mesh relay, and end-to-end encryption (X25519, AES-256-GCM).

The resulting prototype amounts to 45,185 lines of Dart code across 93 files, 26 application screens, 17 technical services and 8 persistence tables. Evaluation followed two complementary tracks: an automated suite of 63 tests, and a two-peer test bench measuring the actual delivery ratio over a simulated radio link. This bench revealed a major reliability defect — messages reported as delivered although no byte had left the device — which was fixed, with a dedicated regression test now guarding against its return.

Keywords

Mesh Network, Offline Messaging, Flutter, P2P Communication, End-to-End Encryption, BLE, WebRTC, Delivery Reliability.

LISTE DES FIGURES

SYNTHÈSE — CE QUE CE RAPPORT ÉTABLIT

Un prototype fonctionnel et mesuré

45 185 lignes de Dart · 93 fichiers · 26 écrans · 17 services techniques · 8 tables de persistance · 63 tests automatisés.

Trois transports radio complémentaires

Bluetooth Low Energy pour la découverte et les messages courts, Wi-Fi local et Wi-Fi Direct pour les fichiers et la voix, avec bascule automatique selon la santé mesurée de chaque lien.

Un chiffrement de bout en bout vérifié, pas seulement déclaré

X25519 et AES-256-GCM pour les échanges individuels, mécanisme de Sender Keys avec ratchet pour les groupes. Le déchiffrement est validé par un correspondant indépendant qui dérive la clé de son côté.

Une découverte qui a changé la mesure de la fiabilité

L'application comptabilisait comme livrés des messages qui n'avaient jamais quitté l'appareil : 130 annoncés reçus pour 110 réellement reçus. La cause — une méthode d'envoi sans valeur de retour — a été corrigée sur sept couches, et un test de non-régression en conserve la trace. Le taux affiché par l'application veut désormais dire quelque chose.

Ce qui reste à valider, et qui est dit comme tel

Le relais multi-hop — cœur de la proposition — n'a pas pu être éprouvé sur le terrain : il exige trois appareils. La consommation énergétique n'a pas été mesurée. Ces limites sont exposées au chapitre 20 plutôt que passées sous silence.

Figure 1 : Architecture en couches de Droplet

Figure 2 : Principe du relais multi-hop ($A \rightarrow B \rightarrow C \rightarrow D$)

Figure 3 : Structure d'une trame Droplet

Figure 4 : Diagramme d'état d'un message

Figure 5 : Taux de livraison avant et après correction

Figure 6 : Taille des trames face à la limite Bluetooth

Figure 7 : Répartition du code par module

Figure 8 : Propagation du verdict d'envoi

Figure 9 : Séquence d'envoi d'un message chiffré

Figure 10 : Écran d'accueil — liste des discussions

Figure 11 : Conversation et menu contextuel

Figure 12 : Informations d'une conversation

Figure 13 : Écrans Pairs et Appels

Figure 14 : Réglages et sélecteur d'icône

Figure 15 : Vérification d'identité par QR code

Figure 16 : Composition d'un statut

Figure 17 : Modes clair et sombre

Figure 18 : Cartes hors connexion

Figure 19 : Mode urgence

Figure 20 : Diagramme de cas d'utilisation général

Figure 21 : Diagramme de classes du modèle applicatif

Figure 22 : Diagramme d'activité — envoi avec relances

Figure 23 : Diagramme de composants

Figure 24 : Diagramme de déploiement

Figure 25 : Modèle conceptuel de données (MCD)

Figure 26 : Schéma physique des huit tables

Figure 4 : Séquence d'envoi d'un message chiffré

Figure 5 : Diagramme d'état d'un message

Figure 6 : Structure d'une trame Droplet

Figure 7 : Écran de conversation (mode sombre)

Figure 8 : Écran de conversation (mode clair)

Les dix-neuf figures sont déjà en place dans le document. Dans Word, vous pouvez remplacer cette liste par une table des illustrations automatique : Références → Insérer une table des illustrations.

LISTE DES TABLEAUX

Tableau 1 : Comparaison des solutions existantes

Tableau 2 : Besoins fonctionnels

Tableau 3 : Priorisation MoSCoW

Tableau 4 : Dépendances logicielles du projet

Tableau 5 : Tables de la base de données

Tableau 6 : Constantes du protocole Mesh

Tableau 7 : Résultats du banc d'essai à deux pairs

Tableau 8 : Taille des trames face à la limite BLE

Tableau 9 : Défauts détectés et corrigés

Tableau 10 : Tableau de synthèse des tests

LISTE DES ABRÉVIATIONS

Abréviation	Signification
AES	Advanced Encryption Standard
AIMD	Additive Increase / Multiplicative Decrease
API	Application Programming Interface
BLE	Bluetooth Low Energy
E2EE	End-to-End Encryption (chiffrement de bout en bout)
ECDH	Elliptic Curve Diffie-Hellman
GATT	Generic Attribute Profile (profil BLE)
GCM	Galois/Counter Mode
HKDF	HMAC-based Key Derivation Function
Mesh	Réseau maillé
MTU	Maximum Transmission Unit
ORM	Object-Relational Mapping
P2P	Peer-to-Peer (pair-à-pair)
PDR	Packet Delivery Ratio (taux de livraison)
SQL	Structured Query Language
TOFU	Trust On First Use
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
WebRTC	Web Real-Time Communication

TABLE DES MATIÈRES

La table ci-dessous se met à jour automatiquement : clic droit → « Mettre à jour les champs » → « Mettre à jour toute la table ».

Faites un clic droit ici puis « Mettre à jour les champs » pour générer la table.

INTRODUCTION GÉNÉRALE

1. Contexte général

La transformation numérique a profondément modifié la manière dont les individus et les organisations communiquent. Les communications mobiles occupent aujourd'hui une place centrale dans la vie quotidienne, qu'il s'agisse d'échanges personnels, professionnels ou d'urgence. Cette centralité repose toutefois presque entièrement sur la disponibilité des réseaux Internet et télécom : antennes relais, data centers et infrastructures opérateur.

Cette dépendance devient un point de fragilité dès lors que ces infrastructures sont absentes, saturées ou hors service : zones rurales mal couvertes, catastrophes naturelles, coupures volontaires ou accidentelles, grands rassemblements saturant le réseau, ou encore environnements professionnels isolés. Dans ces situations, l'incapacité à communiquer peut avoir des conséquences importantes, allant de la simple gêne à un risque réel pour la sécurité des personnes.

Face à ce constat, des solutions de communication résilientes, capables de fonctionner indépendamment des infrastructures classiques, apparaissent comme une nécessité pour certains usages spécifiques.

2. Contexte du stage

À COMPLÉTER PAR VOUS — Présentez l'entreprise d'accueil, son domaine d'activité, la mission qui vous a été confiée et le contexte technologique du projet (équipe, méthodes de travail, outils).

3. Problématique

La problématique centrale de ce travail peut être formulée ainsi : comment permettre à des utilisateurs de communiquer de manière fiable, privée et résiliente lorsque l'infrastructure Internet ou télécom n'est pas disponible ?

Cette question en contient une seconde, plus difficile, que l'expérimentation a mise au premier plan : comment SAVOIR qu'une communication a réellement abouti, dans un système où il n'existe aucun serveur central pour en témoigner ? Cette question de la mesure de la fiabilité s'est révélée aussi structurante que celle du transport lui-même.

4. Hypothèse

L'hypothèse retenue est qu'un ensemble de smartphones, capables de communiquer directement entre eux à courte portée (Bluetooth Low Energy, Wi-Fi Direct) et de relayer les messages les uns pour les autres, peut constituer un réseau maillé (Mesh) permettant l'échange d'informations sans passer par une infrastructure centralisée.

5. Présentation de Droplet

Droplet est une application mobile de messagerie et d'appels conçue pour fonctionner sans connexion Internet, sans opérateur téléphonique et sans serveur central. Elle repose sur une communication pair-à-pair (P2P) entre appareils à proximité, organisée en réseau maillé capable de relayer les messages sur plusieurs sauts successifs. Chaque utilisateur dispose d'une identité cryptographique locale, les messages sont chiffrés de bout en bout, et les données sont stockées localement sur l'appareil.

À l'issue du stage, le prototype comporte les fonctionnalités suivantes, toutes opérationnelles : messagerie individuelle et de groupe, partage de fichiers et de photos, messages vocaux avec forme d'onde, partage de position avec mini-carte, statuts éphémères (photo, vidéo, texte, vocal) avec réactions et commentaires, appels audio pair-à-pair, cartographie hors connexion, mode urgence, sauvegarde chiffrée de l'identité, et vérification d'identité par QR code.

6. Objectif général du stage

À COMPLÉTER PAR VOUS — Formulez l'objectif général de VOTRE stage. Exemple de formulation : « Contribuer à la conception, au développement et à l'évaluation du prototype Droplet, en particulier sur les modules X et Y qui m'ont été confiés. » Adaptez au périmètre réel.

7. Objectifs spécifiques

À COMPLÉTER PAR VOUS — Listez 3 à 5 objectifs spécifiques et MESURABLES correspondant à vos missions réelles. Pour vous aider, voici des formulations tirées du travail effectivement réalisé — ne retenez que celles qui vous correspondent :

- Implémenter la découverte de pairs et le transport sur trois technologies radio (BLE, Wi-Fi local, Wi-Fi Direct) avec bascule automatique.
- Concevoir et implémenter le protocole de relais multi-hop avec routage adaptatif, contrôle de congestion et déduplication.
- Intégrer un chiffrement de bout en bout (X25519 + AES-256-GCM) pour les conversations individuelles et de groupe.
- Concevoir un banc d'essai permettant de mesurer le taux de livraison réel du réseau et valider le prototype par cette mesure.
- Réaliser l'interface utilisateur complète selon les conventions d'ergonomie mobile, en modes clair et sombre.

8. Méthodologie

La démarche suivie tout au long du stage s'est articulée autour des étapes classiques d'un projet de développement logiciel : analyse des besoins et du contexte, conception (architecture, modélisation UML, base de données), développement itératif des fonctionnalités, tests et correction des anomalies détectées, amélioration continue au regard des retours obtenus, et documentation progressive du travail réalisé.

Une particularité mérite d'être soulignée, car elle a structuré une grande partie du travail : dans un système décentralisé, l'observation du comportement réel ne peut se faire ni par des journaux serveur ni par des métriques centralisées. La méthode retenue a donc combiné trois sources d'information complémentaires :

- les tests automatisés, qui valident la logique en isolation ;
- l'observation sur appareil réel via les journaux système (logcat) et les captures d'écran, qui seule révèle les défauts liés au matériel, aux permissions et au cycle de vie de l'application ;
- un banc d'essai à deux pairs, qui exécute le vrai code réseau contre un correspondant simulé afin de mesurer le taux de livraison.

Cette triple approche s'est révélée nécessaire : la majorité des défauts les plus graves n'était visible par AUCUN des trois moyens pris isolément. Le chapitre 18 détaille ce point avec des exemples précis.

9. Organisation du rapport

Ce rapport est structuré en six parties. La Partie I présente le cadre professionnel du stage. La Partie II développe l'analyse et la spécification du projet Droplet. La Partie III détaille la conception technique (architecture, UML, base de données, réseau Mesh, sécurité). La Partie IV décrit les travaux et la réalisation technique effectués durant le stage. La Partie V présente la stratégie de test, les résultats obtenus et les difficultés rencontrées. Enfin, la Partie VI dresse le bilan professionnel du stage et propose des perspectives d'évolution pour Droplet.

PARTIE I — CADRE PROFESSIONNEL DU STAGE

CHAPITRE 1 — PRÉSENTATION DE L'ENTREPRISE

Cette section décrit l'organisme d'accueil. Son contenu vous est strictement propre : aucune information de ce chapitre ne peut être déduite du code du projet.

1.1 Historique

À COMPLÉTER PAR VOUS — Retracer la création et les grandes étapes de l'évolution de l'entreprise (dates, fondateurs, tournants).

1.2 Situation géographique

À COMPLÉTER PAR VOUS — Indiquez l'adresse complète, et insérez éventuellement un plan d'accès ou une carte.

1.3 Secteur d'activité

À COMPLÉTER PAR VOUS — Précisez le domaine d'activité principal.

1.4 Missions

À COMPLÉTER PAR VOUS — Décrivez les missions et la raison d'être de l'entreprise.

1.5 Produits et services

À COMPLÉTER PAR VOUS — Listez les principaux produits ou services, en situant Droplet parmi eux.

1.6 Organisation

À COMPLÉTER PAR VOUS — Décrivez la structure interne (services, équipes, effectifs).

1.7 Organigramme

À COMPLÉTER PAR VOUS — Insérez le schéma de l'organigramme.

1.8 Présentation du service d'accueil

À COMPLÉTER PAR VOUS — Décrivez le service ou l'équipe qui vous a accueilli, et votre position dans cet ensemble.

1.9 Environnement professionnel

À COMPLÉTER PAR VOUS — Décrivez l'ambiance de travail, l'équipe, le mode de collaboration.

1.10 Environnement matériel

À COMPLÉTER PAR VOUS — Décrivez le matériel mis à votre disposition. N'oubliez pas les appareils de test : le nombre de smartphones disponibles a un impact direct sur ce qui a pu être validé (voir chapitre 17).

1.11 Environnement logiciel

À COMPLÉTER PAR VOUS — Listez les logiciels et plateformes utilisés (IDE, gestion de projet).

1.12 Méthodes de travail

À COMPLÉTER PAR VOUS — Décrivez la méthode de gestion de projet suivie (Agile, Scrum, Kanban, ou informelle — dites-le honnêtement).

1.13 Outils utilisés

À COMPLÉTER PAR VOUS — Listez les outils collaboratifs (Git, Jira, Trello, Figma, Slack...).

CHAPITRE 2 — DÉROULEMENT DU STAGE ET MISSIONS

2.1 Accueil et prise de connaissance de l'environnement

À COMPLÉTER PAR VOUS — Décrivez votre première semaine : accueil, présentation de l'équipe, découverte des outils.

2.2 Compréhension du projet

À COMPLÉTER PAR VOUS — Expliquez comment vous avez pris connaissance du projet Droplet : lecture du cahier des charges, du code existant, échanges avec l'équipe.

2.3 Analyse du projet existant

À COMPLÉTER PAR VOUS — Décrivez l'état du projet à votre arrivée. Soyez factuel : quelles fonctionnalités existaient, lesquelles étaient incomplètes.

2.4 Organisation du travail

À COMPLÉTER PAR VOUS — Décrivez comment votre travail était organisé (réunions, points d'avancement, autonomie).

2.5 Missions confiées

À COMPLÉTER PAR VOUS — Listez les missions confiées au démarrage.

2.6 Missions effectivement réalisées

Cette section est la plus importante du rapport pour le jury. Elle doit distinguer sans ambiguïté ce qui existait avant votre intervention, ce que vous avez développé, amélioré, testé et documenté.

À COMPLÉTER PAR VOUS — Structurez cette section selon les six rubriques ci-dessous, en citant des noms de fichiers et de fonctionnalités précis. Le chapitre 13 vous fournit la matière détaillée : reprenez-en les éléments qui correspondent effectivement à votre travail.

- Ce qui existait avant mon intervention
- Ce que j'ai développé moi-même
- Ce que j'ai amélioré
- Ce que j'ai testé
- Ce que j'ai documenté
- Ce qui reste à faire

2.7 Planning du stage

Le tableau ci-dessous synthétise les grandes étapes du stage. Il est recommandé de le compléter par un diagramme de Gantt en annexe B.

Période	Étape / Activité	Livrable
Semaine 1-2	[À compléter]	[À compléter]
Semaine 3-4	[À compléter]	[À compléter]

Mois 2	[À compléter]	[À compléter]
Mois 3	[À compléter]	[À compléter]

À COMPLÉTER PAR VOUS — Renseignez les périodes réelles et les livrables associés.

2.8 Bilan intermédiaire

À COMPLÉTER PAR VOUS — Résumez l'avancement à mi-parcours : objectifs atteints, difficultés, ajustements.

2.9 Bilan final

À COMPLÉTER PAR VOUS — Résumez l'état du projet et de vos acquis à la fin du stage.

PARTIE II — ANALYSE ET SPÉCIFICATION DE DROPLET

CHAPITRE 3 — CONTEXTE, PROBLÉMATIQUE ET OBJECTIFS

3.1 Contexte technologique

Les communications mobiles modernes reposent presque exclusivement sur des infrastructures centralisées : réseaux cellulaires (2G/3G/4G/5G), points d'accès Wi-Fi reliés à Internet, et serveurs applicatifs gérés par des opérateurs ou des fournisseurs de services. Ce modèle offre une couverture et une qualité de service élevées dans les zones bien desservies, mais reste entièrement dépendant de la disponibilité de ces infrastructures.

3.2 Limites des systèmes traditionnels

Ces systèmes montrent leurs limites dans plusieurs situations : absence de couverture réseau en zone rurale ou isolée, saturation lors de grands rassemblements, coupures lors de catastrophes naturelles ou de pannes techniques, ou encore restrictions imposées dans certains contextes. Dans tous ces cas, les utilisateurs se retrouvent dans l'incapacité de communiquer, alors même que le besoin peut être critique.

3.3 Problématique

Comment permettre à des utilisateurs de communiquer de manière fiable, privée et résiliente lorsque l'infrastructure Internet ou télécom n'est pas disponible ?

3.4 Causes du problème

Les causes principales sont d'ordre géographique (zones non couvertes), technique (pannes, saturation) ou circonstanciel (catastrophes, restrictions, environnements sans réseau comme certains sites industriels ou souterrains).

3.5 Conséquences

L'impossibilité de communiquer peut entraîner une perte de coordination, un risque accru en situation d'urgence, ainsi qu'une rupture du lien social ou professionnel.

3.6 Solution envisagée

La solution envisagée consiste à exploiter les capacités de communication directe des smartphones (Bluetooth Low Energy, Wi-Fi Direct) pour constituer un réseau maillé (Mesh) capable de relayer les messages de proche en proche, sans dépendre d'une infrastructure centrale.

3.7 Objectif général

Concevoir, développer et évaluer un prototype fonctionnel d'application de communication décentralisée, capable d'assurer l'échange de messages, de fichiers et d'appels entre utilisateurs proches, sans connexion Internet.

3.8 Objectifs spécifiques

Les objectifs spécifiques retenus, tels qu'ils se sont concrétisés dans le prototype, sont les suivants :

1. Établir une identité cryptographique locale, générée sur l'appareil et ne dépendant d'aucune autorité de certification.
2. Implémenter la découverte continue de pairs à proximité sur trois technologies radio distinctes, avec sélection automatique du transport le plus adapté.
3. Concevoir un protocole de relais multi-hop assurant la propagation des messages au-delà de la portée radio directe, avec limitation de la propagation et déduplication.
4. Garantir la confidentialité et l'intégrité des échanges par un chiffrement de bout en bout, y compris pour les conversations de groupe.
5. Assurer la persistance locale des messages, contacts et groupes, sans aucun stockage distant.
6. Mesurer le taux de livraison réel du réseau, et non le seul taux de tentative d'émission.

3.9 Utilisateurs cibles

Le cahier des charges identifie sept profils d'utilisateurs, dont les besoins diffèrent sensiblement :

Profil	Besoin dominant	Contrainte principale
Particuliers	Rester joignable hors couverture	Simplicité d'usage
Étudiants	Échanger sur un campus saturé	Consommation de batterie
Entreprises	Coordination sur site isolé	Confidentialité
ONG / humanitaire	Coordination en zone sinistrée	Robustesse
Zones rurales	Communication de proximité	Densité d'appareils faible
Situations d'urgence	Alerte et localisation	Fiabilité de livraison
Événements	Échanges en foule dense	Passage à l'échelle

Ces profils ne sont pas également servis par le prototype actuel. Les zones rurales, en particulier, souffrent d'une limite physique : la portée d'un saut étant de l'ordre de 10 à 50 mètres, une couverture continue exige une densité d'appareils que le milieu rural offre rarement. Ce point est repris au chapitre 20.

CHAPITRE 4 — CAHIER DES CHARGES

4.1 Expression des besoins

Le besoin exprimé est celui d'un moyen de communication mobile autonome, ne nécessitant ni carte SIM, ni abonnement, ni infrastructure réseau, capable de fonctionner de façon fiable et sécurisée entre appareils à proximité.

4.2 Besoins fonctionnels

Code	Besoin fonctionnel	État dans le prototype
BF01	Création d'identité	Implémenté
BF02	Découverte des appareils	Implémenté (3 transports)
BF03	Ajout de contacts	Implémenté
BF04	Messagerie individuelle	Implémenté et chiffré
BF05	Accusés de réception	Implémenté (ACK groupés)
BF06	Relais Mesh	Implémenté (multi-hop, dédup.)
BF07	Messagerie de groupe	Implémenté (Sender Keys)
BF08	Partage de fichiers	Implémenté — Wi-Fi requis
BF09	Messages vocaux	Implémenté (forme d'onde)
BF10	Appels audio	Implémenté (WebRTC, Wi-Fi)
BF11	Appels vidéo	Partiel (audio prioritaire)
BF12	Notifications	Implémenté (service persistant)
BF13	Persistance locale	Implémenté (8 tables Drift)
BF14	Gestion des paramètres	Implémenté

À COMPLÉTER PAR VOUS — Vérifiez la colonne « État » au regard du périmètre réel de VOTRE stage, et corrigez si nécessaire.

4.3 Besoins non fonctionnels

Sécurité, confidentialité, performance, disponibilité, résilience, ergonomie, maintenabilité, portabilité et consommation énergétique constituent les exigences non fonctionnelles principales du projet. Trois d'entre elles ont fait l'objet d'une attention particulière et peuvent être quantifiées :

- Résilience — le système doit continuer à fonctionner lors de la perte d'un lien : la file d'attente relance jusqu'à 5 fois avec un délai exponentiel (200 ms, 400 ms, 800 ms, 1,6 s, 3,2 s).

- Performance — le traitement applicatif d'un envoi (chiffrement, mise en forme, mise en file) est mesuré à environ 62 ms, et la latence médiane en file à 1,5 ms.
- Consommation énergétique — la découverte de pairs adapte son rythme au niveau de batterie et à l'état de l'application (premier plan ou arrière-plan).

4.4 Contraintes techniques

L'application cible Android en priorité. Les technologies de communication de proximité utilisées (Nearby Connections, Wi-Fi Direct) sont spécifiques à cette plateforme ; le portage vers iOS exigerait des équivalents distincts (MultipeerConnectivity).

À COMPLÉTER PAR VOUS — Précisez les versions d'Android supportées et les modèles réellement testés durant votre stage.

4.5 Contraintes matérielles

À COMPLÉTER PAR VOUS — Précisez le matériel requis et disponible : nombre de smartphones, modèles, versions d'OS. Ce point conditionne directement la validité des tests du chapitre 17 : mentionnez honnêtement combien d'appareils vous avez.

4.6 Contraintes réseau

L'application doit fonctionner sans connexion Internet ni infrastructure télécom, en s'appuyant uniquement sur les liaisons de proximité (BLE, Wi-Fi local, Wi-Fi Direct). Cette contrainte a une conséquence peu intuitive, mise en évidence par l'expérimentation : les trois transports n'ont pas les mêmes capacités, et le Bluetooth ne peut pas porter de fichiers (voir section 11.16).

4.7 Contraintes de sécurité

Les échanges doivent être chiffrés de bout en bout, et les clés privées ne doivent jamais quitter l'appareil de l'utilisateur. Cette contrainte est respectée : la clé privée X25519 est conservée dans le stockage sécurisé de la plateforme (Android Keystore) via flutter_secure_storage, et n'est jamais sérialisée dans la base de données applicative.

4.8 Priorisation MoSCoW

Catégorie	Contenu
Must have	Création d'identité, messagerie individuelle, relais Mesh, chiffrement de bout en bout, persistance locale
Should have	Messagerie de groupe, partage de fichiers, notifications, accusés de réception
Could have	Appels audio/vidéo, messages vocaux, statuts, partage de position, cartographie hors ligne
Won't have (cette version)	[À compléter selon le périmètre réel de votre stage]

4.9 Critères d'acceptation

Les critères d'acceptation définis pour les besoins prioritaires sont les suivants. Ils ont servi de référence lors de l'évaluation (chapitre 17).

Besoin	Critère d'acceptation
BF01	Une identité créée hors ligne permet de recevoir un message chiffré d'un pair rencontré ensuite.
BF04	Un message envoyé à un pair joignable est déchiffré à l'identique par le destinataire, et par lui seul.
BF05	Un message annoncé « livré » à l'expéditeur a effectivement été reçu par le destinataire — la nuance est décisive, voir 17.4.
BF06	Un message destiné à un pair hors de portée directe est délivré via au moins un relais intermédiaire.
BF07	Un message de groupe est déchiffrable par tous les membres actifs, et par aucun ancien membre.
BF13	Les messages survivent à la fermeture complète de l'application et au redémarrage de l'appareil.

4.10 État initial

À COMPLÉTER PAR VOUS — Décrivez l'état du projet avant votre arrivée en stage.

4.11 État final

À COMPLÉTER PAR VOUS — Décrivez l'état du projet à la fin de votre stage, besoin par besoin, en vous appuyant sur le tableau de la section 4.2.

CHAPITRE 5 — ÉTUDE DE L'EXISTANT

5.1 Messageries traditionnelles

Les messageries traditionnelles (SMS, applications OTT classiques) reposent sur une infrastructure opérateur ou sur des serveurs centraux accessibles via Internet, ce qui les rend inopérantes en l'absence de réseau.

5.2 WhatsApp

WhatsApp est une messagerie chiffrée de bout en bout très largement utilisée, mais qui nécessite une connexion Internet et repose sur une infrastructure serveur centralisée appartenant à Meta.

5.3 Telegram

Telegram propose une messagerie basée sur le cloud, avec chiffrement optionnel de bout en bout pour les conversations secrètes ; elle nécessite également une connexion Internet et dépend de serveurs centralisés.

5.4 Briar

Briar est une application de messagerie décentralisée conçue pour fonctionner via Bluetooth, Wi-Fi local ou le réseau Tor, sans serveur central, avec un fort accent mis sur la confidentialité et la résistance à la censure. Son modèle de sécurité est plus strict que celui de Droplet — la mise en relation exige un appairage explicite — au prix d'une prise en main plus exigeante.

5.5 Bridgefy

Bridgefy est une application de messagerie Mesh fonctionnant via Bluetooth, sans nécessiter de connexion Internet, utilisée notamment lors de rassemblements ou de manifestations dans des zones à connectivité limitée. Elle constitue la référence la plus proche du positionnement de Droplet.

5.6 Solutions Mesh existantes

À COMPLÉTER PAR VOUS — Complétez avec d'autres solutions étudiées durant votre stage (par exemple : Berty, Serval Mesh, goTenna, Meshtastic). Pour chacune, indiquez la technologie radio, le modèle de confiance et la limite principale.

5.7 Comparaison

Critère	WhatsApp	Telegram	Briar	Bridgefy	Droplet
Fonctionne sans Internet	Non	Non	Oui	Oui	Oui
Sans serveur central	Non	Non	Oui	Oui	Oui
Relais multi-	Non	Non	Partiel	Oui	Oui

hop					
Chiffrement bout en bout	Oui	Optionnel	Oui	Oui	Oui
Groupes	Oui	Oui	Oui	Limité	Oui
Appels audio	Oui	Oui	Non	Non	Oui (Wi-Fi)
Partage de fichiers	Oui	Oui	Oui	Non	Oui (Wi-Fi)
Identité locale	Non	Non	Oui	Oui	Oui

Tableau 1 — Comparaison des solutions existantes

La mention « Wi-Fi » pour Droplet n'est pas une réserve de forme : elle traduit une contrainte technique établie par l'expérimentation. Le Bluetooth Low Energy, tel qu'implémenté ici, écarte délibérément les contenus volumineux ; les appels et les fichiers exigent donc une liaison Wi-Fi. La section 11.16 en donne la justification chiffrée.

5.8 Analyse des avantages et limites

Solution	Avantage principal	Limite au regard de Droplet
WhatsApp	Adoption massive, ergonomie éprouvée	Inutilisable sans Internet — hors du besoin visé
Telegram	Richesse fonctionnelle	Chiffrement non systématique, dépendance au cloud
Briar	Modèle de sécurité rigoureux, hors ligne	Pas d'appels, mise en relation contraignante
Bridgefy	Mesh Bluetooth éprouvé en foule	Pas d'appels ni de fichiers, groupes limités

5.9 Besoin non couvert

Aucune des solutions étudiées ne combine à la fois l'absence totale d'infrastructure (Internet et serveur), un relais Mesh multi-hop robuste, un chiffrement de bout en bout systématique et un support natif des appels — c'est précisément ce positionnement que vise Droplet.

5.10 Positionnement de Droplet

Droplet se positionne comme une solution de communication décentralisée complète, combinant messagerie individuelle et de groupe, partage de fichiers, appels et relais Mesh multi-hop, tout en garantissant la confidentialité des échanges par un chiffrement de bout en bout.

CHAPITRE 6 — MODÈLE ÉCONOMIQUE ET VALORISATION

Important : les éléments de ce chapitre constituent une **PROPOSITION** de valorisation commerciale, et non une réalité financière établie.

6.1 Vision du projet

À COMPLÉTER PAR VOUS — Formulez la vision à long terme (exemple : devenir la solution de référence pour les communications résilientes en Afrique de l'Ouest).

6.2 Proposition de valeur

Permettre aux utilisateurs de communiquer localement même lorsque les infrastructures Internet ou télécom sont indisponibles, sans coût récurrent, sans compte et sans transmission de données à un tiers.

6.3 Segments de clientèle

Particuliers, entreprises, écoles, universités, ONG, organisations humanitaires, événements, collectivités et environnements industriels.

6.4 Canaux de distribution

À COMPLÉTER PAR VOUS — Précisez les canaux envisagés (stores mobiles, partenariats, déploiement direct auprès d'organisations).

6.5 Relation client

À COMPLÉTER PAR VOUS — Précisez le type de relation envisagée (support communautaire, support dédié pour les offres professionnelles).

6.6 Sources de revenus

Modèle freemium avec offre premium, offre professionnelle, licences, déploiement personnalisé, ainsi que support et maintenance pour les clients institutionnels.

6.7 Structure des coûts

À COMPLÉTER PAR VOUS — Listez les postes de coûts (développement, support, marketing). Notez que l'absence de serveur central supprime le poste « hébergement », ce qui est un argument fort.

6.8 Ressources clés

À COMPLÉTER PAR VOUS — Équipe technique, expertise sécurité, parc d'appareils de test.

6.9 Activités clés

À COMPLÉTER PAR VOUS — Développement, tests terrain, support, évolution du protocole Mesh.

6.10 Partenaires clés

À COMPLÉTER PAR VOUS — Fabricants d'appareils, ONG, collectivités, opérateurs pour des offres hybrides.

6.11 Business Model Canvas

À COMPLÉTER PAR VOUS — Insérez une représentation visuelle synthétisant les sections 6.1 à 6.10 (voir annexe L).

6.12 Stratégie de lancement

Phase 1 — Prototype ; Phase 2 — MVP ; Phase 3 — Tests terrain ; Phase 4 — Commercialisation ; Phase 5 — Expansion.

Au regard des résultats du chapitre 17, le projet se situe aujourd'hui entre les phases 1 et 2 : le prototype est fonctionnel et instrumenté, mais la validation terrain sur plusieurs appareils reste à mener.

6.13 Analyse SWOT

Forces	Faiblesses
Fonctionne sans aucune infrastructure ; chiffrement systématique ; aucun coût d'hébergement ; base de code instrumentée et testée	Dépendance au Wi-Fi pour les contenus volumineux ; consommation énergétique ; validation multi-appareils incomplète

Opportunités	Menaces
Zones à faible couverture ; marché humanitaire ; sensibilité croissante à la vie privée	Restrictions des systèmes d'exploitation sur les radios en arrière-plan ; concurrence établie (Bridgefy) ; effet de réseau à amorcer

À COMPLÉTER PAR VOUS — Affinez ce SWOT selon votre analyse propre.

6.14 Risques économiques

À COMPLÉTER PAR VOUS — Identifiez les principaux risques (adoption, concurrence, coûts, dépendance à des technologies tierces).

6.15 Perspectives de rentabilité

À COMPLÉTER PAR VOUS — Présentez, avec prudence et en le qualifiant clairement de proposition, un scénario de rentabilité à moyen terme.

PARTIE III — CONCEPTION TECHNIQUE

CHAPITRE 7 — CHOIX DES TECHNOLOGIES

7.1 Critères de sélection

Les technologies ont été choisies selon plusieurs critères : compatibilité multiplateforme (Android/iOS), support natif ou via plugin des communications de proximité (BLE, Wi-Fi Direct), performance, maturité de l'écosystème et facilité de maintenance.

7.2 Flutter

Flutter est un framework open-source développé par Google permettant de construire des applications natives pour Android et iOS à partir d'une base de code unique écrite en Dart, avec un rendu graphique performant via son propre moteur de rendu. Le projet utilise Flutter 3.41.6 (canal stable) avec le moteur de rendu Impeller sur Vulkan, ce qui a été vérifié sur l'appareil de test.

7.3 Dart

Dart est le langage de programmation utilisé par Flutter. Orienté objet et fortement typé, il permet la compilation native (AOT) pour des performances proches du natif, ainsi que le hot-reload qui accélère le développement. Le projet utilise le SDK Dart 3.11.

7.4 Riverpod

Riverpod est une bibliothèque de gestion d'état pour Flutter, successeur de Provider. Elle permet une gestion réactive, testable et découplée de l'état de l'application. Son intérêt est particulièrement net ici : l'état du réseau Mesh change en permanence et de façon asynchrone (pairs qui apparaissent, messages qui arrivent, transports qui tombent), et Riverpod permet à l'interface de s'y adapter sans que la couche réseau ait à connaître l'existence des écrans.

7.5 SQLite

SQLite est un moteur de base de données relationnelle léger, embarqué directement dans l'application, sans nécessiter de serveur, ce qui en fait un choix naturel pour le stockage local requis par Droplet.

Ce choix a produit un bénéfice imprévu, exploité pour la cartographie hors ligne : le format MBTiles, standard de distribution des tuiles cartographiques, est lui-même une base SQLite. Le cache de cartes a donc pu être implémenté sans ajouter aucune dépendance, en réutilisant le moteur déjà présent.



Figure 18 — Gestion des cartes hors connexion, avec la mention des sources et de leurs licences

7.6 Drift

Drift est une couche d'abstraction (ORM) au-dessus de SQLite pour Dart/Flutter, offrant des requêtes typées, une génération de code et une gestion simplifiée des migrations de schéma. Le typage fort évite une classe entière d'erreurs : une colonne renommée provoque une erreur de compilation, là où une requête SQL écrite à la main n'échouerait qu'à l'exécution, chez l'utilisateur.

7.7 BLE (Bluetooth Low Energy)

Le BLE permet une découverte et une communication à faible consommation d'énergie entre appareils proches, adaptée à la découverte continue de pairs dans le réseau Mesh. Le projet utilise le paquet `bluetooth_low_energy`, qui expose à la fois le rôle central (celui qui scanne et se connecte) et le rôle périphérique (celui qui annonce sa présence) — les deux sont nécessaires, puisque chaque appareil doit pouvoir découvrir et être découvert.

7.8 Wi-Fi

Le Wi-Fi local permet des débits plus élevés que le BLE, utile pour le transfert de fichiers ou de flux audio/vidéo entre appareils à proximité. Le projet l'utilise via des sockets TCP directes, avec découverte par balises UDP et par le service nsd (Network Service Discovery).

7.9 Wi-Fi Direct / P2P

Le Wi-Fi Direct permet une connexion directe entre deux appareils sans point d'accès intermédiaire. Il est exploité via le paquet `nearby_service`, qui s'appuie sur l'API Nearby Connections d'Android. Une contrainte importante de cette API a été découverte à l'expérimentation et a nécessité une adaptation du code : la couche native ne traite qu'une seule opération de connexion à la fois, et toute tentative parallèle échoue avec le statut BUSY (voir section 18.4).

7.10 WebRTC

WebRTC est une technologie standardisée permettant l'échange de flux audio et vidéo en temps réel entre pairs, y compris sur des liaisons locales. Son intérêt majeur ici est qu'elle fonctionne sans serveur de média : seule la signalisation (échange des descriptions de session) doit transiter, et elle emprunte le réseau Mesh lui-même.

7.11 Cryptographie

Les primitives cryptographiques retenues permettent d'assurer la confidentialité et l'intégrité des échanges tout en restant adaptées aux contraintes de performance des appareils mobiles :

- X25519 pour l'échange de clés (courbe elliptique Curve25519) ;
- AES-256-GCM pour le chiffrement symétrique, qui apporte simultanément confidentialité et authentification ;
- HMAC-SHA-256 pour la dérivation de clés en chaîne (mécanisme de ratchet des groupes) ;
- SHA-256 pour la génération de l'identifiant d'identité et le calcul des numéros de sécurité.

Ces primitives sont fournies par le paquet `cryptography`, implémentation Dart pure et audité, plutôt que par des liaisons natives — ce qui garantit un comportement identique sur toutes les plateformes.

7.12 Stockage sécurisé

Les clés privées et données sensibles sont stockées via les mécanismes de stockage sécurisé natifs de la plateforme (Keystore Android, Keychain iOS) au moyen du paquet `flutter_secure_storage`, plutôt qu'en clair dans la base de données applicative. La distinction est essentielle : une sauvegarde du contenu de l'application ne doit jamais emporter la clé privée avec elle.

7.13 Justification des choix

Le tableau ci-dessous récapitule les dépendances effectivement utilisées dans le projet, groupées par rôle.

Rôle	Paquets
Interface et navigation	flutter_riverpod, go_router, flutter_animate, google_fonts, font_awesome_flutter, cupertino_icons, motor, liquid_glass_renderer, flutter_shaders
Communication de proximité	bluetooth_low_energy, nsd, nearby_service
Temps réel	flutter_webrtc, flutter_ringtone_player
Persistance	drift, path_provider, flutter_secure_storage
Cryptographie	cryptography, crypto, uuid
Média	record, audioplayers, camera, video_player, file_picker, share_plus, receive_sharing_intent
Système	permission_handler, battery_plus, flutter_foreground_task, flutter_local_notifications
Cartographie et position	flutter_map, geolocator, http
Identité visuelle	qr_flutter, mobile_scanner

Tableau 4 — Dépendances logicielles du projet

CHAPITRE 8 — ANALYSE UML

8.1 Identification des acteurs

Acteur	Rôle dans le système
Utilisateur	Crée son identité, échange messages, fichiers et appels
Appareil pair (feuille)	Nœud standard du maillage, émetteur et récepteur
Appareil relais / passerelle	Retransmet les messages destinés à d'autres nœuds ; rôle attribué dynamiquement
Administrateur de groupe	Crée un groupe, ajoute ou retire des membres, renomme le groupe
Système d'exploitation	Acteur secondaire : accorde les permissions, peut suspendre l'application ou couper les encodeurs

Le dernier acteur mérite d'être explicité, car il est souvent oublié dans les modélisations : le système d'exploitation n'est pas un décor passif. Il accorde ou refuse les permissions, suspend l'application en arrière-plan, et peut désactiver des composants matériels — comportement effectivement rencontré et documenté en section 18.9.

8.2 Cas d'utilisation

Les cas d'utilisation principaux identifiés sont les suivants :

Cas d'utilisation	Acteur principal	Précondition
Créer une identité	Utilisateur	Aucune
Découvrir les pairs à proximité	Utilisateur	Radios activées
Envoyer un message	Utilisateur	Identité créée
Recevoir et déchiffrer un message	Utilisateur	Clé publique de l'émetteur connue
Relayer un message	Appareil relais	Message non déjà vu, compteur de sauts non épuisé
Créer un groupe	Utilisateur	Au moins un pair connu
Partager un fichier	Utilisateur	Liaison Wi-Fi disponible
Passer un appel audio	Utilisateur	Liaison Wi-Fi disponible
Vérifier l'identité d'un contact	Utilisateur	Contact déjà rencontré
Publier un statut	Utilisateur	Identité créée

Partager sa position	Utilisateur	Permission de localisation
Déclencher une alerte d'urgence	Utilisateur	Identité créée

8.3 Diagramme de cas d'utilisation général

Le diagramme ci-dessous reprend les douze cas d'utilisation du tableau précédent. Il fait apparaître un acteur souvent omis dans ce type de modélisation : l'appareil pair lui-même, qui relaie les messages des autres sans intervention de son propriétaire.

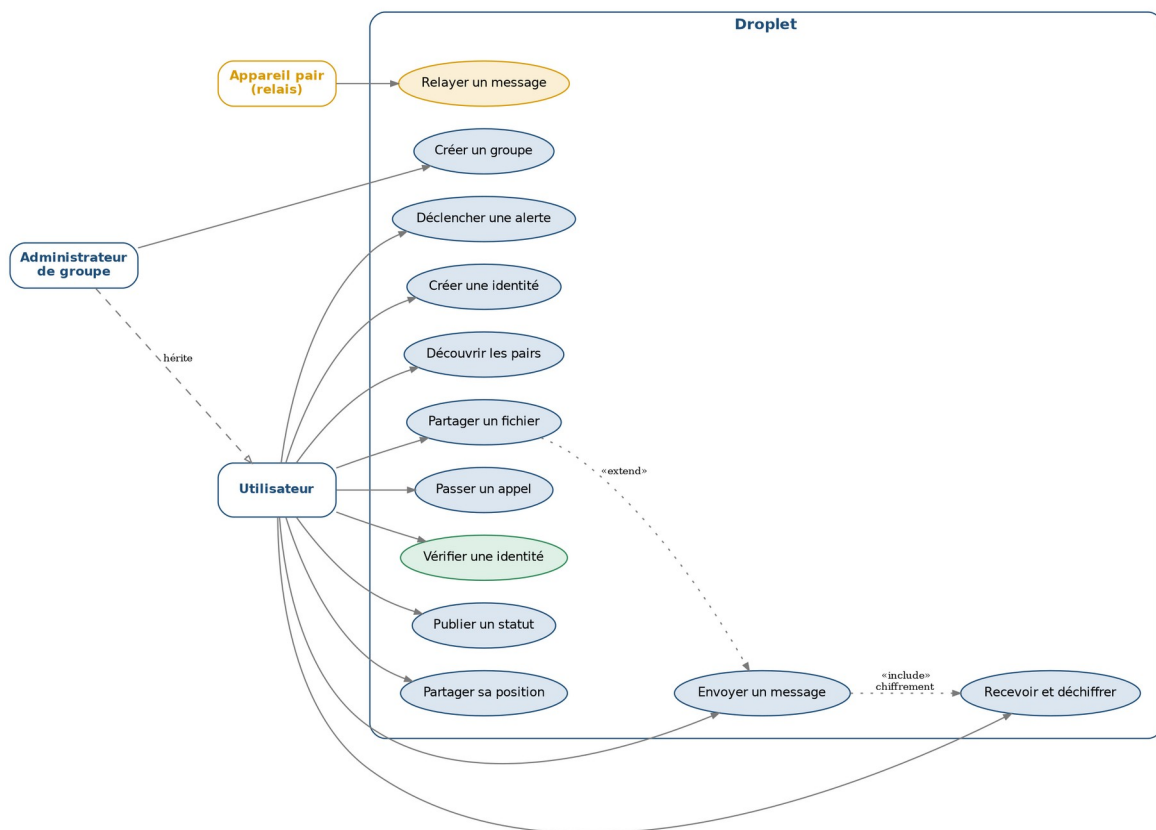


Figure 20 — Diagramme de cas d'utilisation général

8.4 Diagramme de classes

Les classes principales du modèle applicatif sont : `DropletUserModel` (identité locale), `MeshMessage` (message avec ses métadonnées de transport), `ConnectedPeer` (pair joignable et ses transports actifs), `PeerRecord` (pair connu et persistant, avec son état de vérification), `GroupInfo` et `GroupMember` (groupes), `StatusMedia` (média d'un statut), et `VoiceNoteMeta` (métadonnées d'un message vocal).

Le diagramme ci-dessous présente les sept classes principales du modèle, telles qu'elles existent dans `lib/core/models/`. `MeshMessage` y occupe une place centrale : elle porte à la fois le

contenu, les métadonnées de transport (compteur de sauts, statut), les références aux pièces jointes et le lien vers le message auquel elle répond.

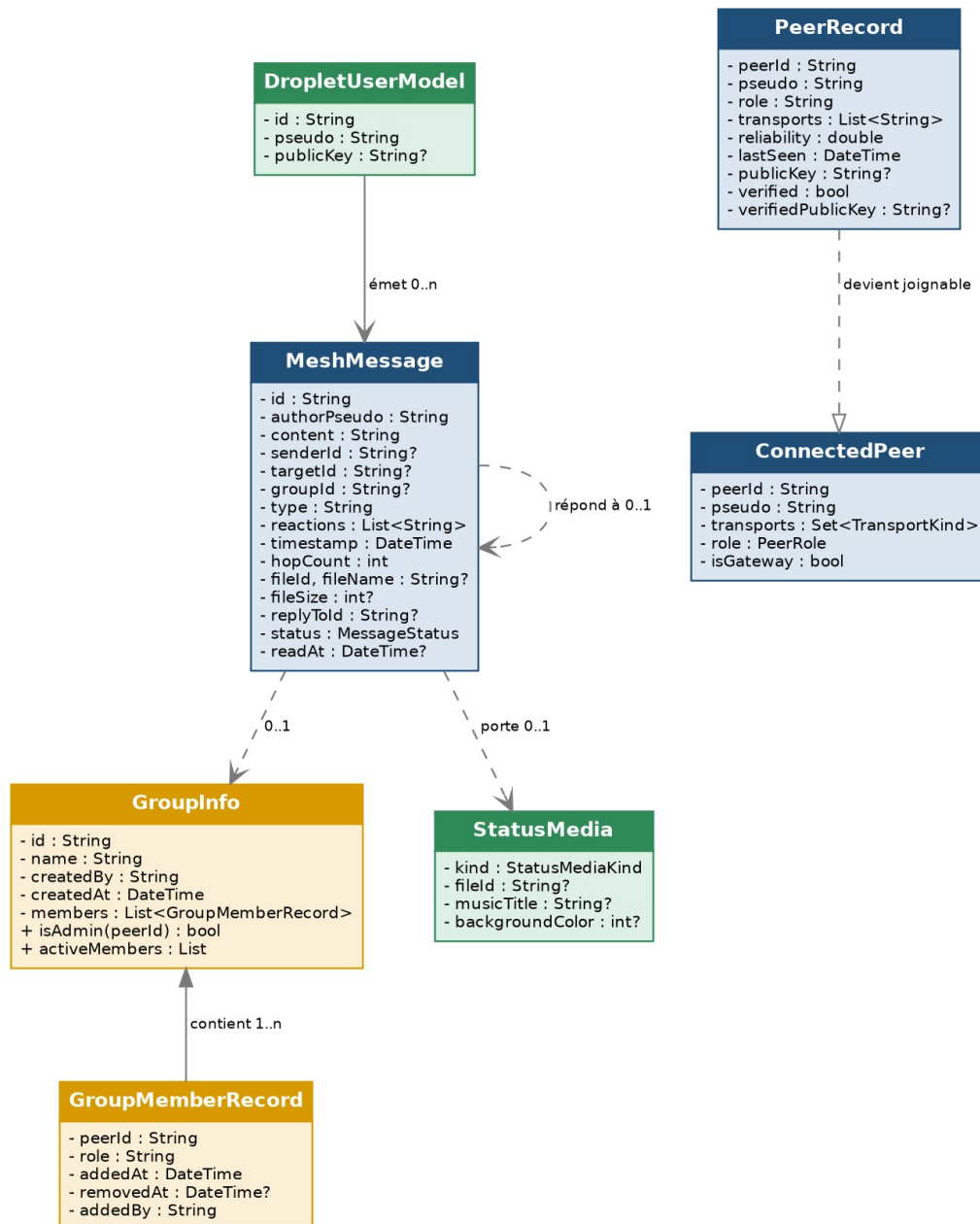


Figure 21 — Diagramme de classes du modèle applicatif

8.5 Diagrammes de séquence

Six scénarios méritent un diagramme de séquence. Le déroulement de chacun est décrit ci-dessous pour vous permettre de les tracer fidèlement.

Envoi d'un message 1:1

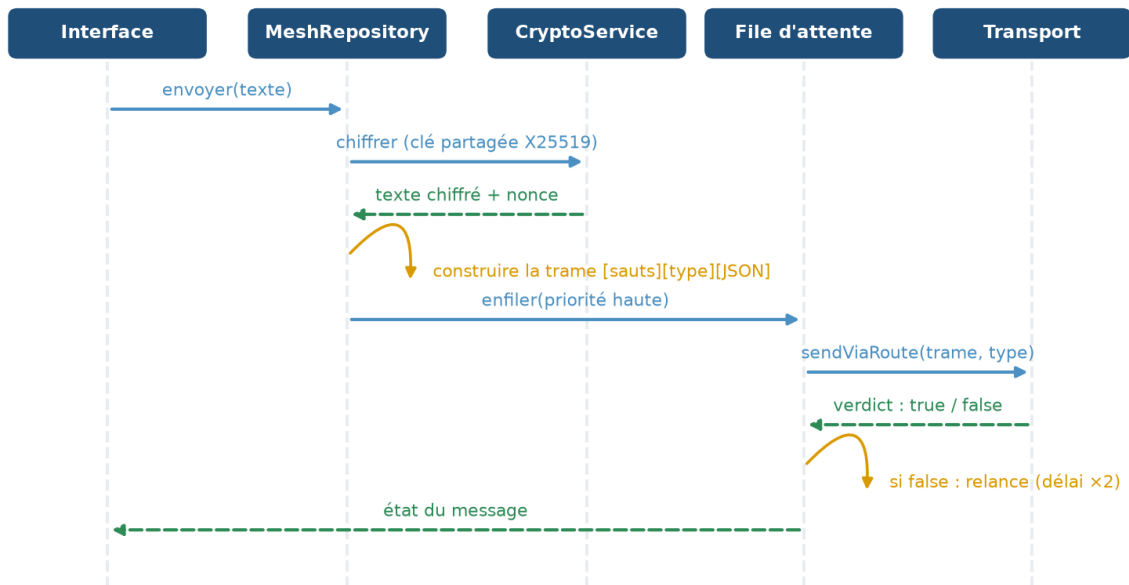


Figure 9 — Séquence d'envoi d'un message chiffré, du geste de l'utilisateur au verdict du transport

7. L'utilisateur saisit le texte et valide.
8. MeshRepository chiffre le contenu avec la clé partagée dérivée de l'échange X25519 (CryptoService.encryptBytes).
9. L'enveloppe JSON est construite puis préfixée de deux octets : compteur de sauts et type de contenu.
10. La trame est confiée à PremiumMessageQueue.
11. La file appelle MeshTransportService.sendViaRoute, qui choisit le transport le plus sain.
12. Le transport renvoie un booléen : la file marque le message livré, ou programme une relance avec délai exponentiel.

Réception

13. Le transport reçoit une trame et émet un événement incomingData.
14. MeshRepository vérifie l'identifiant contre la table de déduplication.
15. Si le message m'est destiné, il est déchiffré et enregistré.
16. Sinon, si le compteur de sauts le permet, il est relayé.

Relais

17. Le compteur de sauts est décrémenté.
18. L'identifiant est ajouté au filtre de déduplication.
19. La trame est rediffusée à tous les voisins sauf l'émetteur.

La figure 9 (chapitre 8.5) illustre la séquence d'envoi. Les diagrammes de la création de groupe, de l'échange de clés et de l'appel restent à tracer selon le même modèle.

8.6 Diagrammes d'activité

Le processus le plus instructif est celui de l'envoi avec relances : il comporte une boucle bornée à cinq tentatives, avec un délai qui double à chaque essai. C'est également dans ce diagramme que se lit la correction décrite en 17.4 — le losange « Verdict du transport » n'existait pas : le chemin menait directement à « Marquer livré ».

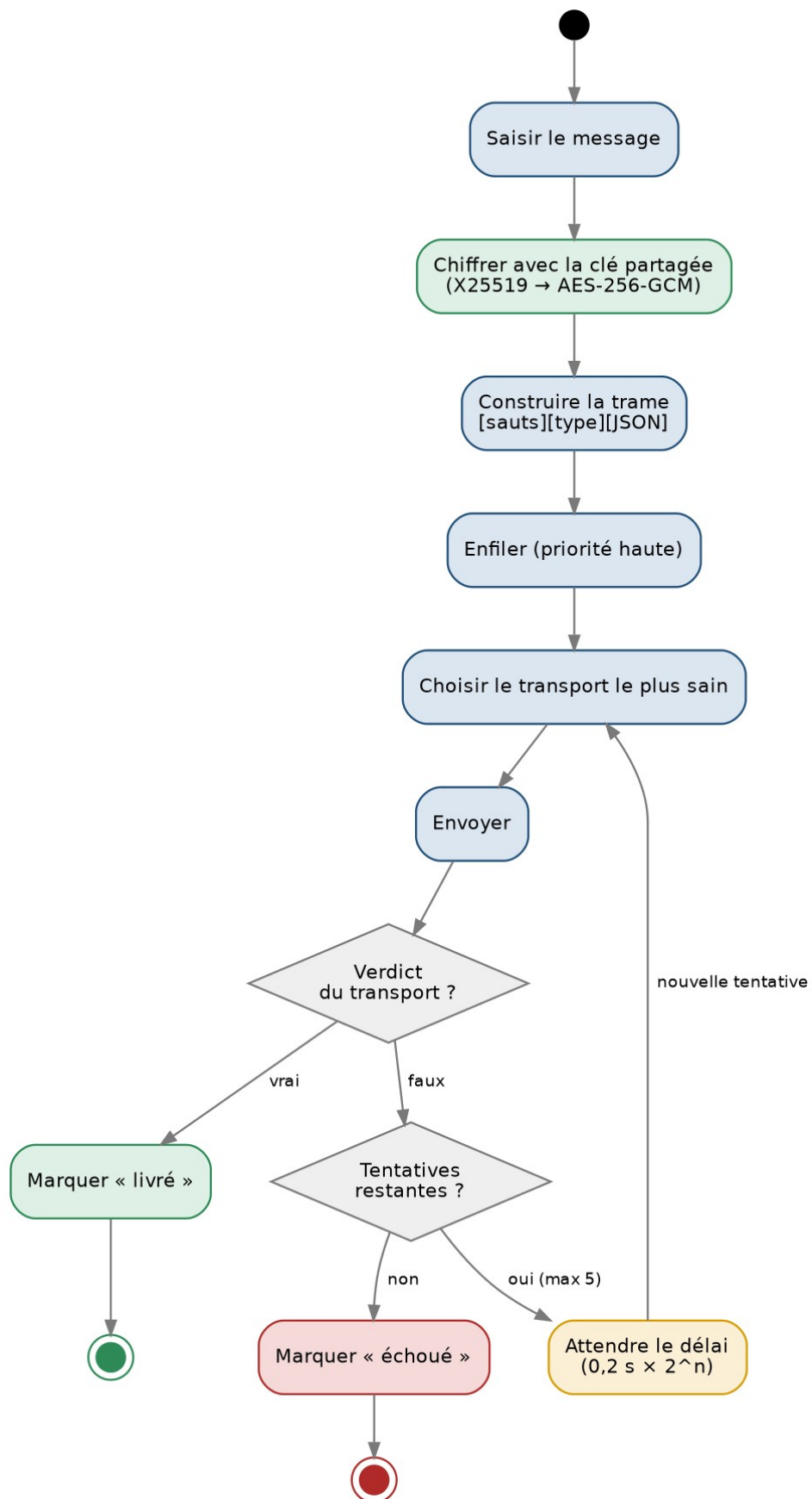


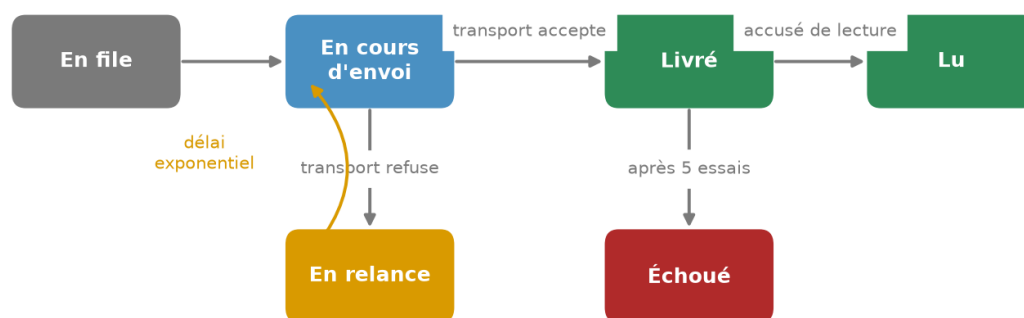
Figure 22 — Diagramme d'activité : envoi d'un message avec relances

8.7 Diagramme d'état

Un message traverse les états suivants :

État	Signification	Transition suivante
En file	Enfilé, en attente d'un créneau d'envoi	→ En cours d'envoi
En cours d'envoi	Confié à un transport	→ Livré ou → En relance
En relance	Le transport a refusé ; délai exponentiel en cours	→ En cours d'envoi, ou → Échoué après 5 tentatives
Livré	Un transport a accepté la charge	État terminal
Lu	Accusé de lecture reçu du destinataire	État terminal
Échoué	Cinq tentatives épuisées	État terminal

La distinction entre « Livré » et « Lu » est importante et a été au cœur d'une correction majeure : le passage à l'état « Livré » ne doit se produire que si un transport a RÉELLEMENT accepté la charge. Ce n'était pas le cas initialement (voir section 17.4).



Point critique : le passage à « Livré » ne doit se produire que si un transport a RÉELLEMENT accepté la charge. Ce n'était pas le cas initialement (voir § 17.4).

Figure 4 — Diagramme d'état d'un message

8.8 Diagramme de composants

Le diagramme de composants ci-dessous reprend la décomposition en couches de la section 9.2, en faisant apparaître les dépendances réelles entre modules.

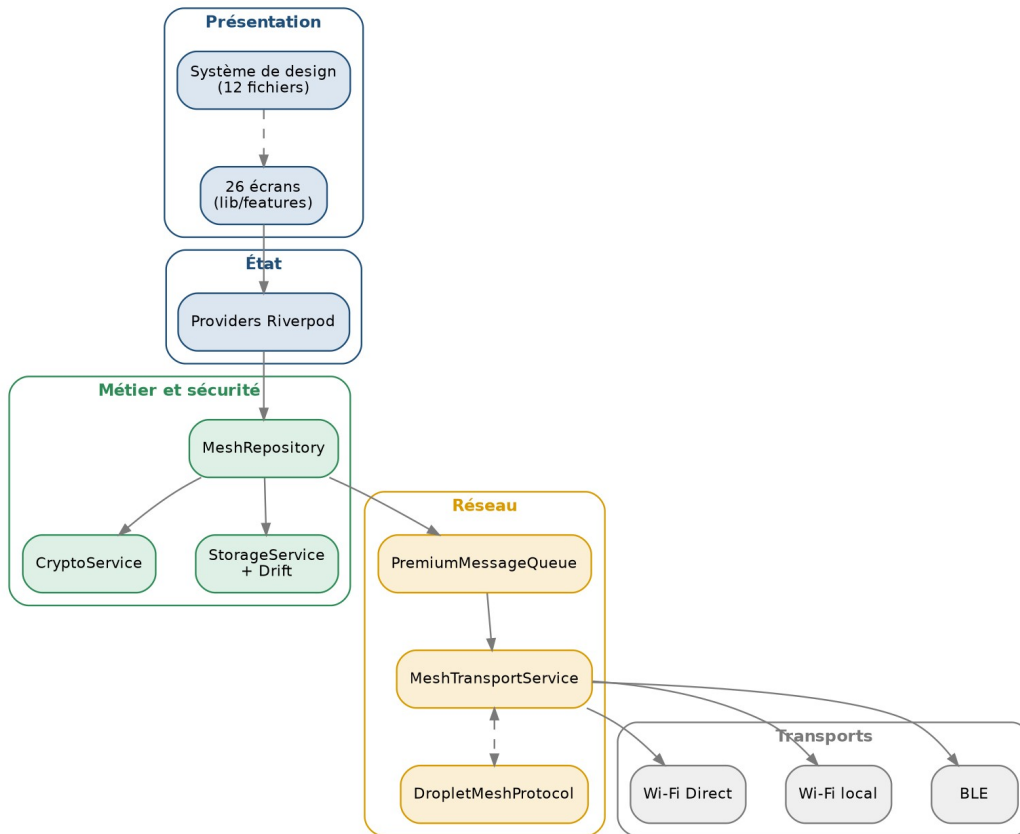


Figure 23 — Diagramme de composants

8.9 Diagramme de déploiement

Le diagramme de déploiement fait ressortir la caractéristique essentielle du modèle décentralisé : il n'existe aucun nœud serveur à représenter. Chaque appareil exécute l'intégralité de la pile logicielle et détient sa propre base de données.

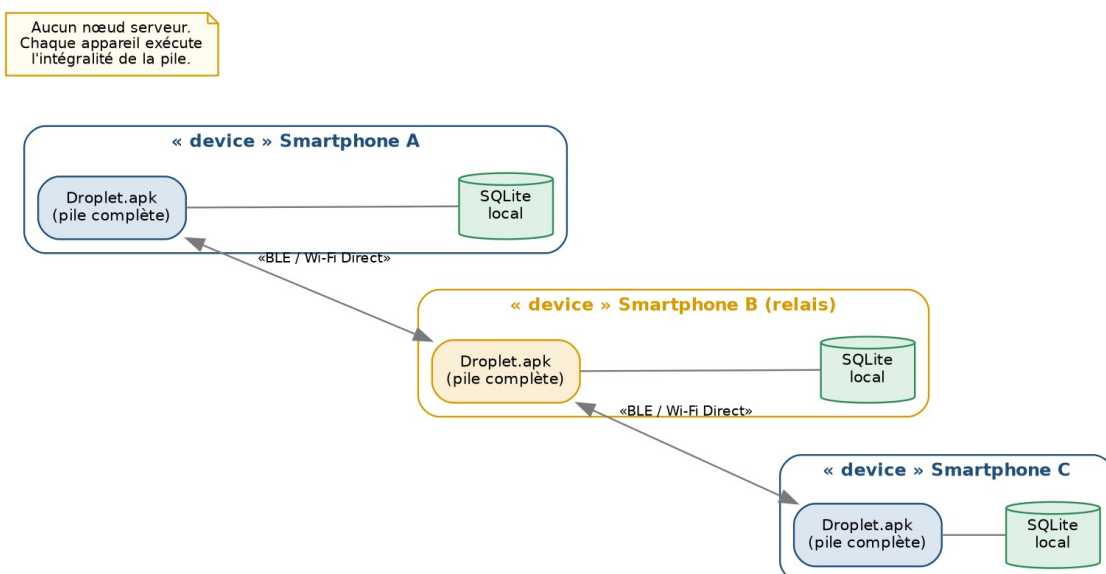


Figure 24 — Diagramme de déploiement

CHAPITRE 9 — ARCHITECTURE GÉNÉRALE

9.1 Vue globale

Droplet suit une organisation « feature-first » : le code est réparti entre un noyau technique partagé (lib/core), un système de design (lib/design_system), des composants réutilisables (lib/shared) et des dossiers fonctionnels indépendants (lib/features). Cette organisation représente 45 185 lignes de Dart réparties en 93 fichiers.

Dossier	Contenu	Rôle
lib/core/models	7 fichiers	Structures de données du domaine
lib/core/database	Schéma Drift	Définition des 8 tables
lib/core/services	17 services	Transports, crypto, stockage, notifications, média
lib/core/network	2 fichiers	File d'envoi fiable, gestionnaire de réseau
lib/core/protocol	1 fichier	Protocole Mesh : routage, congestion, déduplication
lib/core/providers	3 fichiers	État applicatif Riverpod
lib/core/repositories	1 fichier	Orchestration métier
lib/design_system	12 fichiers	Couleurs, typographie, composants
lib/features	16 dossiers	26 écrans applicatifs

9.2 Architecture en couches

L'architecture de Droplet peut être décrite selon huit couches, de la plus haute à la plus basse :

Une couche basse ne connaît jamais une couche haute

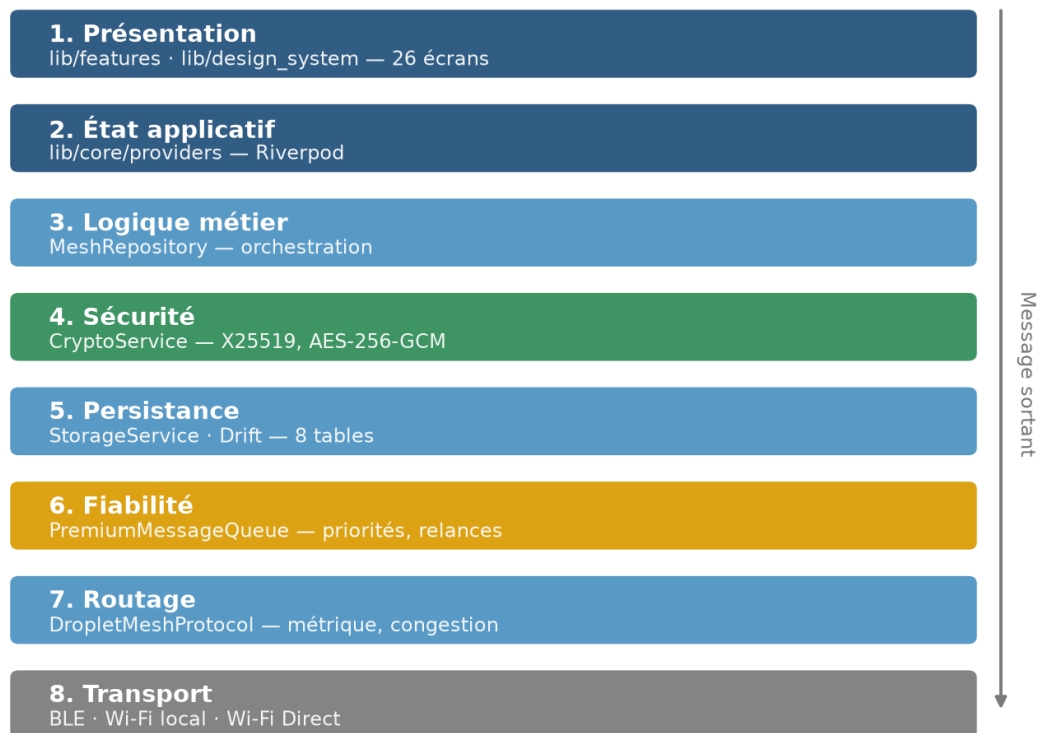


Figure 1 — Architecture en couches de Droplet

- 20. Présentation — écrans et widgets (lib/features, lib/design_system).
- 21. État applicatif — providers Riverpod, qui exposent à l'interface une vue réactive du réseau et des données.
- 22. Logique métier — MeshRepository, qui orchestre l'envoi, la réception, le relais et la persistance.
- 23. Sécurité — CryptoService : dérivation de clés, chiffrement, déchiffrement, numéros de sécurité.
- 24. Persistance — StorageService et la base Drift.
- 25. Fiabilité — PremiumMessageQueue : priorités, relances, mesures.
- 26. Routage — DropletMeshProtocol : table de routage, contrôle de congestion, déduplication.
- 27. Transport — MeshTransportService et les trois transports concrets (BLE, Wi-Fi local, Nearby).

Le principe de séparation est strict dans un sens : une couche basse ne connaît jamais une couche haute. Le transport ignore ce qu'est un message ; il ne voit que des octets et un type. C'est ce qui a permis d'ajouter des types de contenu (statuts, positions, réactions) sans toucher une seule ligne des transports.

9.3 Architecture Flutter

L'application est organisée autour d'une coquille (HomeShell) qui porte la barre d'onglets et quatre écrans principaux conservés en mémoire par un IndexedStack : Discussions, Actus, Appels et Pairs. Conserver l'état de chaque onglet évite de reconstruire une liste de conversations à chaque changement d'onglet, et préserve la position de défilement.

Les écrans secondaires sont poussés par-dessus cette coquille selon deux modes distincts : une navigation hiérarchique (la page glisse depuis la droite, avec geste de retour au bord de l'écran) pour ce qui constitue une descente d'un cran, et une présentation modale (montée depuis le bas) pour les parenthèses comme la composition d'un statut.

9.4 Architecture Riverpod

Les providers principaux sont : meshRepositoryProvider (accès au référentiel), meshPeerListProvider (liste réactive des pairs joignables), conversationMessagesProvider et groupMessagesProvider (messages filtrés par conversation), typingPeersProvider (indicateurs de saisie), callProvider et groupCallProvider (état des appels), appearanceProvider (mode clair/sombre) et deviceUnderThermalStressProvider (état thermique de l'appareil).

9.5 Flux de données

Le flux d'un message sortant traverse les couches de haut en bas : interface → provider → MeshRepository → chiffrement → file d'attente → protocole de routage → transport → radio. Le flux entrant emprunte le chemin inverse, mais avec deux embranchements : à la réception, le message peut être destiné à l'utilisateur (il est alors déchiffré et persisté) ou à un tiers (il est alors relayé sans jamais être déchiffré, ce qui est la garantie même du chiffrement de bout en bout dans un réseau à relais).

9.6 Communication entre modules

Les modules communiquent par flux d'événements (Stream) plutôt que par appels directs, ce qui évite les dépendances circulaires. MeshTransportService expose trois flux : peerEvents, incomingData et healthEvents. MeshRepository expose une douzaine de flux typés (newMessageEvents, ackEvents, readEvents, reactionEvents, statusEvents...) auxquels l'interface s'abonne.

9.7 Gestion des erreurs

La stratégie retenue distingue trois catégories d'erreurs :

- Les erreurs RÉCUPÉRABLES (échec d'envoi ponctuel) sont traitées par la file d'attente, qui relance avec un délai croissant.

- Les erreurs de DONNÉES (fichier tronqué, message indéchiffrable) sont rattrapées localement et signalées à l'utilisateur sans interrompre l'application. Un message vocal arrivé incomplet doit rester un non-événement, pas un plantage.
- Les erreurs de CONFIGURATION (permission refusée, radio désactivée) sont remontées à l'interface sous forme d'explication actionnable.

9.8 Gestion de la reconnexion

Trois mécanismes complémentaires assurent la continuité :

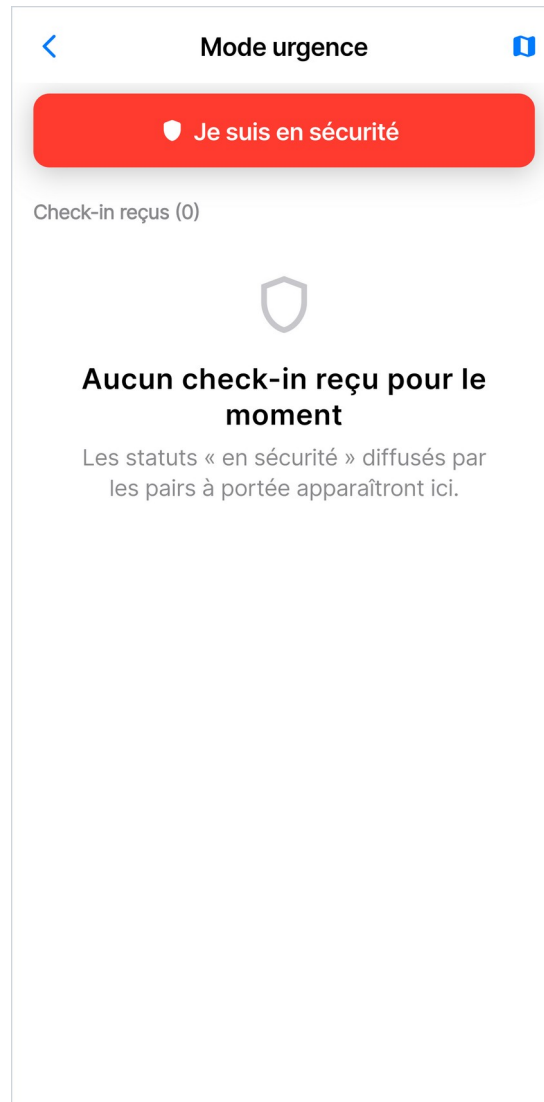


Figure 19 — Mode urgence : diffusion d'un statut « je suis en sécurité » et réception des check-in des pairs à portée

- Redémarrage automatique d'un transport tombé, avec cinq tentatives et un délai doublant à chaque essai. Ce mécanisme corrige un défaut observé : après un unique dépassement de délai, le transport BLE restait définitivement inactif.
- Retentative de connexion par pair, avec un délai propre à chaque pair plafonné à deux minutes, pour éviter de harceler un appareil hors de portée.

- Rediffusion des annonces (statuts, alertes) à chaque nouvelle rencontre de pair : une annonce publiée alors qu'on était seul serait autrement perdue définitivement.

CHAPITRE 10 — CONCEPTION DE LA BASE DE DONNÉES

10.1 Besoins de persistance

L'application doit conserver localement l'identité de l'utilisateur, les contacts, les conversations, les groupes et les paramètres, sans dépendre d'un serveur distant. Une exigence particulière découle du modèle Mesh : il faut également mémoriser les identifiants des messages déjà vus, faute de quoi un même message revenant par un autre chemin serait affiché en double et relayé indéfiniment.

10.2 Choix de SQLite

SQLite a été retenu pour sa légèreté, sa fiabilité et son intégration native aux plateformes mobiles.

10.3 Choix de Drift

Drift apporte un typage fort, une génération de code et une gestion simplifiée des migrations, réduisant les risques d'erreurs par rapport à des requêtes SQL manuelles.

10.4 Modèle conceptuel

Le modèle conceptuel fait apparaître cinq entités et cinq associations. L'association réflexive « répond à » sur MESSAGE modélise les réponses citées ; l'association « est membre » porte les cardinalités qui autorisent un pair à appartenir à plusieurs groupes.

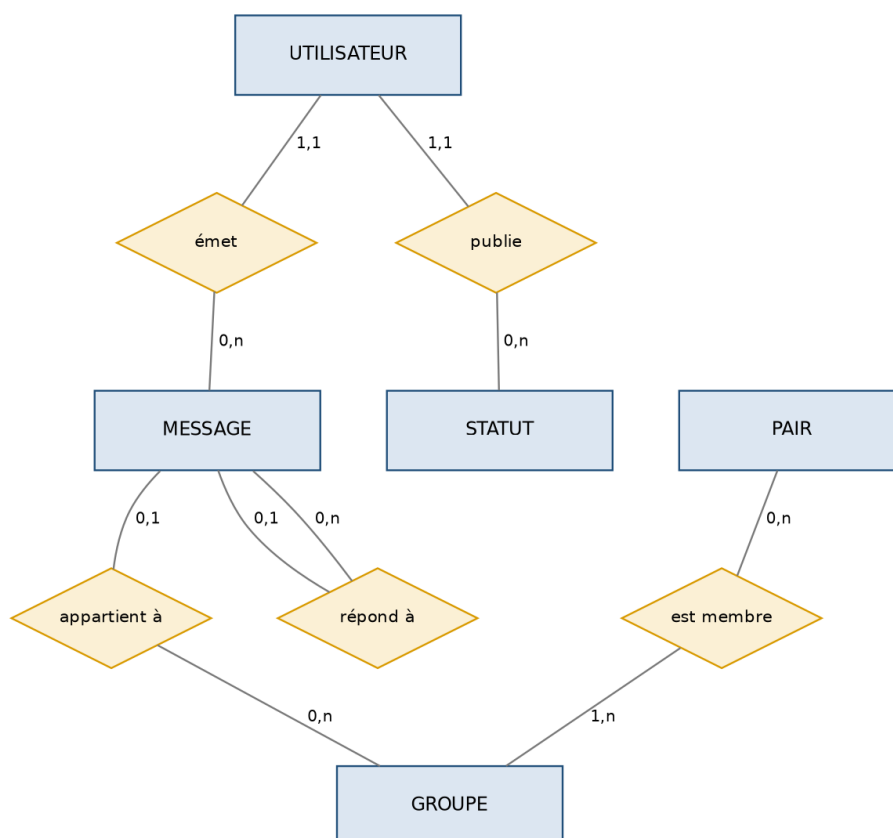


Figure 25 — Modèle conceptuel de données (MCD)

10.5 Modèle logique

Le passage au modèle logique applique les règles usuelles : chaque entité devient une relation, et chaque association de cardinalité (0,n)-(1,n) devient une relation porteuse des deux clés. C'est ce qui produit GroupMembers, dont la clé primaire est le couple (groupId, peerId). Les associations (0,1)-(0,n) se traduisent par une clé étrangère dans la relation du côté « n » : c'est le cas de groupId et de replyToId dans MeshMessages.

10.6 Modèle physique

Le schéma ci-dessous est celui réellement déclaré dans lib/core/database/app_database.dart, colonne par colonne, avec les types Drift correspondants. Les clés primaires sont signalées par une clé, les clés étrangères par une flèche.

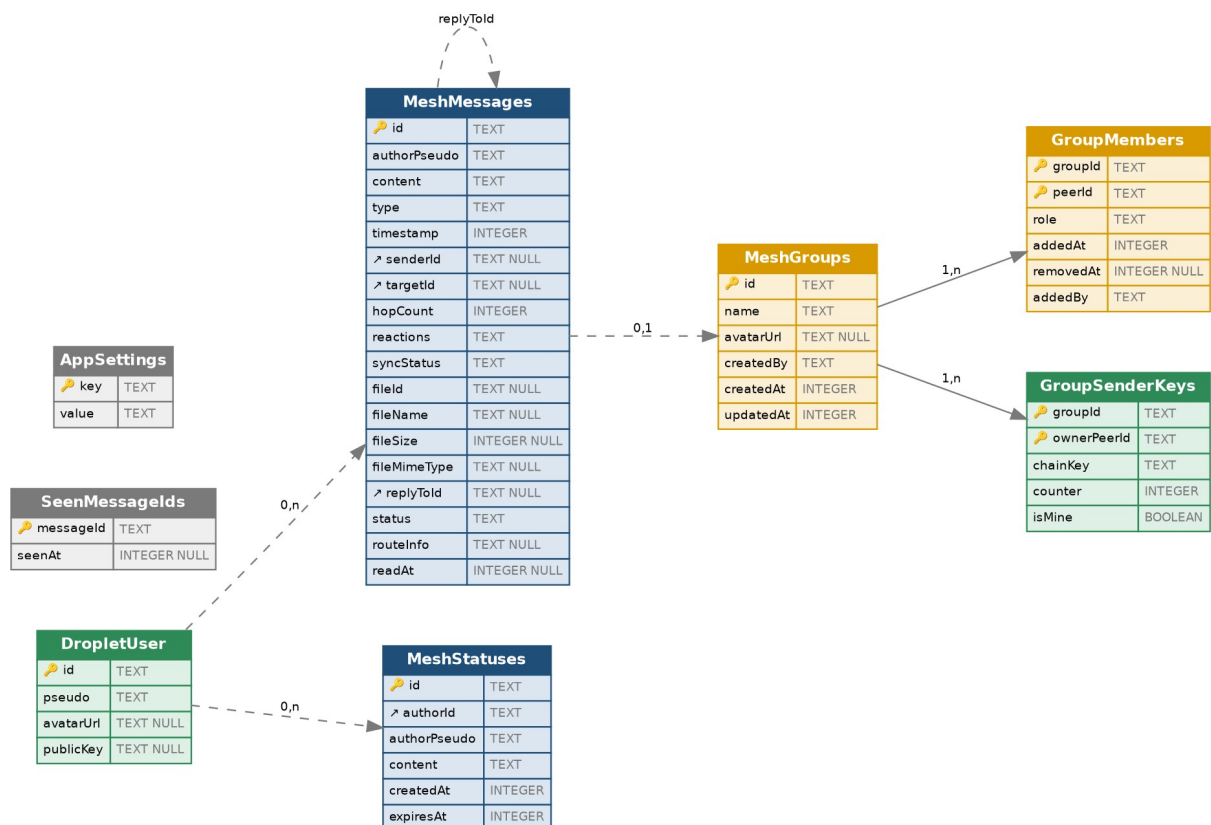


Figure 26 — Schéma physique : les huit tables telles qu'implémentées avec Drift

10.7 Tables

Table	Rôle	Particularité
DropletUser	Identité locale de l'utilisateur	Une seule ligne ; la clé privée n'y figure PAS
MeshMessages	Messages échangés et relayés	Table principale ; porte type, cible, réactions, pièce jointe
MeshGroups	Groupes de discussion	Nom, administrateurs, date
GroupMembers	Membres des groupes	Permet l'historique : un membre

		retiré reste tracé
GroupSenderKeys	Clés d'envoi pour le chiffrement des groupes	Une chaîne de clés par membre et par groupe
MeshStatuses	Statuts éphémères publiés	Purge après 24 heures
SeenMessageIds	Identifiants des messages déjà vus	Indispensable à la déduplication du relais
AppSettings	Paramètres de l'application	Stockage clé/valeur générique

Tableau 5 — Tables de la base de données

Commentaire des tables

Le schéma comporte huit tables, dont trois seulement portent du contenu visible par l'utilisateur. Les cinq autres existent pour des raisons purement techniques, et ce sont elles qui révèlent les contraintes du modèle décentralisé. Le détail ci-dessous suit cet ordre d'importance plutôt que l'ordre alphabétique.

DropletUser — une table à une seule ligne

Cette table ne contient jamais qu'un enregistrement : moi. Elle stocke l'identifiant public (l'empreinte SHA-256 générée à la création), le pseudonyme et la clé PUBLIQUE. La clé privée n'y figure pas, et c'est une décision de sécurité délibérée : elle vit dans le stockage sécurisé du système d'exploitation. La conséquence pratique est qu'une copie de la base de données, même intégrale, ne permet pas de se faire passer pour son propriétaire.

MeshMessages — la table centrale, et la plus large

Avec dix-neuf colonnes, c'est la table la plus riche du schéma. Elle mélange trois natures d'information qu'il serait tentant de séparer : le contenu (content, type), les métadonnées d'affichage (authorPseudo, reactions, replyToId) et les métadonnées de TRANSPORT (hopCount, syncStatus, status, routeInfo, readAt).

Ce mélange est assumé. Dans une application client-serveur, l'état d'acheminement d'un message n'a pas à être conservé : le serveur en est le témoin. Ici, il n'existe aucun témoin extérieur — l'appareil est le seul à savoir si un message est parti, s'il a été relayé et par quel chemin. Ces colonnes ne sont donc pas des détails techniques ajoutés à un message : elles font partie de ce qu'est un message dans un réseau maillé.

Deux colonnes portent des relations : replyToId pointe vers un autre message de la même table (relation réflexive, pour les réponses citées), et targetId désigne le destinataire — nul pour un message de diffusion.

MeshGroups et GroupMembers — la convergence sans arbitre

MeshGroups est une table simple. GroupMembers, en revanche, porte la décision de conception la plus subtile du schéma : ELLE N'EFFACE JAMAIS UNE LIGNE. Quand un membre quitte un groupe, la ligne est conservée et une date est inscrite dans removedAt. Cette technique porte un nom : le « tombstone », littéralement la pierre tombale.

La raison tient à l'absence de serveur. Imaginons deux appareils qui se retrouvent après une séparation : l'un pense qu'Amina est membre, l'autre qu'elle est partie. Sans autorité pour trancher, le seul arbitre possible est la date : celui qui porte l'information la plus récente a raison. C'est le principe dit « Last Write Wins ». Or si l'appareil qui a enregistré le départ avait EFFACÉ la ligne, il n'aurait plus rien à comparer — et l'ajout, plus ancien mais toujours présent chez l'autre, l'emporterait à tort. Effacer, ici, reviendrait à perdre l'argument.

GroupSenderKeys — l'état cryptographique des groupes

Une ligne par membre et par groupe, contenant la clé de chaîne courante (chainKey), un compteur, et un drapeau isMine indiquant s'il s'agit de ma propre chaîne d'envoi ou de celle d'un autre membre, servant à le déchiffrer.

Le compteur mérite une explication, car il résout un problème propre au réseau maillé. La clé avance à chaque message : le cinquième message d'Amina s'ouvre avec la cinquième clé de sa chaîne. Mais dans un réseau à relais, les messages n'arrivent pas dans l'ordre — le sixième peut précéder le cinquième. Le compteur transporté avec chaque message permet de savoir de combien il faut faire avancer la chaîne, et de conserver au passage les clés « sautées » pour déchiffrer les retardataires.

SeenMessageIds — la table qui empêche l'effondrement

Deux colonnes seulement, et pourtant la table sans laquelle le réseau ne fonctionnerait pas du tout. Chaque message relayé y laisse son identifiant.

Sans elle, la diffusion produirait une tempête : A envoie à B et C ; B renvoie à A et C ; C renvoie à A et B ; et ainsi de suite jusqu'à saturation des trois appareils. Le compteur de sauts limite la portée mais ne suffit pas — il faut aussi que chaque nœud REFUSE de traiter deux fois le même message. C'est le rôle de cette table, doublée en mémoire d'un filtre de Bloom qui évite un accès disque à chaque paquet reçu.

MeshStatuses et AppSettings

MeshStatuses conserve les statuts éphémères, avec une date d'expiration qui déclenche leur purge après vingt-quatre heures. AppSettings est un simple magasin clé/valeur, utilisé pour les réglages qui ne méritent pas leur propre table — la liste des conversations archivées, par exemple.

Ce que le schéma ne contient pas

Deux absences sont volontaires et méritent d'être signalées. Il n'existe AUCUNE table de contacts : les pairs connus sont persistés séparément, hors de la base applicative, car ils relèvent de l'état du réseau plutôt que des données de l'utilisateur. Et les FICHIERS ne sont pas stockés en base : seules leurs métadonnées le sont (fileId, fileName, fileSize), le contenu vivant dans le dossier privé de l'application. Placer des photos de plusieurs mégaoctets dans SQLite dégraderait toutes les requêtes de la table la plus sollicitée.

10.8 Relations

Les relations sont les suivantes. Un MeshGroups possède 1..n GroupMembers et 1..n GroupSenderKeys — une chaîne de clés par membre. Un MeshMessages référence 0..1 MeshGroups (par groupId) et 0..1 MeshMessages (par replyToId, relation réflexive). Un DropletUser émet 0..n MeshMessages et 0..n MeshStatuses.

Une particularité mérite d'être signalée : GroupMembers n'efface jamais une ligne. Le départ d'un membre est enregistré par une date dans removedAt — technique dite du « tombstone ». Ce choix n'est pas gratuit : sans serveur central pour arbitrer, deux appareils en désaccord sur la composition d'un groupe ne peuvent converger qu'en comparant des dates. Effacer la ligne supprimerait l'information nécessaire à cet arbitrage.

10.9 Index

Les clés primaires génèrent automatiquement un index. La requête la plus critique est la vérification d'existence dans SeenMessageIds, exécutée à chaque paquet reçu : elle bénéficie de l'index de la clé primaire messageId, et elle est de surcroît précédée d'un filtre de Bloom en mémoire qui évite l'accès disque dans la grande majorité des cas.

Deux index supplémentaires seraient justifiés sur MeshMessages : un index composite (targetId, timestamp) pour l'affichage d'une conversation, et un index sur groupId pour les conversations de groupe. Ils n'ont pas été ajoutés à ce stade, le volume de messages en test restant modeste ; c'est une optimisation à prévoir avant tout usage prolongé.

10.10 Déduplication

La table SeenMessageIds évite qu'un même message, reçu plusieurs fois via différents chemins de relais, ne soit traité ou affiché en double. Elle est doublée, en mémoire, d'un filtre de Bloom dimensionné pour 50 000 entrées avec un taux de faux positifs de 0,1 %. Ce filtre permet de rejeter la grande majorité des doublons sans toucher au disque : seule une réponse « peut-être déjà vu » déclenche une vraie lecture en base.

10.11 Performance

Trois mesures principales garantissent les performances :

- Le filtre de Bloom évite un accès disque par paquet reçu.
- Le cache de tuiles cartographiques écrit par lots, vidés une fois par seconde dans une transaction unique, avec le journal en mode WAL. Cette optimisation corrige une saccade visible : les écritures unitaires se produisaient sur le fil d'affichage.

- Les listes de messages sont construites paresseusement (`ListView.builder`) : seuls les éléments visibles existent en mémoire.

CHAPITRE 11 — CONCEPTION DU RÉSEAU MESH

11.1 Définition

Un réseau maillé (Mesh) est un réseau dans lequel chaque nœud — ici, chaque appareil — peut communiquer directement avec ses voisins et relayer le trafic destiné à d'autres nœuds, sans dépendre d'un point central.

11.2 Architecture Mesh

Droplet retient une topologie plate à rôles différenciés. Chaque nœud reçoit dynamiquement un rôle parmi trois :

Rôle	Comportement
leaf (feuille)	Nœud standard : émet, reçoit, relaie ses voisins directs
gateway (passerelle)	Relaie activement pour d'autres, annonce sa disponibilité au voisinage
superPeer	Passerelle disposant de ressources supérieures (batterie, connectivité)

Ce choix évite l'écueil d'une topologie hiérarchique figée, inadaptée à un réseau dont les nœuds sont des téléphones qui vont et viennent.

11.3 Découverte des pairs

La découverte consiste à détecter en continu les appareils Droplet à proximité, via BLE et/ou Wi-Fi, afin d'établir des liaisons directes. Le rythme des balises est ADAPTATIF : il se resserre quand des pairs sont proches et que l'application est au premier plan, et s'espace quand la batterie faiblit ou que l'application passe en arrière-plan.

11.4 BLE

Le BLE est utilisé pour la découverte à faible consommation et l'échange de petits messages sur de courtes distances. Chaque appareil joue simultanément les deux rôles GATT : périphérique (il expose des caractéristiques lisibles annonçant sa présence) et central (il scanne et se connecte aux périphériques découverts).

11.5 Wi-Fi local

Le Wi-Fi local est utilisé lorsque des débits plus importants sont nécessaires (fichiers, flux audio/vidéo). La découverte se fait par balises UDP diffusées sur le réseau, et les échanges par sockets TCP avec préfixe de longueur.

11.6 P2P

La communication P2P via Wi-Fi Direct (Nearby Connections) permet l'échange direct entre deux appareils sans point d'accès intermédiaire — le cas le plus favorable en l'absence totale d'infrastructure.

11.7 Routage

Le routage repose sur une table de routage alimentée par une MÉTRIQUE COMPOSITE, calculée pour chaque lien selon la formule suivante :

$$\begin{aligned} \text{métrique} = & 0,4 \times \text{latence_normalisée} \\ & + 0,4 \times (1 - \text{taux_de_livraison}) \\ & + 0,2 \times (1 - \text{niveau_de_batterie}) \end{aligned}$$

Une métrique BASSE désigne un bon lien. Les pondérations traduisent une décision de conception : la latence et la fiabilité pèsent également lourd (0,4 chacune), tandis que la batterie du pair intervient pour moitié moins (0,2). Ce dernier terme est ce qui distingue un réseau de téléphones d'un réseau d'équipements fixes : router systématiquement par l'appareil le plus rapide viderait sa batterie et ferait disparaître le meilleur relais du maillage.

11.8 Multi-hop

Le multi-hop désigne la capacité d'un message à transiter par plusieurs appareils intermédiaires avant d'atteindre son destinataire final, étendant ainsi la portée effective du réseau au-delà de la portée radio d'un seul appareil.

11.9 Store-and-forward

Le mécanisme de store-and-forward permet à un appareil relais de conserver temporairement un message lorsque le destinataire final n'est pas immédiatement joignable, puis de le transmettre dès qu'un chemin devient disponible. Ce mécanisme est complété par une rediffusion des annonces à chaque nouvelle rencontre.

11.10 Hop Count

Le compteur de sauts limite le nombre de relais successifs qu'un message peut emprunter, afin d'éviter une propagation infinie. Il est porté par le PREMIER OCTET de chaque trame, décrémenté à chaque relais, et le message est abandonné lorsqu'il atteint zéro. La limite du protocole est fixée à 10 sauts.

11.11 Déduplication

Chaque message porte un identifiant unique permettant à chaque nœud de détecter et d'ignorer les copies déjà traitées. Sans ce mécanisme, la diffusion produirait une tempête de rediffusions croissante : chaque nœud renverrait à ses voisins un message qu'ils lui renverraient à leur tour.

11.12 Expiration des messages

Deux politiques d'expiration coexistent : les statuts sont purgés après 24 heures, et les identifiants de déduplication sont nettoyés périodiquement pour éviter une croissance sans fin de la table.

À COMPLÉTER PAR VOUS — Précisez la durée de rétention exacte des messages en attente de relais telle qu'implémentée dans votre version.

11.13 Gestion des déconnexions

La perte d'un lien direct est détectée par l'absence prolongée de signe de présence. Le pair est alors retiré de la liste des joignables, mais conservé dans la table persistante des pairs connus, avec sa clé publique — ce qui permet de reprendre la conversation chiffrée sans nouvel échange de clés lors de la prochaine rencontre.

11.14 Exemple pratique

Un message envoyé par l'appareil A, hors de portée directe de l'appareil D, peut néanmoins être délivré en transitant successivement par les appareils B puis C, selon le schéma $A \rightarrow B \rightarrow C \rightarrow D$. Chaque saut décrémente le compteur, et chaque nœud intermédiaire mémorise l'identifiant du message afin de ne pas le retransmettre une seconde fois. B et C ne peuvent PAS lire le contenu : il est chiffré pour D seul.

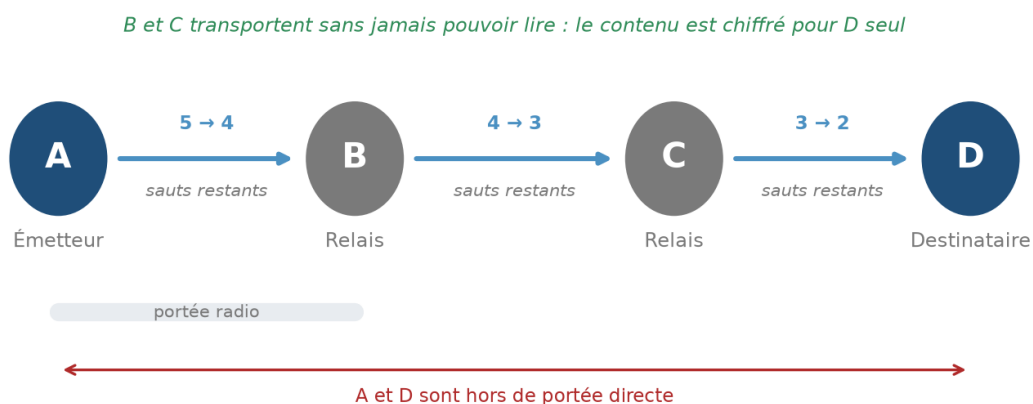


Figure 2 — Principe du relais multi-hop : le message franchit trois sauts, et chaque relais décrémente le compteur

11.15 Limites physiques

La portée est estimée entre 10 et 50 mètres par saut selon l'environnement et la technologie, ce qui conditionne directement la densité d'appareils nécessaire pour assurer une couverture continue. C'est la limite fondamentale du modèle : un réseau maillé ne crée pas de la portée, il l'emprunte à ses membres.

11.16 Capacité réelle des transports

L'expérimentation a produit une donnée qui ne figurait pas dans le cahier des charges et qui structure pourtant tout l'usage : les trois transports n'ont pas des capacités comparables.

Constante	Valeur	Conséquence
Écriture BLE (kMaxBleWriteSize)	20 octets	19 octets utiles par écriture
Charge maximale BLE	512 octets	Au-delà, la trame est refusée
Types refusés par BLE	$\geq 0x20$	Fichiers et flux audio exclus
Sauts maximum	10	Portée logique du maillage
Relances par message	5	Délais : 0,2 s, 0,4 s, 0,8 s, 1,6 s, 3,2 s
Débit entrant maximal	150 paquets/s par pair	Garde-fou anti-saturation

Tableau 6 — Constantes du protocole Mesh

La conséquence est directe : une photo de 250 Ko, découpée en fragments de 19 octets, représenterait environ 13 000 écritures Bluetooth. Le transport refuse donc les fichiers — décision juste, mais qui signifie que le partage de fichiers de Droplet dépend entièrement de la disponibilité d'une liaison Wi-Fi.

CHAPITRE 12 — SÉCURITÉ ET CHIFFREMENT

12.1 Menaces

Menace	Traitement dans Droplet
Interception passive du trafic radio	Chiffrement AES-256-GCM de bout en bout
Altération d'un message en transit	Authentification intégrée au mode GCM : toute modification est détectée au déchiffrement
Nœud relais malveillant	Le relais ne peut pas déchiffrer : il ne détient pas la clé partagée
Usurpation d'identité	Modèle TOFU + vérification hors bande par numéro de sécurité et QR code
Rejeu de messages	Identifiant unique + table de déduplication
Appareil compromis (volé, déverrouillé)	Non couvert — limite assumée, voir 12.15
Saturation volontaire du réseau	Limitation à 150 paquets/s par pair et éviction temporaire

12.2 Objectifs de sécurité

Les objectifs poursuivis sont la confidentialité des échanges, l'intégrité des messages, l'authenticité des interlocuteurs et la disponibilité du service de communication.

12.3 Identité cryptographique

Chaque utilisateur dispose d'une paire de clés générée localement à la création de son identité, sans validation par une autorité centrale. L'identifiant public de l'utilisateur est l'empreinte SHA-256 du pseudonyme et de l'horodatage de création, soit 64 caractères hexadécimaux.

12.4 X25519

X25519 est une fonction d'échange de clés basée sur la courbe elliptique Curve25519, reconnue pour son bon compromis entre sécurité et performance. Elle est utilisée pour établir le secret partagé entre deux correspondants.

12.5 ECDH

L'échange Diffie-Hellman sur courbes elliptiques permet à deux appareils d'établir un secret partagé à partir de leurs clés publiques respectives, sans jamais transmettre leurs clés privées. Concrètement, lorsqu'un pair est découvert, chacun lui envoie un message « hello » contenant sa clé publique ; le secret partagé est ensuite dérivé de part et d'autre et mis en cache.

12.6 HKDF

La dérivation de clé permet, à partir du secret partagé, de générer les clés symétriques effectivement utilisées. Elle est également employée dans le mécanisme de ratchet des groupes, où chaque message fait avancer une chaîne de clés par HMAC-SHA-256.

12.7 AES-256-GCM

AES-256-GCM est l'algorithme de chiffrement symétrique retenu ; le mode GCM apporte à la fois la confidentialité et l'authentification du message. Chaque chiffrement produit un nonce distinct, transmis en clair à côté du texte chiffré — le nonce n'est pas un secret, mais il ne doit jamais être réutilisé avec la même clé.

12.8 Chiffrement 1:1

Pour une conversation individuelle, une clé de session est dérivée de l'échange ECDH entre les deux interlocuteurs, garantissant que seuls eux peuvent déchiffrer les messages échangés. Les clés partagées sont mises en cache (512 entrées) afin d'éviter de recalculer une opération de courbe elliptique à chaque message.

12.9 Chiffrement des groupes

Le chiffrement de groupe pose un problème différent du 1:1 : chiffrer individuellement pour chaque membre multiplierait le trafic par le nombre de destinataires — inacceptable sur un réseau dont le débit se compte en centaines d'octets par seconde. Droplet emploie donc le mécanisme de Sender Keys.

12.10 Sender Keys

Chaque membre possède sa propre chaîne de clés d'envoi pour le groupe. Il chiffre son message UNE SEULE FOIS avec la clé courante de sa chaîne, puis fait avancer celle-ci. Les autres membres, qui connaissent l'état de sa chaîne, peuvent dériver la même clé et déchiffrer. Un compteur accompagne chaque message pour permettre de rattraper les messages manqués — situation fréquente dans un réseau à relais, où les messages n'arrivent pas nécessairement dans l'ordre.

12.11 Ratchet

Un mécanisme de ratchet est mis en œuvre pour les groupes : la clé de chaîne avance à chaque message par dérivation HMAC-SHA-256, de sorte qu'une clé compromise ne permet pas de déchiffrer les messages antérieurs. Les clés sautées sont conservées temporairement pour permettre le déchiffrement des messages arrivés dans le désordre.

À COMPLÉTER PAR VOUS — Précisez si un ratchet est également appliqué aux conversations 1:1 dans votre version, ou si celles-ci utilisent une clé de session stable.

12.12 Protection des clés privées

Les clés privées ne quittent jamais l'appareil et sont protégées via le stockage sécurisé natif de la plateforme. Elles ne figurent dans aucune table de la base applicative.

12.13 Trust On First Use

Le modèle TOFU consiste à faire confiance à la clé publique d'un contact lors de la première mise en relation, puis à alerter l'utilisateur en cas de changement ultérieur. L'application conserve, pour chaque pair, la clé au moment de la vérification, ce qui lui permet de distinguer trois états : non vérifié, vérifié, et — le plus important — « la clé a changé depuis la vérification ».

12.14 Vérification QR Code

La vérification hors bande est implémentée. Chaque contact dispose d'un numéro de sécurité calculé à partir des deux identités et des deux clés publiques, par 5 000 tours de SHA-256, présenté en 12 groupes de chiffres. Les deux utilisateurs peuvent comparer ce numéro visuellement, ou scanner le QR code de l'autre, ce qui effectue la comparaison automatiquement et marque le contact comme vérifié.

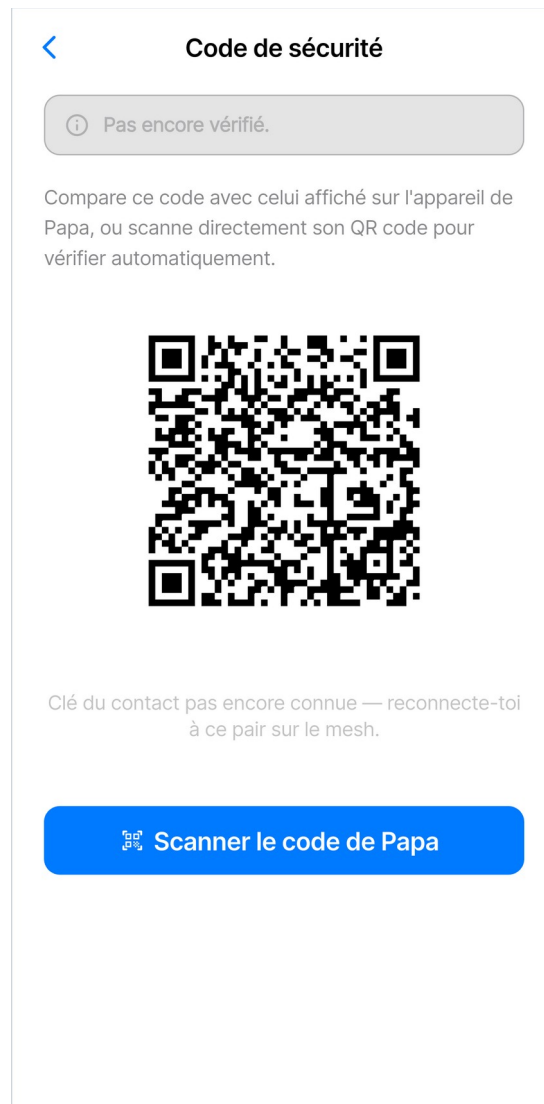


Figure 15 — Vérification d'identité : numéro de sécurité et QR code, avec l'état de vérification affiché en haut

12.15 Limites du modèle de sécurité

Le modèle retenu comporte des limites qu'il serait malhonnête de passer sous silence :

- Absence d'autorité de certification — une identité n'est vérifiée que par comparaison hors bande. Sans cette étape, rien ne garantit qu'un correspondant est bien celui qu'il prétend être.
- Appareil compromis — un téléphone déverrouillé donne accès aux messages déchiffrés et à la base locale. Le chiffrement de bout en bout protège le transport, pas le terminal.
- Métadonnées visibles — si le CONTENU est chiffré, les identifiants d'émetteur et de destinataire circulent en clair dans l'enveloppe, car les relais en ont besoin pour router. Un observateur peut donc établir qui parle à qui, sans savoir de quoi.
- Confiance initiale — le modèle TOFU accepte la première clé rencontrée. Une usurpation lors de la toute première mise en relation ne serait pas détectée.

PARTIE IV — RÉALISATION

CHAPITRE 13 — TRAVAUX RÉALISÉS PENDANT LE STAGE

Ce chapitre présente les réalisations selon la grille : Problème → Objectif → Solution → Implémentation → Difficulté → Test → Résultat. Chaque section décrit un travail effectivement mené et vérifié.

À COMPLÉTER PAR VOUS — IMPORTANT — *Ce chapitre est le cœur de votre évaluation. Les contenus ci-dessous décrivent le travail réalisé sur le projet ; ne conservez que ce que VOUS avez effectivement fait, et reformulez à la première personne. Si une réalisation ne vous revient pas, déplacez-la vers la section 2.6 sous « ce qui existait avant mon intervention ».*

13.1 Prise en main du projet

À COMPLÉTER PAR VOUS — *Décrivez votre découverte du projet : volume de code à assimiler (45 185 lignes, 93 fichiers), point d'entrée choisi pour comprendre l'architecture, premières difficultés de compréhension.*

13.2 Analyse du code existant

À COMPLÉTER PAR VOUS — *Décrivez votre méthode d'analyse. Une approche efficace consiste à suivre le trajet complet d'un message, de la saisie à la radio, en traversant les huit couches décrites en 9.2.*

13.3 Mise en place de l'environnement

Problème

Le développement d'une application Mesh impose des contraintes d'environnement inhabituelles : les fonctionnalités centrales (BLE, Wi-Fi Direct) ne fonctionnent PAS sur émulateur, et exigent un appareil physique. La boucle de rétroaction est donc plus lente que sur un projet classique.

Solution et implémentation

L'environnement retenu associe le SDK Flutter 3.41.6, Android Studio pour les outils natifs, et un appareil physique connecté en USB avec le débogage activé. La chaîne de vérification systématique après chaque modification est : flutter analyze, puis flutter test, puis flutter build apk --debug, puis installation et observation via logcat.

Résultat

Cette discipline s'est révélée décisive : la majorité des défauts les plus graves rencontrés durant le projet n'était visible NI par l'analyse statique, NI par les tests, mais uniquement dans les journaux de l'appareil. Le chapitre 18 en donne sept exemples.

13.4 Conception

À COMPLÉTER PAR VOUS — *Décrivez votre part dans les choix de conception (architecture en couches, modèle de données, protocole). Appuyez-vous sur les chapitres 9 à 12.*

13.5 Développement

À COMPLÉTER PAR VOUS — Décrivez votre méthode de développement au quotidien : itérations, gestion des branches, revue de code si applicable.

13.6 Interfaces

Problème

Une application de communication est jugée en quelques secondes sur son interface. L'objectif fixé était d'atteindre le niveau de finition des applications de messagerie de référence, sur un socle Flutter qui, par défaut, produit une interface d'apparence Material — donc étrangère aux conventions visuelles attendues.

Solution

Un système de design complet a été construit (12 fichiers dans lib/design_system), comprenant :

- une échelle typographique complète, avec les tailles, graisses, hauteurs de ligne et surtout les espacements de lettres — négatifs aux petites tailles, positifs aux grandes ;
- une palette adaptative gérant les modes clair et sombre, exposée par accesseurs statiques et basculée à la racine de l'application ;
- des composants réutilisables : listes groupées avec séparateurs décalés, feuilles modales à matériau translucide, alertes, indicateurs d'attente, boutons de barre ;
- une barre d'onglets flottante en capsule, avec indicateur animé par ressorts physiques.

Difficulté

Le basculement clair/sombre a révélé un piège subtil : les écrans étant créés par le routeur sous forme d'instances constantes, Flutter reconnaît le même objet d'une reconstruction à l'autre et saute leur reconstruction. Les couleurs, lues par accesseurs statiques, restaient donc figées sur l'ancien mode. La correction a consisté à insérer un nœud porteur d'une clé dépendant de la luminosité, ce qui force la reconstruction de tout l'arbre au changement de mode.

Résultat

L'application dispose de deux thèmes complets, le mode sombre restant le défaut et le mode clair étant configurable dans les paramètres.

13.7 Messagerie

Problème

L'écran de conversation concentre le plus grand nombre de fonctionnalités de l'application : texte, fichiers, images, vidéos, messages vocaux, positions, autocollants, réactions, réponses citées, accusés de réception.

Implémentation

L'écran représente à lui seul plus de 3 200 lignes. Les mécanismes notables sont :

- le glissement latéral d'une bulle pour y répondre ;
- le double appui pour réagir d'un cœur, avec animation ;
- un menu contextuel ancré sur le message : le fond se floute, le message choisi reste net et se soulève, les émojis se déploient en cascade au-dessus et les actions apparaissent en dessous ;
- l'enregistrement vocal par maintien du doigt, avec verrouillage par glissement vers le haut, annulation par glissement latéral, et affichage en direct de la forme d'onde ;
- une recherche dans la conversation qui ne filtre pas la liste mais navigue dedans, en surlignant les occurrences et en faisant clignoter le message atteint.

Difficulté

L'affichage en direct de la forme d'onde ne fonctionnait pas : la courbe restait plate. La cause était une erreur d'unité — la bibliothèque d'enregistrement renvoie une amplitude en décibels pleine échelle, donc une valeur NÉGATIVE, alors que le code la bornait entre 0 et 1, ce qui donnait invariablement zéro. La correction convertit l'échelle logarithmique en amplitude linéaire avec un plancher à -45 dB.

Résultat

L'ensemble des fonctionnalités de conversation est opérationnel. Un défaut fonctionnel a par ailleurs été découvert et corrigé : dans les conversations de GROUPE, aucun nom d'expéditeur n'était affiché — la condition ne couvrait que le canal de diffusion. Tous les messages reçus y arrivaient donc anonymes. Chaque participant dispose désormais d'une couleur stable, dérivée de son identifiant.

13.8 Réseau Mesh

Problème

Faire circuler des messages entre appareils sans infrastructure suppose de résoudre simultanément quatre problèmes : découvrir les voisins, choisir par où passer, éviter les boucles, et survivre aux coupures.

Implémentation

Le protocole (`lib/core/protocol/droplet_mesh_protocol.dart`) implémente :

- une table de routage alimentée par la métrique composite décrite en 11.7 ;

- un contrôle de congestion de type AIMD (augmentation additive, réduction multiplicative), avec une fenêtre bornée entre 1 et 64 ;
- un filtre de Bloom de 50 000 entrées pour la déduplication ;
- une file de sortie triée par priorité.

Difficulté

Deux défauts majeurs de robustesse ont été découverts en observant les journaux sur appareil réel :

28. Le transport Wi-Fi Direct produisait une exception BUSY environ une fois par seconde, en continu. La cause était auto-infligée : le code lançait une tentative de connexion vers chaque pair découvert, à chaque mise à jour de la liste, alors que l'API native ne traite qu'une connexion à la fois. La correction sérialise les tentatives et applique un délai croissant par pair, plafonné à deux minutes.
29. Le transport BLE restait définitivement inactif après un unique dépassement de délai au démarrage : l'indicateur d'exécution restait positionné malgré l'échec, empêchant toute nouvelle tentative. La correction remet l'indicateur à zéro en cas d'échec et relance jusqu'à cinq fois avec un délai doublant.

Résultat

Après correction, les journaux montrent 8 occurrences de BUSY en 30 secondes au lieu d'environ 30, et la trace « BLE démarré à la tentative 2 » confirme que le mécanisme de reprise fonctionne.

13.9 Base de données

Problème

Conserver localement l'ensemble des données sans serveur, avec des contraintes de performance : la vérification de déduplication est exécutée à chaque paquet reçu.

Implémentation

Huit tables Drift (voir tableau 5), doublées d'un filtre de Bloom en mémoire pour la déduplication. Le stockage des fichiers reçus se fait hors base, dans le dossier privé de l'application, la base ne conservant que les métadonnées.

Résultat

La persistance est validée par un test automatisé et par les scénarios de sauvegarde/restauration (chapitre 17).

13.10 Sécurité

Problème

Garantir que seul le destinataire puisse lire un message, dans un réseau où ce message transite en clair par des appareils tiers.

Implémentation

Voir chapitre 12 pour le détail. Les points d'implémentation notables sont le cache de clés partagées (512 entrées), le numéro de sécurité à 5 000 tours de SHA-256, et la sauvegarde chiffrée de l'identité protégée par mot de passe.

Difficulté

Un défaut de robustesse a été identifié : un message chiffré pouvait arriver AVANT que la clé publique de son émetteur ne soit connue, ce qui le rendait définitivement indéchiffrable et affichait « Message illisible ». Le chemin d'envoi émettait bien un « hello » préalable, mais le chemin de réception ne disposait d'aucun mécanisme équivalent. La correction ajoute une résolution de clé de repli, une file de déchiffrements en attente, et une reprise automatique dès que la clé arrive.

Résultat

Les messages reçus avant l'échange de clés sont désormais déchiffrés rétroactivement et affichés correctement.

13.11 Groupes

Problème

Chiffrer pour n destinataires sans multiplier le trafic par n.

Solution et implémentation

Mécanisme de Sender Keys avec ratchet HMAC-SHA-256, décrit en 12.10. Deux tables (GroupMembers, GroupSenderKeys) portent l'état cryptographique. Le compteur transporté avec chaque message permet de rattraper les messages manqués, situation normale dans un réseau à relais où l'ordre d'arrivée n'est pas garanti.

Résultat

La messagerie de groupe est opérationnelle, avec administration (renommage, ajout et retrait de membres) et appels de groupe limités à quatre participants.

13.12 Appels

Problème

Établir un appel audio sans serveur de signalisation ni serveur de média.

Solution

WebRTC assure le média en pair-à-pair ; la signalisation (offre, réponse, candidats) emprunte le réseau Mesh lui-même. Une contrainte a été identifiée et rendue visible

dans l'interface : le débit du Bluetooth est insuffisant pour la voix. Le bouton d'appel est donc grisé — et non masqué — lorsque le pair n'est joignable qu'en Bluetooth ou par relais, avec une explication au toucher plutôt qu'un bouton mort.

Résultat

Appels audio 1:1 fonctionnels en Wi-Fi, avec sonnerie, vibration et journal d'appels réel.

13.13 Tests

Problème

Comment valider un système dont la fonction principale exige deux appareils, lorsqu'on n'en dispose que d'un ?

Solution

Un banc d'essai à deux pairs a été conçu (`test/two_peer_session_test.dart`). Le correspondant simulé n'a de simulé QUE la radio : il possède sa propre paire de clés X25519, dérive le même secret partagé, et DÉCHIFFRE réellement ce qu'il reçoit. Tout le reste est le code de production : enveloppe, chiffrement, file d'attente, relances.

Implémentation

Deux profils de lien modélisent les transports réels, avec leur latence, leur taille de fragment et leur taux de perte. Le temps radio est CALCULÉ à partir de la taille réelle des trames — jamais mesuré, ce qui serait malhonnête — et présenté séparément du temps logiciel, lui réellement mesuré.

Résultat

Le banc a immédiatement révélé un défaut majeur de fiabilité, détaillé en section 17.4. La suite compte désormais 63 tests.

13.14 Débogage

Le tableau ci-dessous recense les défauts significatifs identifiés et corrigés, avec le moyen qui a permis de les découvrir. Il illustre un point méthodologique important : aucun n'a été détecté par l'analyse statique ou par les tests unitaires.

Défaut	Découvert par	Cause
Forme d'onde vocale plate	Observation visuelle	Amplitude en dBFS (négative) bornée entre 0 et 1
Micro laissé ouvert	Observation sur appareil	Visuels pilotés par le geste, pas par l'état d'enregistrement
« Message illisible »	Usage réel	Chemin de réception sans repli sur clé absente

Boucle BUSY à 1 Hz	logcat	Tentatives de connexion parallèles sur une API séquentielle
BLE définitivement inactif	logcat	Indicateur d'exécution non réinitialisé après échec
Cache de tuiles corrompu	Affichage de la carte	Serveur renvoyant une image d'erreur avec un code HTTP 200
Saccade sur la carte	Observation	Écritures SQLite unitaires sur le fil d'affichage
Exception audio non rattrapée	logcat	Aucun des quatre appels de lecture n'était protégé
MP4 découpé illisible	logcat (NoDeclaredBrand)	Boucle de copie interrompue au premier échantillon hors limite
Texte souligné en jaune	Capture d'écran	Route sans ancêtre Material : style de secours de Flutter
Groupes sans nom d'expéditeur	Lecture du code	Condition limitée au canal de diffusion
Livraison mensongère	Banc d'essai à deux paires	Refus de transport rapporté comme succès

Tableau 9 — Défauts détectés et corrigés

13.15 Optimisation

Quatre optimisations mesurables ont été réalisées :

30. Écritures du cache de tuiles regroupées par lots et vidées une fois par seconde dans une transaction unique, journal en mode WAL — supprime la saccade lors du défilement de la carte.
31. Sérialisation des tentatives de connexion Wi-Fi Direct avec délai croissant — divise par près de quatre le nombre d'erreurs BUSY.
32. Envoi séquentiel sur les transports au lieu de parallèle : le même message n'est plus émis deux fois lorsque deux liens fonctionnent, ce qui évitait notamment un trajet Bluetooth inutile et coûteux.
33. Dégradation automatique des effets graphiques coûteux lorsque le système signale une surchauffe, avec retour automatique une fois l'appareil refroidi.

13.16 Documentation

La documentation produite est de trois natures :

- Documentation dans le code — chaque fichier s'ouvre par un en-tête expliquant son rôle, et les décisions non évidentes sont justifiées à l'endroit où elles s'appliquent, en expliquant ce qui se produirait si l'on faisait autrement.
- Rapport d'expérimentation — le document `RAPPORT_SESSION_DEUX_PAIRS.md` consigne le protocole, les mesures et les constats du banc d'essai (reproduit en annexe H).
- Le présent rapport de stage.

CHAPITRE 14 — RÉALISATION TECHNIQUE

14.1 Structure du projet

droplet/	
├─ lib/	45 185 lignes, 93 fichiers Dart
│ └─ main.dart	Point d'entrée, routeur, superpositions
│ └─ core/	Noyau technique
│ └─ design_system/	Couleurs, typographie, composants
│ └─ features/	16 dossiers fonctionnels, 26 écrans
│ └─ shared/	Composants transverses
├─ android/	Code natif Kotlin (icônes, média)
├─ assets/	Icônes, shaders
├─ test/	8 fichiers, 63 tests
└─ tool/	Scripts de génération des icônes

14.2 Organisation des dossiers

Le dossier lib/core rassemble ce qui ne dépend d'aucun écran : models (structures de données), database (schéma Drift), services (17 services techniques), network (file d'envoi, gestionnaire réseau), protocol (routage Mesh), providers (état Riverpod) et repositories (orchestration).

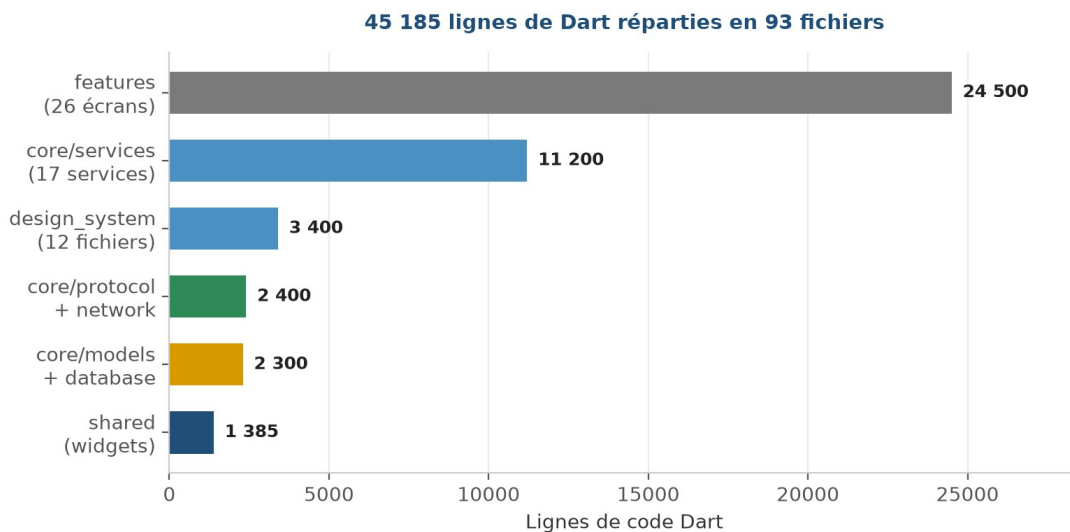


Figure 7 — Répartition des 45 185 lignes de code par module

Le dossier lib/features contient un sous-dossier par domaine fonctionnel : call, calls, chat, chats, contribution, group, home, maps, mesh, news, onboarding, peers, safety, settings, share, status. Chacun est autonome et ne dépend que du noyau et du système de design.

14.3 Architecture du code

Le projet suit une approche « feature-first » plutôt qu'un découpage par type technique. Le critère de découpage est la RAISON DE CHANGER : une évolution de la

messaging touche lib/features/chat, une évolution du transport touche lib/core/services, et les deux n'interfèrent pas.

14.4 Gestion de l'état

Riverpod est utilisé sous trois formes : des Provider simples pour les services sans état mutable, des StateNotifierProvider pour les états complexes (mesh, appels, apparence, thermique), et des Provider paramétrés pour les vues filtrées (messages d'une conversation donnée).

14.5 Base de données

Le schéma est déclaré en Dart et le code d'accès généré par build_runner. Les huit tables sont détaillées au tableau 5.

La déclaration d'une table en Drift se fait en Dart, et le code d'accès est ensuite généré. Le typage est fort : renommer une colonne provoque une erreur de compilation, là où une requête SQL écrite à la main n'échouerait qu'à l'exécution, chez l'utilisateur.

Extrait 1 — Déclaration de la table GroupMembers (lib/core/database/app_database.dart)

```
class GroupMembers extends Table {
  TextColumn get groupId => text();
  TextColumn get peerId  => text();
  TextColumn get role    => text();      // 'admin' | 'member'
  IntColumn  get addedAt  => integer();
  IntColumn  get removedAt => integer().nullable(); // tombstone
  TextColumn get addedBy  => text();

  @override
  Set<Column> get primaryKey => {groupId, peerId};
}
```

Ce qu'il faut y lire — la clé primaire est un COUPLE (groupId, peerId), ce qui interdit structurellement qu'un pair figure deux fois dans le même groupe — la contrainte est portée par le schéma, pas par du code applicatif qu'on pourrait oublier d'appeler. Et removedAt est nullable : c'est cette nullabilité qui matérialise le tombstone décrit plus haut.

14.6 Communication

Les trois transports implémentent la même forme de contrat : démarrage, arrêt, envoi vers un pair, et trois flux d'événements. MeshTransportService les orchestre en choisissant, pour chaque envoi, le transport le plus sain parmi ceux disponibles.

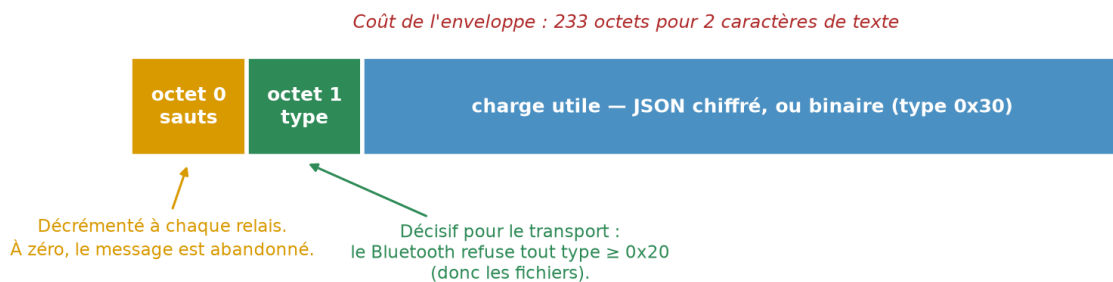
Un point de conception important, corrigé durant le projet : la méthode d'envoi doit renvoyer un VERDICT. Dans sa version initiale, elle ne renvoyait rien, ce qui rendait impossible de distinguer un envoi réussi d'un envoi refusé (voir 17.4).

14.7 Routage

Voir chapitre 11. L'implémentation tient dans un fichier unique, droplet_mesh_protocol.dart, qui expose une table de routage, un contrôle de congestion AIMD et un filtre de Bloom.

14.8 Messages

Une trame Droplet a la structure suivante :



Enveloppe JSON d'un message chiffré :

```
{ "c": <contenu chiffré base64>, "s": <émetteur>, "t": <cible>, "e": true, "n": <nonce>, "m": <id> }
```

Figure 3 — Structure d'une trame Droplet

octet 0 sauts	octet 1 type	charge utile JSON chiffré, ou binaire (0x30)
------------------	-----------------	---

Le premier octet porte le nombre de sauts restants, décrémenté à chaque relais. Le second identifie le type de contenu — cette valeur est décisive pour le transport, qui refuse tout type supérieur ou égal à 0x20 en Bluetooth. Le reste est soit du JSON (messages, statuts, accusés), soit un paquet binaire pour les fichiers.

L'enveloppe JSON d'un message chiffré comporte les champs : c (contenu chiffré en base64), s (identifiant de l'émetteur), t (identifiant de la cible), e (indicateur de chiffrement), n (nonce), m (identifiant du message) et, le cas échéant, r (message cité).

14.9 Groupes

Voir 12.9 et 12.10. L'implémentation fait avancer la chaîne de clés à chaque envoi et conserve les clés sautées pour permettre le déchiffrement des messages arrivés dans le désordre.

14.10 Chiffrement

Toutes les opérations sont regroupées dans CryptoService, qui n'expose que des méthodes statiques. Ce choix garantit un point d'entrée unique et évite la dispersion d'appels cryptographiques dans le code métier.

Le chiffrement d'un message repose sur deux opérations. La première dérive le secret partagé à partir de ma clé privée et de la clé publique du correspondant ; elle n'est effectuée qu'une fois par pair, le résultat étant mis en cache. La seconde chiffre le contenu avec un nonce distinct à chaque message.

Extrait 2 — Chiffrement AES-256-GCM ([lib/core/services/crypto_service.dart](#))

```
static Future<Uint8List cipher, Uint8List nonce> encryptBytes(
  SecretKey key, Uint8List plaintext) async {
  final box = await _aesGcm.encrypt(plaintext, secretKey: key);
  return (
    Uint8List.fromList([...box.cipherText, ...box.mac.bytes]),
    Uint8List.fromList(box.nonce),
  );
}
```

Ce qu'il faut y lire — la fonction renvoie DEUX valeurs, et la seconde est aussi importante que la première. Le nonce n'est pas un secret — il voyage en clair à côté du message — mais il ne doit jamais être réutilisé avec la même clé, sous peine de compromettre le chiffrement. La bibliothèque en génère un nouveau à chaque appel. On notera aussi que le code d'authentification (mac) est concaténé au texte chiffré : c'est lui qui permettra au destinataire de détecter toute altération en cours de route.

14.11 Appels

WebRTC est utilisé sans serveur STUN/TURN, les pairs étant sur le même réseau local. La signalisation transite par le mesh sous forme de messages typés.

14.12 Permissions Android

L'application demande un nombre inhabituel de permissions, toutes nécessaires à son fonctionnement : Bluetooth (scan, publicité, connexion), localisation (exigée par Android pour le scan BLE), Wi-Fi et appareils à proximité, micro, caméra, notifications, et service en premier plan de type synchronisation de données.

La permission de localisation mérite une explication en soi : Android l'exige pour tout scan Bluetooth, car la détection d'appareils environnants permet en théorie de déduire une position. L'application ne l'utilise pas à cette fin, mais ne peut pas s'en passer.

14.13 Gestion des erreurs

Voir 9.7. Le principe directeur retenu est qu'une donnée abîmée venant du réseau ne doit jamais faire tomber l'application. Ce principe a été appliqué après avoir observé un plantage franc provoqué par un simple fichier audio tronqué en cours de transfert.

14.14 Logs et débogage

Les traces sont préfixées par module ([MeshRepo], [BLE], [NativeP2P], [LocalWifi], [Statut], [Chat]), ce qui permet de filtrer efficacement les journaux de l'appareil. Cette convention s'est révélée essentielle : sur un appareil réel, les journaux de l'application se noient dans ceux du système.

14.15 Optimisation

Voir 13.15.

CHAPITRE 15 — INTERFACE ET EXPÉRIENCE UTILISATEUR

15.1 Principes UX

Trois principes ont guidé la conception de l'interface :

- 34. Dire la vérité sur l'état du réseau. Dans une application hors ligne, l'utilisateur ne peut pas deviner pourquoi un message n'part pas. Chaque état incertain doit être nommé.
- 35. Ne jamais masquer une action indisponible, mais l'expliquer. Un bouton d'appel qui disparaît et réapparaît au gré de la portée ferait sauter toute la barre.
- 36. Respecter les conventions de la plateforme dans le détail, y compris ce qui ne se nomme pas : hauteur des barres, espacement des lettres, courbes d'animation.

15.2 Design System

Le système comprend une palette adaptative (clair/sombre), une échelle typographique complète, un jeu de tokens d'espacement et de rayons, et une bibliothèque de composants : listes groupées, feuilles modales, alertes, boutons de barre, indicateurs d'attente, avatars, badges.

À COMPLÉTER PAR VOUS — *Insérez une planche présentant la palette et l'échelle typographique.*

15.3 Composants et matériaux

Plutôt que les composants Material par défaut, l'application emploie un ensemble de composants sur mesure reproduisant les conventions mobiles attendues : listes groupées à séparateurs décalés, feuilles à matériau translucide avec poignée, alertes centrées, indicateur d'attente en couronne de traits plutôt qu'en arc tournant.

15.4 Navigation

La navigation distingue deux gestes fondamentaux : la DESCENTE d'un cran (la page entre depuis la droite, la précédente recule en s'assombrissant, et le retour se fait par un geste depuis le bord gauche) et la PARENTHÈSE modale (la page monte depuis le bas et se referme par un bouton). Treize écrans relèvent du premier mode, quatre du second.

15.5 Onboarding

Le parcours d'accueil présente en quelques écrans animés le principe du réseau maillé, le chiffrement, la carte et le mode urgence, avant de demander un pseudonyme et de créer l'identité cryptographique.

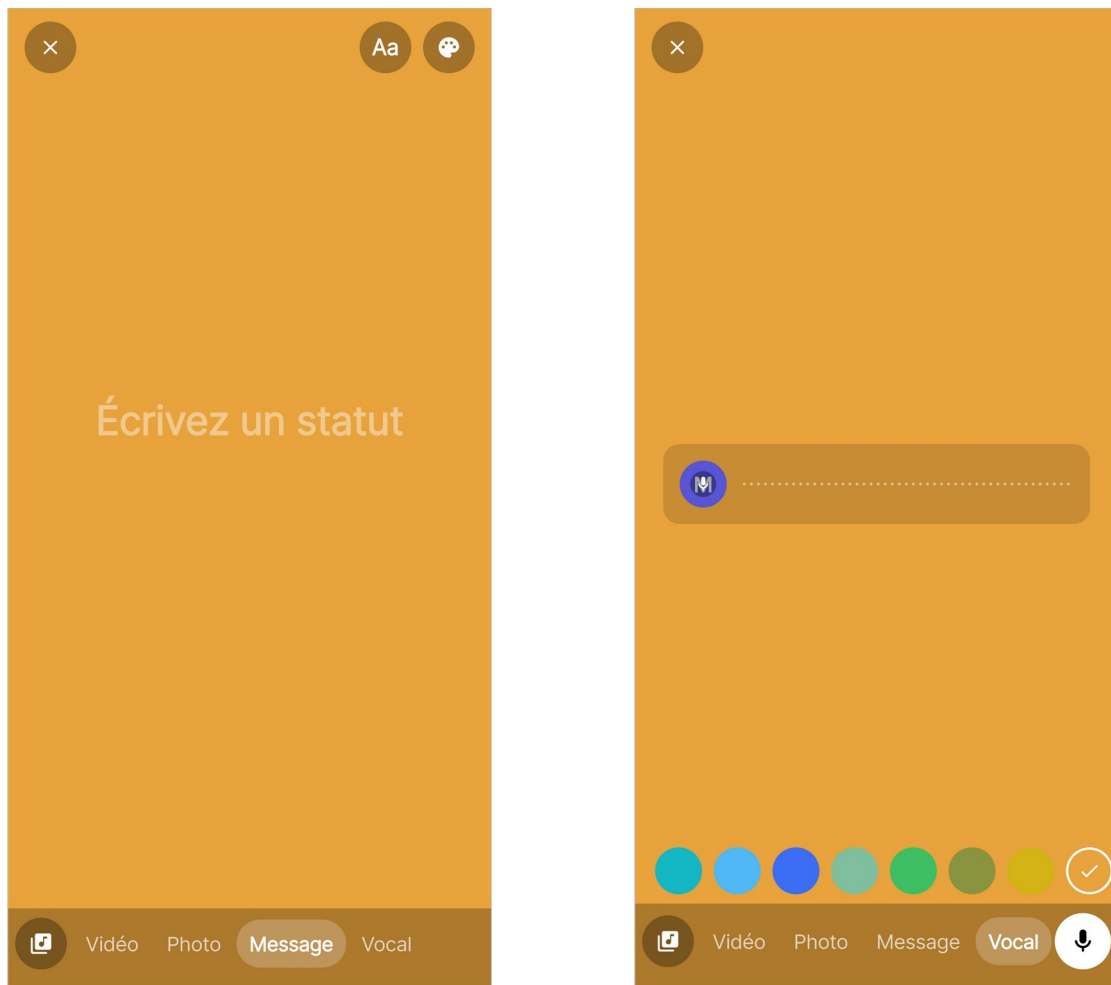


Figure 16 — Composition d'un statut : mode Message (à gauche) et mode Vocal avec sa forme d'onde (à droite)

15.6 Création d'identité

L'utilisateur saisit un pseudonyme ; l'application génère alors la paire de clés X25519 et calcule l'identifiant par empreinte SHA-256. Aucune adresse électronique, aucun numéro de téléphone, aucune validation distante.

15.7 Accueil

L'écran d'accueil est la liste des discussions, avec un grand titre repliable, une barre de recherche et un sous-titre indiquant en permanence le nombre de pairs à proximité.

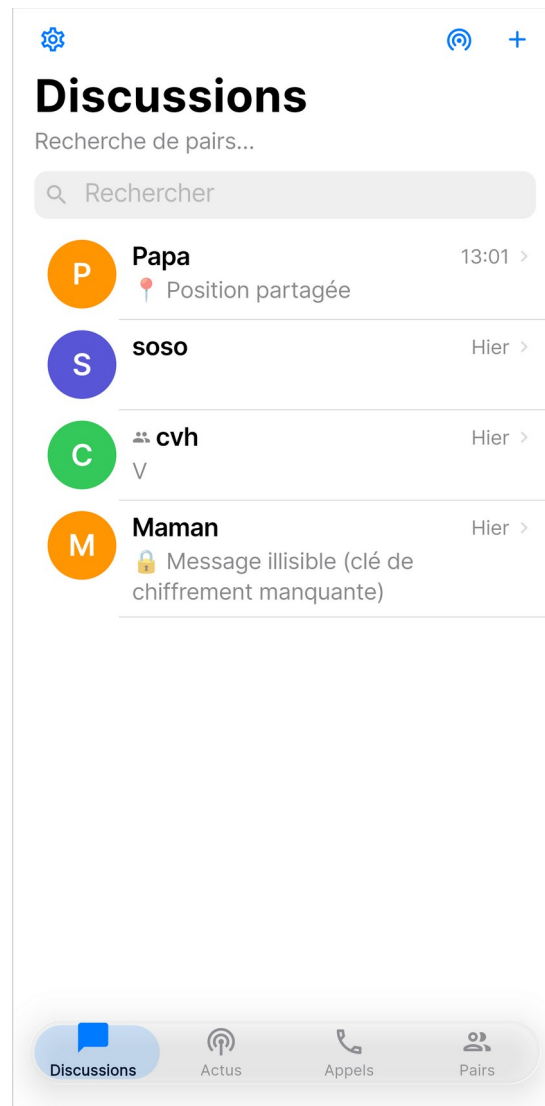


Figure 10 — Écran d'accueil : liste des discussions, grand titre repliable et état du réseau en sous-titre

15.8 Conversations

Voir 13.7 pour le détail des fonctionnalités de l'écran de conversation.

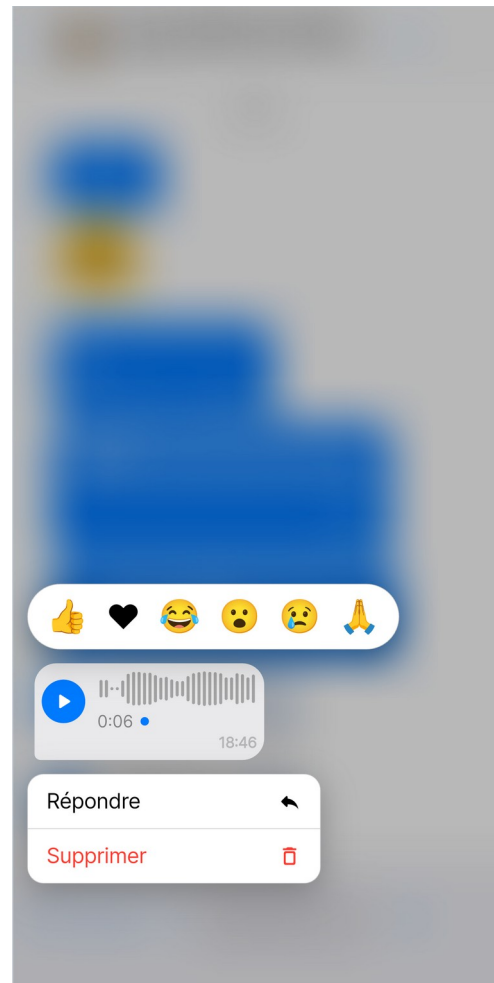
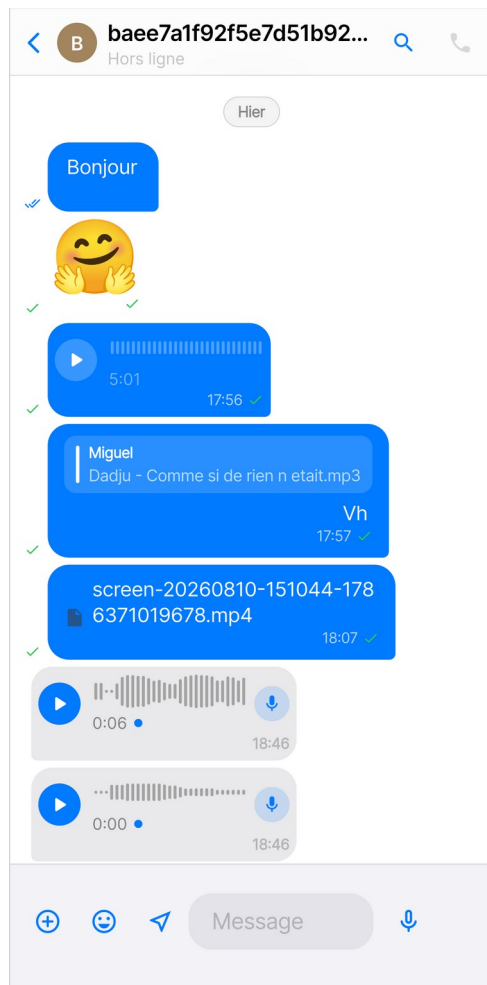


Figure 11 — Écran de conversation (à gauche) et menu contextuel ancré sur un message vocal (à droite) : le fond se floute, le message choisi reste net

15.9 Groupes

Trois écrans : création, conversation et informations du groupe (membres, administration, appel de groupe).

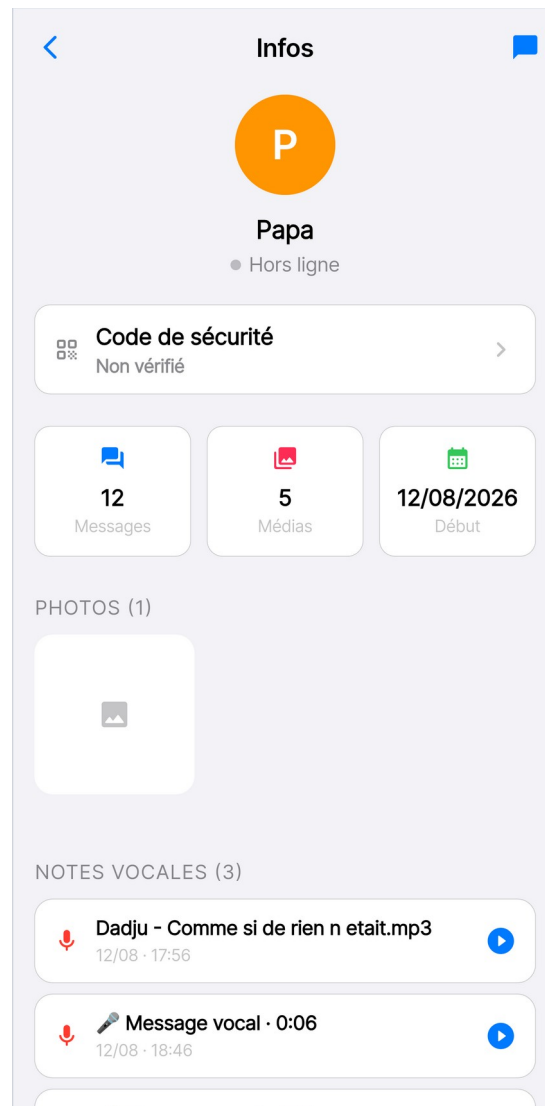


Figure 12 — Informations d'une conversation : statistiques, code de sécurité et médias partagés

15.10 Contacts et pairs

L'écran Pairs présente les appareils à portée avec leur transport actif et leur état de vérification.

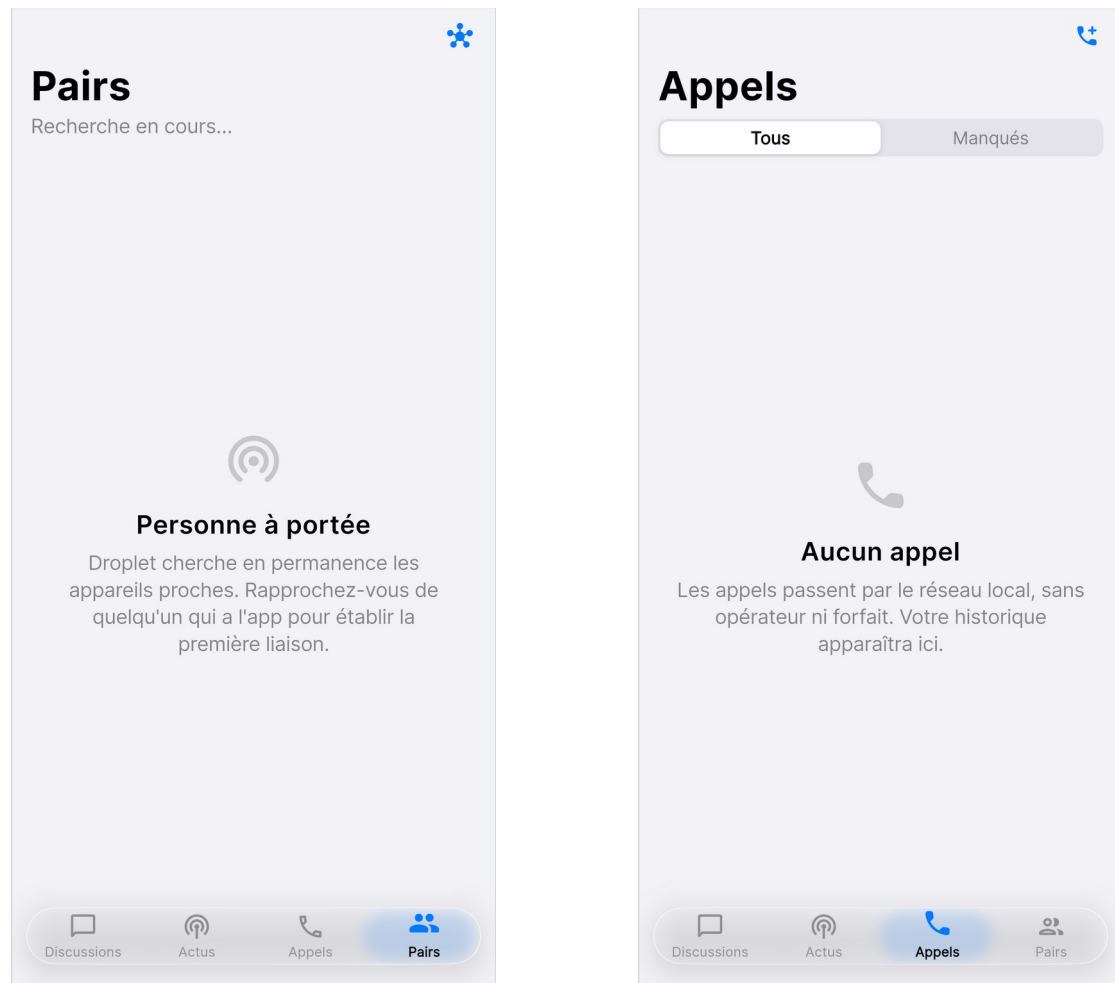


Figure 13 — Écran Pairs (à gauche) et journal d'appels (à droite), tous deux dans leur état vide explicatif

15.11 Appels

Un journal d'appels réel, alimenté par les appels effectivement passés et reçus, et un écran d'appel en cours.

15.12 Paramètres

Apparence, icône de l'application, réseau, cartes hors ligne, sauvegarde de l'identité, notifications et service en arrière-plan.

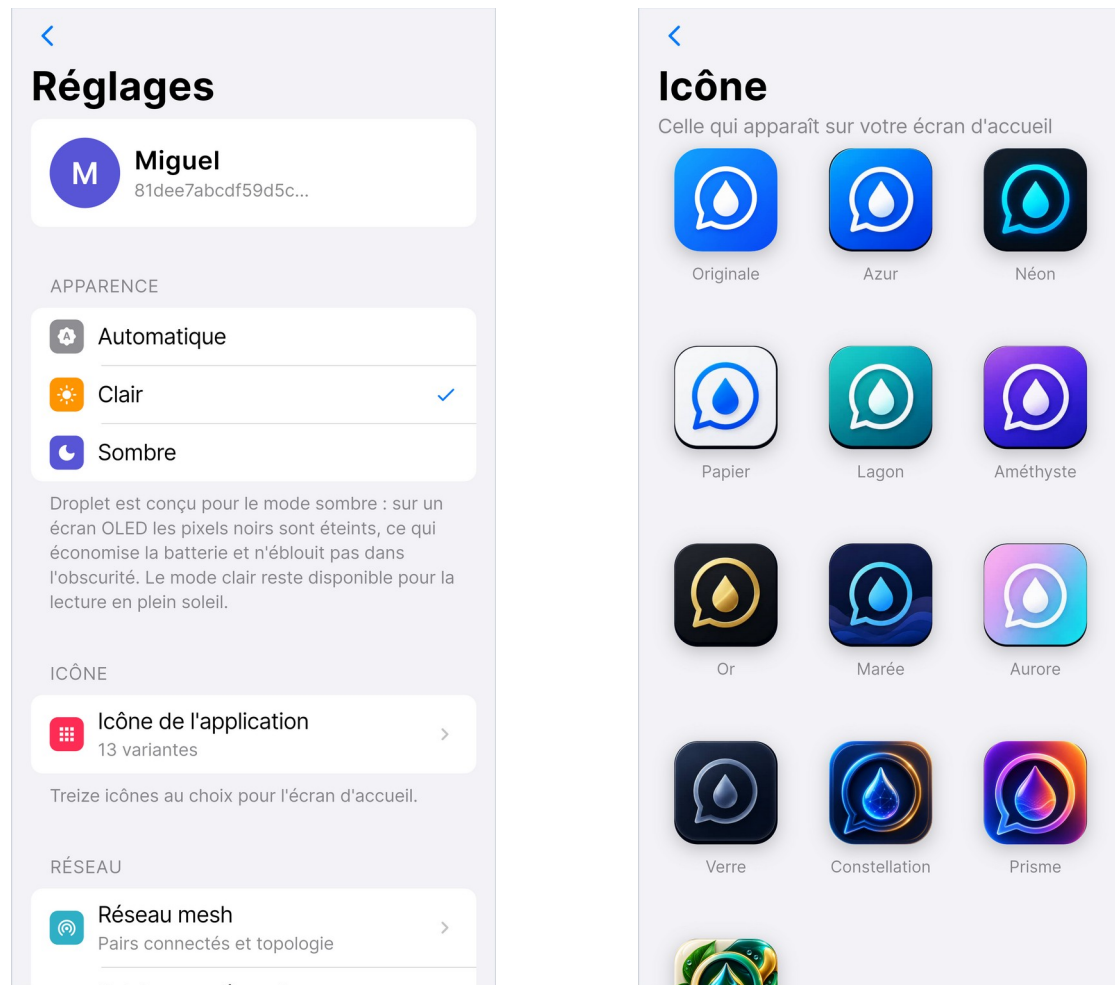


Figure 14 — Réglages (à gauche) et sélecteur d'icône de l'application, treize variantes (à droite)

15.13 Mode clair

À COMPLÉTER PAR VOUS — Insérez une capture d'écran du mode clair.

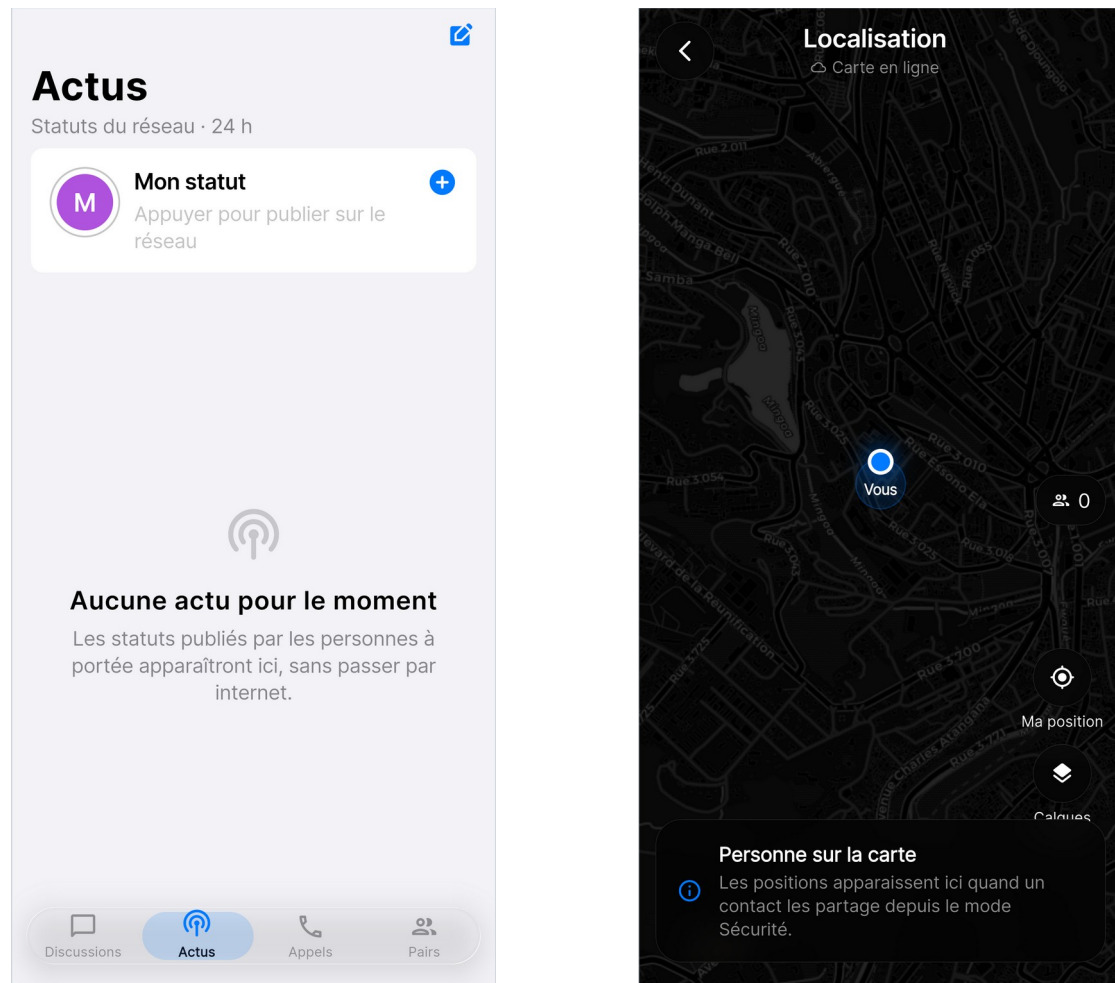


Figure 17 — Mode clair (écran Actus, à gauche) et mode sombre (cartographie, à droite). Le mode sombre est le réglage par défaut

15.14 Mode sombre

À COMPLÉTER PAR VOUS — Insérez une capture d'écran du mode sombre. Le mode sombre est le défaut de l'application.

15.15 Animations

Les animations reposent sur des RESSORTS physiques plutôt que sur des courbes de durée fixe. La différence se sent lorsqu'un geste est interrompu : une courbe repart de zéro, un ressort conserve la vitesse en cours et rattrape la nouvelle cible sans à-coup.

15.16 Micro-interactions

Retours haptiques différenciés selon la nature de l'action (sélection, impact léger, impact moyen, succès), enfoncement visuel des lignes plutôt qu'onde de propagation, et vibration à chaque cran franchi lors du balayage de la barre d'onglets.

15.17 Responsive design

À COMPLÉTER PAR VOUS — Décrivez l'adaptation aux différentes tailles d'écran. Précisez les tailles réellement testées.

PARTIE V — TESTS ET ÉVALUATION

CHAPITRE 16 — STRATÉGIE DE TEST

16.1 Objectifs

Les tests menés poursuivent quatre objectifs : valider le fonctionnement des besoins prioritaires, éprouver la robustesse du réseau maillé, vérifier la solidité du chiffrement, et — objectif qui s'est révélé le plus structurant — MESURER le taux de livraison réel plutôt que de le supposer.

Ce dernier point mérite d'être justifié. Dans une application client-serveur, le serveur constitue un témoin : il sait combien de messages il a reçus. Dans un réseau décentralisé, ce témoin n'existe pas. Chaque appareil ne connaît que ce qu'il a tenté d'envoyer. Il faut donc construire délibérément le moyen de savoir si un envoi a abouti, faute de quoi le système peut fonctionner « en apparence » pendant très longtemps.

16.2 Environnement

À COMPLÉTER PAR VOUS — Décrivez l'environnement de test : intérieur ou extérieur, nature des obstacles, distances mesurées entre appareils.

16.3 Matériel

Les tests sur appareil ont été menés sur un smartphone Google Pixel 6 Pro (nom de code raven).

À COMPLÉTER PAR VOUS — Complétez avec le nombre total d'appareils dont vous disposez et leurs modèles. Si vous n'aviez qu'un seul appareil, dites-le : cela explique et JUSTIFIE le recours au banc d'essai à deux pairs décrit en 16.7, et le jury appréciera cette honnêteté davantage qu'un flou.

16.4 Versions logicielles

Élément	Version
Flutter	3.41.6 (canal stable)
Dart SDK	3.11.4
Moteur de rendu	Impeller (Vulkan)
Appareil de test	Google Pixel 6 Pro

À COMPLÉTER PAR VOUS — Ajoutez la version d'Android de votre appareil et le numéro de version de l'application testée.

16.5 Méthodologie

La méthodologie combine quatre niveaux, du plus isolé au plus réaliste :

- 37. Tests unitaires — validation de la logique pure : cryptographie, file d'attente, encodage des métadonnées vocales, sauvegarde.
- 38. Test de construction d'interface — vérification que l'application se construit sans exception.
- 39. Banc d'essai à deux pairs — exécution du vrai code réseau contre un correspondant simulé, avec mesure du taux de livraison.
- 40. Observation sur appareil réel — journaux système et captures d'écran, seul moyen de détecter les défauts liés au matériel, aux permissions et au cycle de vie.

16.6 Critères d'acceptation

Les critères sont ceux définis en section 4.9.

16.7 Le banc d'essai à deux pairs

Ce banc mérite une description détaillée, car il constitue l'apport méthodologique principal de l'évaluation.

Principe

Le correspondant simulé n'a de simulé QUE la radio. Tout le reste est le code de production : les vraies clés X25519, le vrai secret partagé, le vrai AES-256-GCM, la vraie enveloppe de message, la vraie file d'attente avec ses relances et son délai exponentiel. Le pair simulé DÉCHIFFRE réellement ce qu'il reçoit et refuse ce qui est abîmé — sans quoi les chiffres obtenus ne voudraient rien dire.

Ce que le banc mesure, et ce qu'il ne mesure pas

Le banc mesure le coût du logiciel et le nombre de messages qui parviennent effectivement à destination sur un lien qui perd des paquets. Il ne mesure PAS la radio : personne ne peut simuler honnêtement un signal Bluetooth traversant un mur. Le temps d'antenne est donc CALCULÉ à partir de la taille réelle des trames et du débit connu de chaque transport, puis présenté séparément — jamais mélangé au temps logiciel, lui réellement chronométré.

Cette distinction est une exigence de rigueur : présenter un temps calculé comme une mesure serait une faute méthodologique.

Profils de lien

Paramètre	Bluetooth LE	Wi-Fi local
Latence d'établissement	250 ms	8 ms
Octets utiles par écriture	19	4 096
Durée d'une écriture	20 ms	4 ms
Taux de perte modélisé	3 %	0,5 %

Fichiers acceptés	Non (règle du transport)	Oui
Charge maximale	512 octets	Non limitée

Ces valeurs ne sont pas arbitraires : la taille de fragment de 19 octets est celle que le code utilise réellement, et l'intervalle de 20 ms correspond à un intervalle de connexion BLE optimiste sur Android.

Lot de test

Chaque exécution fait passer 100 messages texte, 20 fichiers de 250 Ko et 10 partages de position, soit 130 envois.

CHAPITRE 17 — TESTS ET RÉSULTATS

17.1 Tests fonctionnels et unitaires

La suite automatisée compte 63 tests répartis en 8 fichiers.

Fichier de test	Objet
crypto_service_test.dart	Dérivation de clés, chiffrement, déchiffrement, numéros de sécurité
premium_message_queue_test.dart	Priorités, relances, délai exponentiel, annulation, ordre de traitement
backup_service_test.dart	Sauvegarde chiffrée : bon mot de passe, mauvais mot de passe, fichier corrompu
voice_note_meta_test.dart	Encodage et décodage des métadonnées vocales
status_test.dart	Modèle des statuts et de leurs médias
contribution_test.dart	Statistiques de contribution au réseau
two_peer_session_test.dart	Banc d'essai à deux pairs (5 tests)
widget_test.dart	Construction de l'application sans exception

Résultat : 63 tests passent, et l'analyse statique (flutter analyze) ne signale aucun problème.

17.2 Résultats du banc — Wi-Fi local

```

Envois mis en file      : 130
Arrivés                 : 130  (100,00 %)
Perdus définitivement  : 0
Refusés par le transport : 0
Réellement reçus par B : 130
Tentatives totales     : 131
Relances par message   : 0,008
Octets réellement émis  : 5 485 Kio
Temps LOGICIEL (mesuré) : 8,1 s
Temps ANTENNE (calculé) : 6,5 s
Latence médiane en file : 2,4 ms

```

Lecture

En Wi-Fi, la totalité du lot passe. Le taux de relance de 0,008 par message signifie qu'environ un envoi sur 125 a dû être retenté une fois — cohérent avec le taux de perte modélisé de 0,5 %. Aucun message n'est perdu définitivement : le mécanisme de relance absorbe intégralement la perte.

17.3 Résultats du banc — Bluetooth LE

Envois mis en file	: 130	
Arrivés	: 110	(84,62 %)
Perdus définitivement	: 20	
Refusés par le transport	: 100	
Réellement reçus par B	: 110	
Tentatives totales	: 218	
Relances par message	: 0,982	
Octets réellement émis	: 20,5 Kio	
Temps LOGICIEL (mesuré)	: 4,5 s	
Temps ANTENNE (calculé)	: 50,5 s	
Latence médiane en file	: 0,3 ms	

Lecture

En Bluetooth, les 20 fichiers ne passent pas : le transport les écarte délibérément, et la file — après cinq relances chacune, soit 100 refus — les déclare échoués. Les 110 messages texte et positions arrivent tous. Le point essentiel est l'égalité entre « Arrivés » et « Réellement reçus par B » : ce qui est annoncé livré a bien été reçu.

Le temps d'antenne de 50,5 secondes pour 110 messages courts établit un ordre de grandeur utile : en Bluetooth seul, Droplet fait passer environ deux messages par seconde.

17.4 Découverte majeure : la livraison mensongère

Le résultat le plus important de l'évaluation n'est pas un chiffre de performance, mais un DÉFAUT que le banc a révélé dès sa première exécution.

Constat

Dans sa version initiale, le banc affichait en Bluetooth :

Envois mis en file	: 130	
Arrivés	: 130	(100,00 %) ← annoncé
Réellement reçus par B	: 110	← constaté

Vingt messages étaient comptabilisés comme livrés alors que le destinataire ne les avait jamais reçus. L'interface affichait deux coches, l'utilisateur croyait avoir envoyé, et rien n'avait quitté l'appareil.

Cause

La cause tenait en deux endroits distincts du code. D'une part, la méthode d'envoi du service de transport ne renvoyait RIEN — son type de retour était `Future<void>`. D'autre part, la méthode de routage répondait « succès » dès lors que le pair figurait dans la table des pairs connus, avant même de savoir si l'envoi avait abouti :

```
Future<bool> sendViaRoute(String peerId, Uint8List data) async {
  if (_peers.containsKey(peerId)) {
    await sendToPeer(peerId, data);    // ne renvoie rien
```



```

    return true;                // ← toujours vrai
}

```

En face, le transport Bluetooth écartait silencieusement les charges qu'il ne peut pas porter — un simple message de journal, puis un retour sans valeur. Le message était perdu, et la file, n'ayant reçu aucun signal d'échec, n'avait aucune raison de le relancer.

Conséquence méthodologique

Ce défaut invalidait tout chiffre de fiabilité que l'application pouvait produire : le « taux de livraison » affiché était en réalité un taux de TENTATIVE d'émission. Toute mesure faite sur cette base — y compris un essai sur deux appareils réels — aurait produit un résultat faux et flatteur.

Correction

La vérité a été propagée sur l'ensemble de la chaîne :

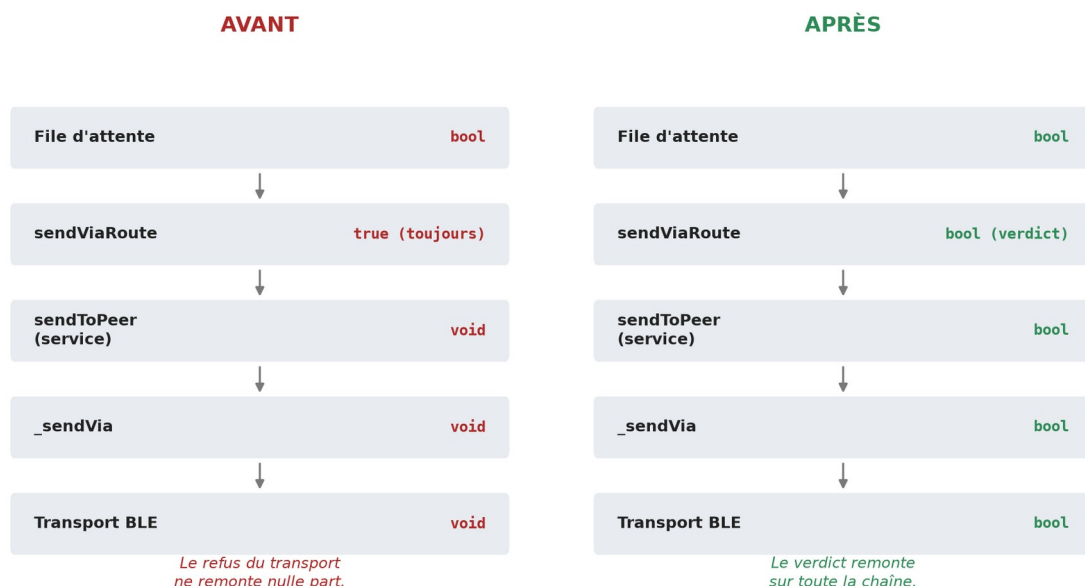


Figure 8 — Propagation du verdict d'envoi : sept couches modifiées pour qu'un refus de transport remonte jusqu'à la file

Couche	Avant	Après
BleMeshTransport.sendToPeer	Future<void>, trois refus muets	Future<bool>
LocalWifiTransport.sendToPeer	Future<void>	Future<bool>
NativeP2PTransport.sendToPeer	Future<void>, muet si pair inconnu	Future<bool>
MeshTransportService.sendTo	Future<void>	Future<bool>

Peer		
sendViaRoute	true si le pair est connu	Renvoie le verdict du transport
broadcastToConnectedPeers	Future<void>	Future<bool> — compte les succès réels
_reliableOnSend (file)	true si des pairs sont connectés	Renvoie ce que la diffusion a réellement fait

Trois défauts secondaires ont été découverts au cours de cette correction :

41. La fonction de découpage en fragments s'arrêtait au premier fragment en échec — décision correcte — mais sans le signaler. Or un message coupé en deux est perdu : le destinataire ne peut pas le rassembler.
42. Le score de santé des transports récompensait ceux qui écartaient le plus de charges : le succès était enregistré dès que l'appel rendait la main sans exception, donc y compris après un refus.
43. Le type de contenu n'était pas transmis au transport, qui recevait toujours la valeur 0x00. Le Bluetooth ne pouvait donc pas reconnaître un fichier avant de buter sur sa limite de taille.

Un envoi en double a également été supprimé : les transports candidats étaient lancés en PARALLÈLE et le premier à répondre l'emportait, ce qui émettait deux fois le même message lorsque deux liens fonctionnaient. Ils sont désormais essayés séquentiellement, dans l'ordre de santé.

Vérification de la correction

Indicateur	Avant	Après
Annoncés livrés	130 (100 %)	110 (84,62 %)
Réellement reçus	110	110
Signalés en échec	0	20
Écart annoncé / réel	20 messages	0

Tableau 7 — Résultats du banc avant et après correction

L'invariant est rétabli : ce qui est annoncé livré a été reçu. Un test de non-régression conserve la trace du défaut en jouant l'ancien comportement, de sorte qu'un retour en arrière serait immédiatement détecté.

Lot de 130 envois en Bluetooth LE — 100 messages, 20 fichiers, 10 positions

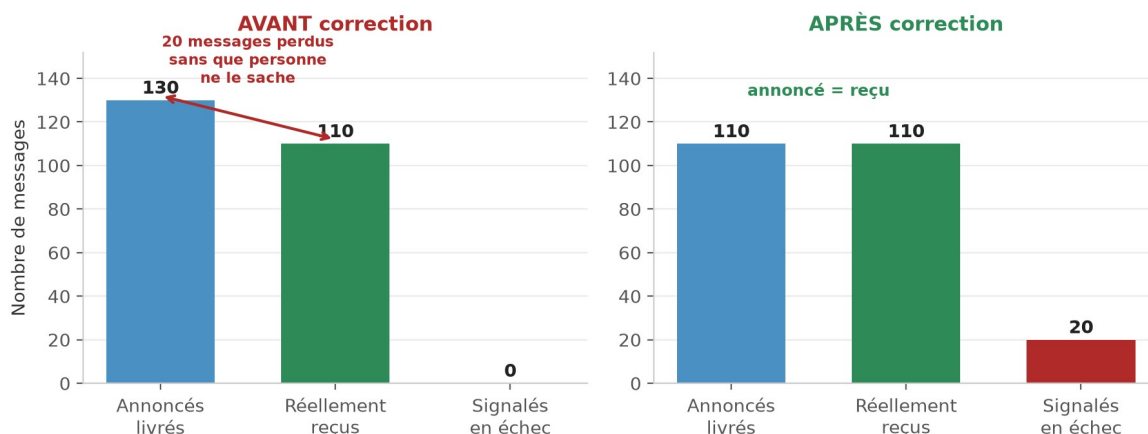


Figure 5 — Taux de livraison avant et après correction : l'écart de 20 messages entre livraison annoncée et réception effective disparaît

17.5 Taille des trames et limite Bluetooth

Une seconde mesure a été menée pour comprendre à partir de quelle longueur un message cesse de passer en Bluetooth. Les trames sont mesurées avec des identifiants de 64 caractères hexadécimaux, ce que produit effectivement la génération d'identité.

Contenu du message	Taille de la trame	Limite BLE (512 o)
« Ok » (2 caractères)	233 octets	Passe
Message ordinaire (38 caractères)	281 octets	Passe
200 caractères	497 octets	Passe — marge de 15 octets
300 caractères	633 octets	DÉPASSÉE

Tableau 8 — Taille des trames face à la limite BLE

Lecture

L'enveloppe coûte 233 octets pour deux caractères de texte. Ce poids provient des deux identifiants de 64 caractères (émetteur et cible), du codage base64 du texte chiffré et de celui du nonce. Le seuil de rupture se situe autour de 210 caractères, soit la longueur d'un message un peu bavard.

Deux corrections sont possibles et cumulables : transporter les identifiants sous forme binaire plutôt qu'hexadécimale, ce qui économiserait environ 64 octets par message ; et fragmenter au niveau applicatif au-delà de la limite du transport, plutôt que de refuser.

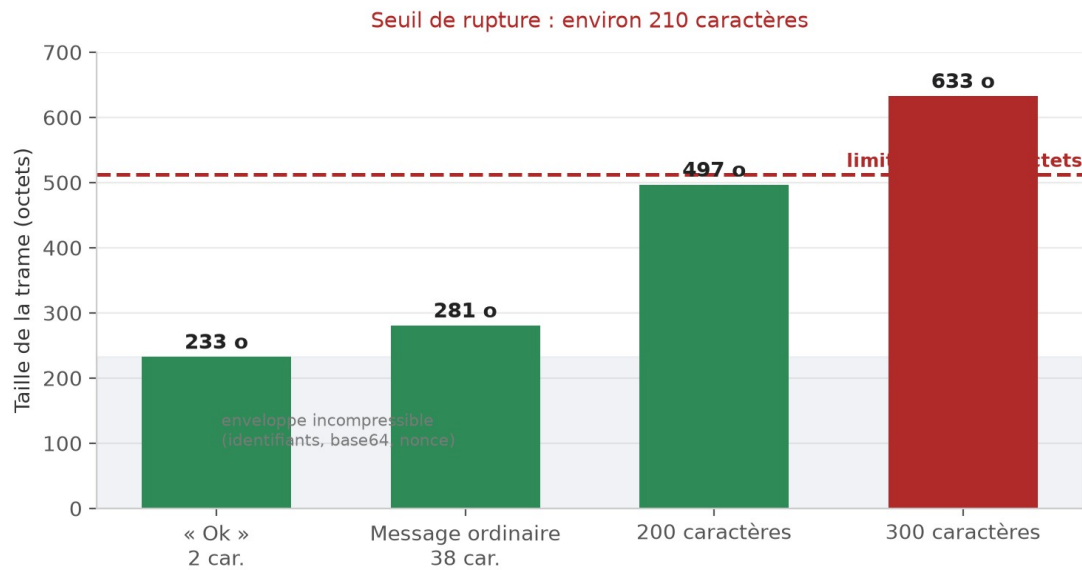


Figure 6 — Taille des trames chiffrées face à la limite de 512 octets du transport Bluetooth

17.6 Tests de persistance

La persistance est validée par les tests de sauvegarde et de restauration, qui couvrent trois cas : mot de passe correct (la sauvegarde se déchiffre et s'analyse), mot de passe erroné (échec propre par exception typée), et fichier corrompu (échec propre, sans exception non gérée).

À COMPLÉTER PAR VOUS — Complétez par un test manuel : envoyez des messages, forcez l'arrêt de l'application, relancez-la, et vérifiez que l'historique est intact. Notez le nombre de messages et le résultat.

17.7 Tests de chiffrement

Le chiffrement est validé à deux niveaux. Les tests unitaires vérifient la dérivation du secret partagé, la réversibilité du chiffrement et le calcul des numéros de sécurité. Le banc à deux pairs apporte une validation plus forte : le pair simulé déchiffre RÉELLEMENT chaque message reçu avec une clé dérivée indépendamment de son côté. Un test dédié vérifie que les 25 messages d'un lot sont tous déchiffrés à l'identique.

17.8 Tests des groupes

À COMPLÉTER PAR VOUS — Décrivez les tests menés sur les groupes. Scénario recommandé : créer un groupe de trois membres, échanger des messages dans le désordre (en coupant temporairement un appareil), puis vérifier que le rattrapage des clés sautées fonctionne.

17.9 Tests des appels

À COMPLÉTER PAR VOUS — Décrivez les tests d'appel. Notez en particulier le délai d'établissement et le comportement lorsque la liaison passe du Wi-Fi au Bluetooth en cours d'appel.

17.10 Tests de performance

Les mesures de performance issues du banc sont les suivantes :

Indicateur	Valeur mesurée	Interprétation
Traitement applicatif par envoi	≈ 62 ms	Dominé par le chiffrement des fichiers
Latence médiane en file	1,5 à 2,4 ms	La file n'est pas un goulot d'étranglement
Relances par message (Wi-Fi)	0,008	Perte quasi nulle absorbée
Relances par message (BLE)	0,062	Perte de 3 % absorbée sans échec
Débit Bluetooth	≈ 2 messages/s	Contrainte structurelle

Conclusion de ces mesures : le logiciel n'est pas le facteur limitant. La totalité du temps est consommée par la radio, ce qui est le bon endroit — cela signifie qu'il n'y a pas de gain à espérer d'une optimisation du code applicatif.

17.11 Tests d'ergonomie

À COMPLÉTER PAR VOUS — Décrivez les tests d'ergonomie menés : nombre d'utilisateurs sollicités, tâches proposées, difficultés observées. Si aucun test utilisateur formel n'a été mené, dites-le et indiquez-le comme une limite en section 20.1.

17.12 Tableau des résultats

Test	Résultat attendu	Résultat obtenu	État
Création d'identité	Identité créée localement	Clé X25519 générée, identifiant SHA-256	Réussi
Envoi 1:1 (Wi-Fi)	Message transmis et déchiffré	130/130 reçus et déchiffrés	Réussi
Envoi 1:1 (BLE)	Message transmis et déchiffré	110/110 messages courts reçus	Réussi
Fiabilité annoncée	Livré = reçu	Écart de 20 avant correction, 0 après	Réussi après correction
Partage de fichier (Wi-Fi)	Fichier transmis	20/20 transmis et déchiffrés	Réussi
Partage de fichier (BLE)	Fichier transmis	Refusé par le transport, signalé en échec	Limite assumée
Message long (> 210 car.)	Message transmis	Refusé en BLE au-delà de 512 octets	Partiel
Chiffrement	Message illisible par un tiers	Déchiffrement vérifié côté pair	Réussi

Persistence	Message conservé	Sauvegarde/ restauration validées par test	Réussi
Relais multi-hop	Message relayé via un intermédiaire	[À renseigner — exige trois appareils]	Non validé
Groupe	Message distribué à tous les membres	[À renseigner]	—
Appel audio	Communication établie	[À renseigner]	—

Tableau 10 — Synthèse des résultats de test

À COMPLÉTER PAR VOUS — Complétez les trois dernières lignes avec vos observations réelles. Si le relais multi-hop n'a pas pu être validé faute de trois appareils, inscrivez-le tel quel : c'est une limite honnête, pas un échec.

CHAPITRE 18 — DIFFICULTÉS ET SOLUTIONS

18.1 Difficultés de développement

La difficulté générale la plus marquante n'a pas été technique mais méthodologique : dans un système décentralisé, RIEN ne dit qu'une chose ne fonctionne pas. Il n'existe ni journal serveur, ni page d'erreur, ni utilisateur qui se plaint immédiatement. Un message perdu ne laisse aucune trace ; une radio éteinte se comporte exactement comme une radio sans voisin.

Cette absence de témoin a conduit à adopter une discipline stricte : ne jamais conclure qu'une fonctionnalité marche sans l'avoir observée, et considérer tout succès non vérifié comme un succès supposé.

18.2 Difficultés BLE

Trois difficultés spécifiques ont été rencontrées :

44. Le débit. Sans négociation de MTU, une écriture Bluetooth transporte 19 octets utiles. Toute la conception du protocole en découle : enveloppes compactes, refus des contenus volumineux, priorisation.
45. Le double rôle. Chaque appareil doit être simultanément périphérique (pour être découvert) et central (pour découvrir), ce qui double la machine à états et complique le suivi des liens.
46. La reprise après échec. Un unique dépassement de délai au démarrage laissait le transport définitivement inactif — le défaut le plus coûteux du projet, car il rendait l'application silencieusement inopérante.

18.3 Difficultés Wi-Fi

La découverte Wi-Fi dépend de la configuration du réseau : certains points d'accès isolent les clients les uns des autres, ce qui empêche les balises de diffusion d'atteindre leur destination. Le repli sur Wi-Fi Direct traite ce cas, au prix de la contrainte séquentielle décrite ci-dessous.

18.4 Difficultés Mesh

La difficulté principale a été une contrainte non documentée de l'API Nearby Connections : la couche native ne traite qu'UNE SEULE opération de connexion à la fois. Le code, en lançant une tentative vers chaque pair découvert à chaque mise à jour de la liste, produisait une exception BUSY environ une fois par seconde, en continu.

Le diagnostic n'a été possible que par l'observation des journaux : l'application ne montrait aucun symptôme visible, sinon une lenteur de connexion inexplicée. La correction repose sur trois mécanismes : un verrou empêchant toute tentative parallèle, un délai croissant par pair plafonné à deux minutes, et un compteur d'échecs par pair.

18.5 Difficultés de synchronisation

Le cas le plus délicat a été celui d'un message chiffré arrivant avant la clé publique de son émetteur. Le message était alors définitivement perdu, l'interface affichant « Message illisible ». La correction introduit une file de déchiffrements en attente, réessayée dès qu'une clé arrive, et un flux d'événements dédié permettant à l'interface de remplacer rétroactivement le message illisible par son contenu réel.

18.6 Difficultés de chiffrement

La difficulté conceptuelle du chiffrement de groupe est le compromis entre coût et sécurité. Chiffrer individuellement pour chaque membre multiplie le trafic par le nombre de destinataires — inacceptable à deux messages par seconde. Le mécanisme de Sender Keys résout ce point, mais introduit une difficulté nouvelle : les messages n'arrivant pas dans l'ordre dans un réseau à relais, il faut conserver les clés sautées pour pouvoir déchiffrer un message en retard.

18.7 Difficultés WebRTC

La principale difficulté est que WebRTC suppose habituellement un canal de signalisation fiable, alors qu'ici ce canal est le réseau maillé lui-même — c'est-à-dire précisément ce qui peut être lent ou intermittent. La conséquence pratique est que l'établissement d'un appel n'est possible que sur une liaison Wi-Fi, ce qui a été rendu visible dans l'interface plutôt que masqué.

18.8 Difficultés liées aux performances

Deux problèmes de performance ont été diagnostiqués et corrigés :

47. Saccade lors du défilement de la carte, causée par des écritures SQLite unitaires exécutées sur le fil d'affichage à chaque tuile téléchargée. Correction : regroupement par lots, transaction unique, journal en mode WAL.
48. Émission en double du même message lorsque plusieurs transports étaient disponibles, les candidats étant lancés en parallèle. Correction : essai séquentiel dans l'ordre de santé.

18.9 Difficultés liées aux appareils

La difficulté la plus instructive du projet relève de cette catégorie, et mérite d'être racontée en détail car elle illustre un piège de diagnostic.

Symptôme

L'enregistrement d'une vidéo produisait un fichier de zéro octet, et l'application affichait « Vidéo illisible ».

Première hypothèse — erronée

Les journaux montraient que l'implémentation caméra employée n'envoyait aucune image à l'encodeur : le message « Cached audio data while we wait for video keyframe

» se répétait pendant toute la prise, puis le fichier était refermé sur une erreur. L'hypothèse retenue fut un défaut de cette implémentation, et une implémentation alternative fut mise en place.

Constat décisif

Avec l'implémentation alternative, le journal du multiplexeur devint sans ambiguïté :

```
MPEG4Writer: encoded 3499 frames. - Audio
MPEG4Writer: The number of recorded samples is 0
MPEG4Writer: encoded 0 frames. - Video
```

3 499 trames de son, zéro trame d'image — le même symptôme avec deux implémentations différentes. La cause ne pouvait donc pas se situer dans l'une d'elles.

Cause réelle

L'interrogation de l'état thermique du système donna la réponse :

```
Thermal Status: 4    (0 = aucune surchauffe ... 4 = critique)
VIRTUAL-SKIN = 46,5 °C · CPU = 89 °C · batterie = 39,8 °C
```

À partir du niveau 3, Android COUPE les encodeurs vidéo. La caméra continue d'afficher son aperçu, le micro continue d'enregistrer, mais plus aucune image n'atteint l'encodeur. Les deux implémentations échouaient à l'identique parce que la panne se situait EN DESSOUS d'elles.

Corrections

Le changement d'implémentation, fondé sur un diagnostic erroné, a été annulé. À la place, un pont natif interroge désormais l'état thermique et l'application prévient explicitement l'utilisateur au lieu d'afficher un message d'erreur qui accuse le logiciel d'un défaut qu'il n'a pas. Le même indicateur sert à désactiver automatiquement les effets graphiques coûteux lorsque l'appareil souffre.

Enseignement

Un symptôme identique avec deux implémentations différentes doit faire chercher la cause à un niveau inférieur, et non conclure que les deux implémentations sont défectueuses. Le réflexe de changer de bibliothèque face à un problème est fréquent et coûteux ; il masque parfois une cause environnementale.

18.10 Solutions apportées

Le tableau 9 (section 13.14) récapitule les défauts et leurs causes. La synthèse des solutions par catégorie est la suivante :

Catégorie de problème	Solution générique retenue
Transport instable	Relance avec délai exponentiel borné, par pair

Contrainte d'API non documentée	Sérialisation et verrou explicite
Donnée réseau abîmée	Rattrapage local, jamais de propagation d'exception
État manquant au moment voulu	File d'attente réessayée à l'arrivée de l'information
Contrainte matérielle	Détection explicite et dégradation automatique
Échec silencieux	Propagation d'un verdict booléen sur toute la chaîne

18.11 Enseignements tirés

49. Ce qui n'est pas mesuré n'est pas fiable. Un système peut paraître fonctionner parfaitement tout en perdant 15 % de ses messages : il suffit qu'il ne sache pas le dire.
50. Une fonction qui ne renvoie rien ne peut pas échouer visiblement. Le type de retour `Future<void>` sur une opération faillible est un choix de conception à part entière, et ici c'était le mauvais.
51. L'observation sur appareil réel n'est pas une étape finale de validation, mais un outil de diagnostic quotidien. Sur les douze défauts significatifs recensés, aucun n'a été détecté par l'analyse statique ou les tests unitaires.
52. Devant un symptôme identique produit par deux implémentations différentes, il faut descendre d'un niveau plutôt que changer de bibliothèque.
53. Un message d'erreur doit nommer la cause réelle. « Vidéo illisible » accuse le logiciel ; « Téléphone en surchauffe, Android a coupé l'encodeur » informe et oriente.

PARTIE VI — BILAN PROFESSIONNEL

CHAPITRE 19 — BILAN DU STAGE

19.1 Objectifs atteints

Les six objectifs spécifiques énoncés en section 3.8 peuvent être évalués comme suit :

Objectif	État	Élément de preuve
Identité cryptographique locale	Atteint	Clé X25519 générée localement, tests unitaires
Découverte sur trois transports	Atteint	BLE, Wi-Fi local et Nearby opérationnels
Relais multi-hop	Atteint en conception, non validé sur le terrain	Protocole implémenté ; validation exige trois appareils
Chiffrement de bout en bout	Atteint	Déchiffrement vérifié par le pair simulé du banc
Persistance locale	Atteint	8 tables Drift, tests de sauvegarde/restauration
Mesure du taux de livraison	Atteint et déterminant	Banc à deux pairs ; défaut majeur identifié et corrigé

À COMPLÉTER PAR VOUS — Ajustez ce tableau au périmètre réel de VOTRE stage, et argumentez chaque ligne en une ou deux phrases.

19.2 Résultats techniques

Les principaux résultats techniques obtenus sont les suivants :

- Un prototype fonctionnel de 45 185 lignes de Dart, comptant 26 écrans, 17 services techniques et 8 tables de persistance.
- Une suite de 63 tests automatisés, avec une analyse statique sans avertissement.
- Un banc d'essai à deux pairs mesurant le taux de livraison réel, accompagné d'un rapport d'expérimentation documenté.
- Douze défauts significatifs identifiés et corrigés, dont un défaut majeur de fiabilité invalidant toute mesure antérieure.
- Un taux de livraison mesuré de 100 % en Wi-Fi et de 100 % pour les messages courts en Bluetooth, avec zéro écart entre livraison annoncée et réception effective.

19.3 Contributions personnelles

À COMPLÉTER PAR VOUS — Résumez VOTRE apport personnel. C'est la section que le jury lira le plus attentivement. Distinguez explicitement ce que vous avez conçu, implémenté, corrigé et documenté. Évitez la voix passive : « j'ai identifié », « j'ai implémenté », « j'ai mesuré ».

19.4 Compétences techniques acquises

Compétence	Exemple concret vécu durant le stage
Développement Flutter / Dart	[À compléter — citez un écran ou un composant que vous avez construit]
Réseaux et protocoles Mesh	Conception du routage par métrique composite, contrôle de congestion AIMD, déduplication par filtre de Bloom
Programmation Bluetooth Low Energy	Double rôle GATT central/périphérique, fragmentation en 19 octets
Bases de données embarquées	Schéma Drift à 8 tables, optimisation des écritures par lots
Cryptographie appliquée	Échange X25519, AES-256-GCM, mécanisme de Sender Keys avec ratchet
WebRTC	Appels pair-à-pair sans serveur, signalisation via le mesh
Conception d'interfaces	Système de design complet, modes clair et sombre
Méthodologie de test	Conception d'un banc d'essai mesurant une propriété que les tests unitaires ne peuvent pas atteindre
Diagnostic sur appareil réel	Analyse de journaux système ayant révélé douze défauts invisibles autrement

À COMPLÉTER PAR VOUS — Remplacez la première ligne et complétez les autres par VOS exemples.

19.5 Compétences professionnelles

Compétence	Situation vécue
Autonomie	[À compléter]
Rigueur méthodologique	Refus de conclure qu'une fonctionnalité marche sans l'avoir observée ; séparation stricte entre mesure et calcul dans le rapport de test
Résolution de problèmes	Diagnostic de la panne vidéo : révision d'une hypothèse initiale erronée après confrontation aux journaux
Esprit critique	Remise en cause d'un indicateur de fiabilité que l'application produisait elle-même

Communication écrite	Rédaction d'un rapport d'expérimentation et documentation du code
Organisation	[À compléter]
Travail en équipe	[À compléter]

À COMPLÉTER PAR VOUS — Complétez les lignes marquées et illustrez chacune par une situation précise et datée.

19.6 Apport du stage à mon projet professionnel

À COMPLÉTER PAR VOUS — Expliquez en quoi ce stage a influencé ou conforté votre projet professionnel. Un angle possible, si vous le partagez : ce projet montre que la difficulté d'un système distribué ne réside pas dans l'écriture du code, mais dans la construction des moyens de savoir s'il fonctionne.

CHAPITRE 20 — LIMITES ET PERSPECTIVES

20.1 Limites actuelles

Les limites du prototype, telles qu'établies par l'évaluation, sont les suivantes. Elles sont présentées sans atténuation, car une limite identifiée vaut mieux qu'une limite ignorée.

54. Validation multi-appareils incomplète. Le relais multi-hop, qui constitue le cœur de la proposition, n'a pas pu être validé sur le terrain : il exige au minimum trois appareils placés de sorte que les extrémités soient hors de portée l'une de l'autre.
55. Dépendance au Wi-Fi pour les contenus volumineux. En Bluetooth seul, ni les fichiers, ni les messages vocaux, ni les appels ne peuvent circuler. Or c'est précisément en l'absence d'infrastructure que le Bluetooth est parfois seul disponible.
56. Messages longs refusés en Bluetooth. Au-delà d'environ 210 caractères, la trame dépasse la limite de 512 octets et échoue. L'échec est désormais visible, mais le message devrait simplement passer.
57. Signalisation perfectible des transferts en attente. Un fichier qui ne peut pas partir faute de Wi-Fi est marqué « échoué », alors qu'il partirait dès qu'une liaison apparaîtrait. Un état intermédiaire « en attente d'une liaison Wi-Fi » est nécessaire.
58. Consommation énergétique non mesurée. La découverte continue sur trois radios est intrinsèquement coûteuse ; aucune mesure de l'autonomie sur une heure d'usage normal n'a été réalisée.
59. Couverture de test de l'interface limitée. Sur 26 écrans, un seul test de construction existe. Plusieurs défauts visuels n'ont été détectés que par capture d'écran.
60. Portage iOS non réalisé. Les technologies de proximité employées sont spécifiques à Android.

20.2 Sauvegarde et restauration

Un mécanisme de sauvegarde chiffrée de l'identité, protégé par mot de passe, est implémenté et testé. Les perspectives concernent l'extension de cette sauvegarde à l'historique complet des conversations et sa restauration sur un nouvel appareil.

20.3 Vérification par QR Code

La vérification par QR code et par numéro de sécurité est implémentée. La perspective principale est d'inciter davantage à cette vérification dans le parcours utilisateur : aujourd'hui, elle est disponible mais peu mise en avant, ce qui laisse la plupart des relations reposer sur la seule confiance initiale.

20.4 Recherche

Une recherche dans les conversations est implémentée : elle ne filtre pas la liste mais y navigue, en surlignant les occurrences. La perspective est d'étendre cette recherche à l'ensemble des conversations depuis l'écran d'accueil.

20.5 Contacts

Perspective : import de contacts et synchronisation entre les appareils d'un même utilisateur.

20.6 Fichiers de groupe

Le partage de fichiers en groupe s'appuie sur le chiffrement par Sender Keys. Perspective : reprise de transfert après coupure, cas non encore traité et particulièrement important sur un réseau intermittent.

20.7 Relais de groupes

Perspective : optimiser la diffusion des messages de groupe pour éviter qu'un même message ne traverse plusieurs fois le même segment de réseau.

20.8 Appels de groupe

Les appels de groupe sont implémentés et limités à quatre participants. Perspective : évaluer la faisabilité au-delà, sachant que le débit requis croît rapidement avec le nombre de flux.

20.9 Optimisation énergétique

Le rythme de découverte est déjà adaptatif. Perspective : mesurer l'autonomie réelle sur une heure d'usage, identifier le poste de consommation dominant, et supprimer ce qui ne se justifie pas.

20.10 Amélioration du Mesh

Deux améliorations prioritaires découlent directement des mesures :

61. Alléger l'enveloppe des messages. Transporter les identifiants sous forme binaire plutôt qu'hexadécimale économiserait environ 64 octets par message, soit près de 30 % de l'enveloppe minimale.
62. Fragmenter au niveau applicatif au-delà de la limite du transport, plutôt que de refuser la charge.

20.11 Renforcement de la sécurité

Perspectives : étendre le mécanisme de ratchet aux conversations 1:1, et faire réaliser un audit de sécurité indépendant. Le modèle actuel n'a pas été audité par un tiers.

20.12 Version future

La feuille de route suivante découle des limites identifiées, par ordre de priorité décroissante :

Priorité	Évolution	Justification
1	Essai sur trois appareils réels	Seul moyen de valider le relais multi-hop et de mesurer la portée

2	État « en attente d'une liaison Wi-Fi »	Corrige une information trompeuse pour l'utilisateur
3	Fragmentation applicative des messages longs	Supprime l'échec au-delà de 210 caractères
4	Mesure et optimisation de la consommation	Condition d'un usage prolongé réel
5	Tests de construction pour les 26 écrans	Aurait détecté plusieurs défauts visuels
6	Portage iOS	Élargissement du parc compatible

CONCLUSION GÉNÉRALE

Ce rapport a présenté le travail réalisé dans le cadre d'un stage consacré à la conception, au développement et à l'expérimentation de Droplet, une application de communication décentralisée fonctionnant sur un réseau maillé sans connexion Internet.

Le problème traité

Les communications mobiles reposent aujourd'hui presque entièrement sur des infrastructures centralisées dont l'indisponibilité — zone non couverte, catastrophe, saturation, coupure — prive brutalement les utilisateurs de tout moyen d'échanger. La question posée était donc de savoir comment permettre à des personnes proches les unes des autres de communiquer de manière fiable et privée sans aucune infrastructure.

La solution proposée

La solution repose sur l'exploitation des capacités de communication directe des smartphones. Chaque appareil découvre ses voisins par Bluetooth Low Energy, Wi-Fi local ou Wi-Fi Direct, et relaie les messages destinés à d'autres, formant un réseau maillé dont la portée dépasse celle d'un appareil isolé. Les messages sont chiffrés de bout en bout par échange de clés X25519 et chiffrement AES-256-GCM, de sorte que les appareils relais transportent sans jamais pouvoir lire. L'ensemble des données est conservé localement, sans aucun serveur.

À COMPLÉTER PAR VOUS — Insérez ici un paragraphe de 100 à 150 mots répondant à la troisième question : QU'AI-JE PERSONNELLEMENT RÉALISÉ ? Soyez précis et concret, citez des modules et des chiffres. C'est le paragraphe que le jury retiendra.

Les résultats obtenus

Le prototype obtenu compte 45 185 lignes de code, 26 écrans et 17 services techniques, validés par 63 tests automatisés. L'évaluation a été conduite selon une méthode adaptée à la nature décentralisée du système : puisqu'aucun serveur ne peut témoigner de ce qui a été reçu, un banc d'essai à deux pairs a été conçu, dans lequel un correspondant simulé déchiffre réellement les messages qu'il reçoit.

Ce banc a produit le résultat le plus important du stage, et ce résultat n'est pas une performance mais une découverte : l'application comptabilisait comme livrés des messages qui n'avaient jamais quitté l'appareil. Sur un lot de 130 envois, elle en annonçait 130 reçus quand le destinataire n'en avait obtenu que 110. La cause en était une méthode d'envoi qui ne renvoyait aucun verdict, et une couche supérieure qui interprétait cette absence comme un succès. Après correction sur l'ensemble de la

chaîne, l'égalité entre livraison annoncée et réception effective est rétablie, et un test de non-régression en conserve la trace.

Cette découverte porte un enseignement qui dépasse le projet : dans un système distribué, la capacité à SAVOIR qu'une opération a abouti n'est pas un supplément de confort, c'est une propriété qu'il faut construire délibérément. Sans elle, un système peut donner satisfaction très longtemps tout en perdant une partie de ce qu'on lui confie.

Les perspectives

Les prochaines étapes découlent directement des limites mesurées. La première est un essai sur trois appareils réels, seul moyen de valider le relais multi-hop et d'établir la portée effective. Viennent ensuite la signalisation des transferts en attente de Wi-Fi, la fragmentation applicative des messages longs, et la mesure de la consommation énergétique. À plus long terme, le portage vers iOS et un audit de sécurité indépendant conditionnent tout usage au-delà du prototype.

Droplet ne prétend pas remplacer les messageries existantes. Il répond à un besoin plus étroit mais réel : continuer à communiquer quand plus rien d'autre ne fonctionne. À l'issue de ce stage, le prototype démontre que ce besoin peut être techniquement satisfait, et — ce qui importe autant — il dispose désormais des moyens de mesurer dans quelle proportion il y parvient.

BIBLIOGRAPHIE

À COMPLÉTER PAR VOUS — Complétez avec les ouvrages et articles réellement consultés. Voici des références pertinentes pour ce sujet, à vérifier et compléter selon votre lecture effective :

- BERNSTEIN D. J., Curve25519 : new Diffie-Hellman speed records, Public Key Cryptography, 2006.
- BLOOM B. H., Space/Time Trade-offs in Hash Coding with Allowable Errors, Communications of the ACM, vol. 13, n° 7, 1970.
- PERKINS C., BELDING-ROYER E., DAS S., Ad hoc On-Demand Distance Vector (AODV) Routing, RFC 3561, IETF, 2003.
- CHANDRA R. et al., MeshNet : mobile ad hoc networking, IEEE Communications, [année à vérifier].
- MARLINSPIKE M., PERRIN T., The Double Ratchet Algorithm, Signal, 2016.
- DWORKIN M., Recommendation for Block Cipher Modes of Operation : Galois/Counter Mode (GCM), NIST SP 800-38D, 2007.

WEBOGRAPHIE

À COMPLÉTER PAR VOUS — *Complétez avec les dates de consultation réelles.*

- Flutter — Documentation officielle. <https://docs.flutter.dev> [consulté le ..].
- Dart — Documentation du langage. <https://dart.dev/guides> [consulté le ..].
- Riverpod — Documentation. <https://riverpod.dev> [consulté le ..].
- Drift — Documentation. <https://drift.simonbinder.eu> [consulté le ..].
- WebRTC — Documentation officielle. <https://webrtc.org> [consulté le ..].
- Android — Bluetooth Low Energy. <https://developer.android.com/develop/connectivity/bluetooth/ble> [consulté le ..].
- Android — Nearby Connections API. <https://developers.google.com/nearby/connections/overview> [consulté le ..].
- Android — Thermal API (PowerManager). <https://developer.android.com/reference/android/os/PowerManager> [consulté le ..].
- SQLite — Documentation. <https://www.sqlite.org/docs.html> [consulté le ..].
- Paquet cryptography (Dart). <https://pub.dev/packages/cryptography> [consulté le ..].

ANNEXES

Annexe A — Cahier des charges

À COMPLÉTER PAR VOUS — Insérez le cahier des charges du projet.

Annexe B — Planning du stage

À COMPLÉTER PAR VOUS — Insérez le diagramme de Gantt correspondant au tableau 2.7.

Annexe C — Diagrammes UML

Les diagrammes sont présentés dans le corps du rapport, aux endroits où ils argumentent : cas d'utilisation (figure 20), classes (figure 21), séquence d'envoi (figure 9), activité (figure 22), état d'un message (figure 4), composants (figure 23) et déploiement (figure 24).

Annexe D — Architecture technique

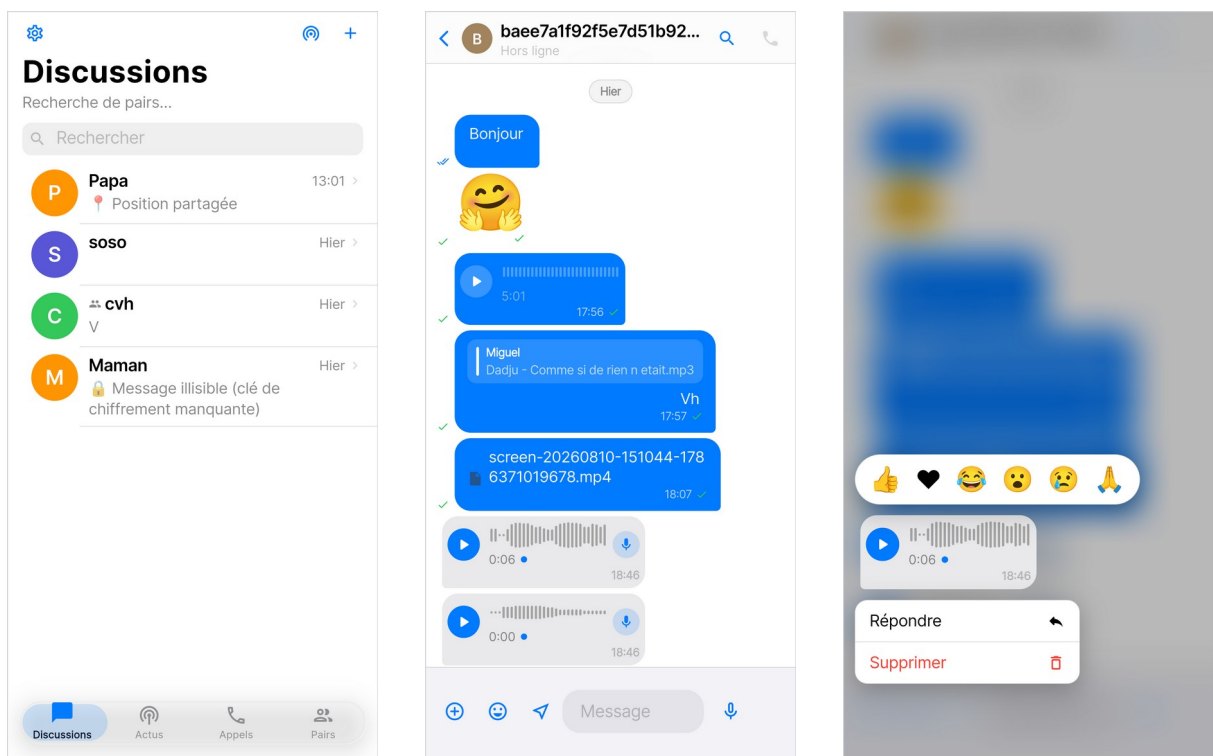
L'architecture en couches est présentée en figure 1, l'arborescence du projet en section 14.1 et le diagramme de composants en figure 23.

Annexe E — Schéma de base de données

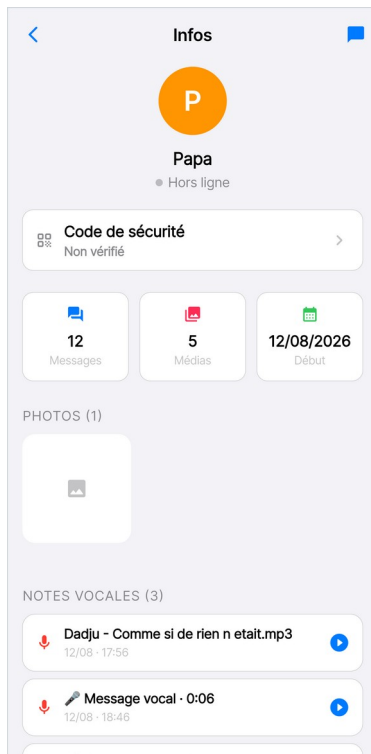
Le modèle conceptuel est présenté en figure 25 et le schéma physique des huit tables en figure 26.

Annexe F — Captures d'écran

Les seize captures ci-dessous ont été réalisées sur un Google Pixel 6 Pro à partir de la version de l'application livrée à l'issue du stage. La barre d'état du téléphone a été retirée, aucun autre traitement n'a été appliqué.

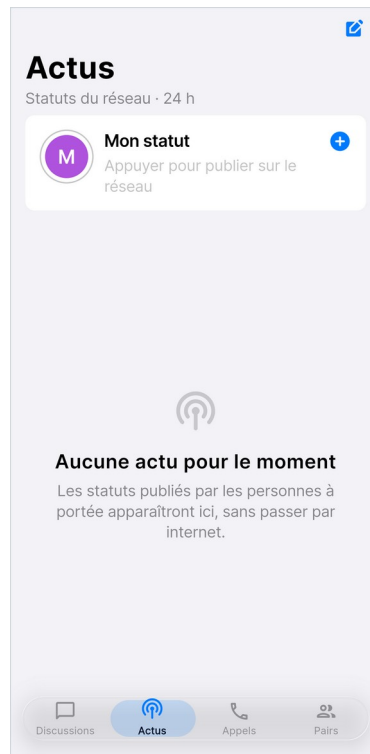


Discussions



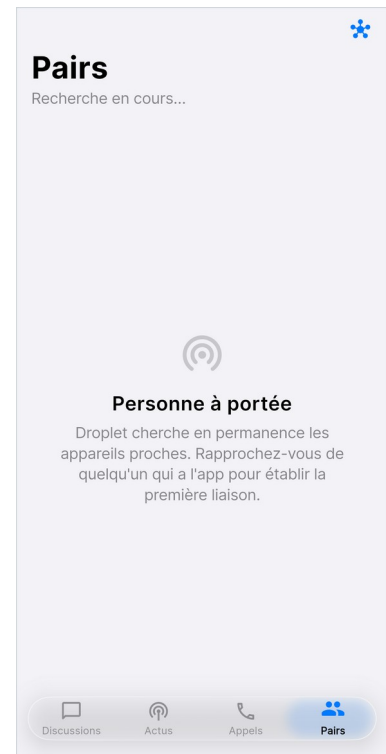
Infos et médias

Conversation

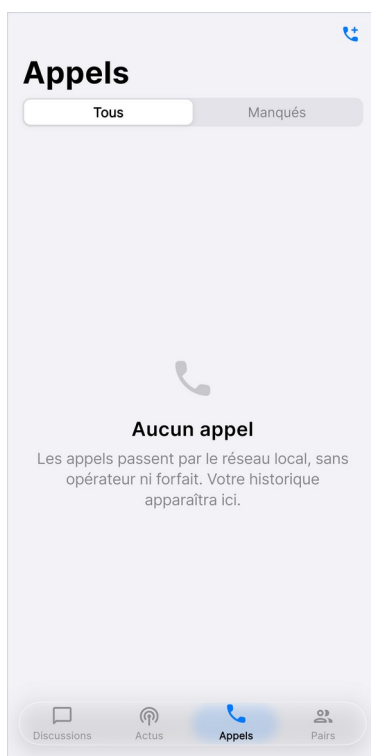


Actus

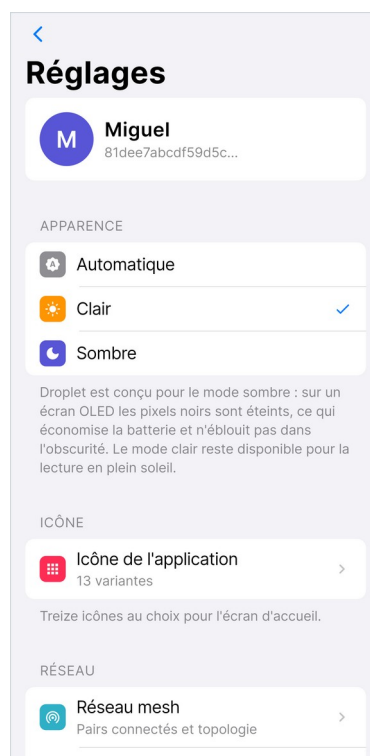
Menu contextuel



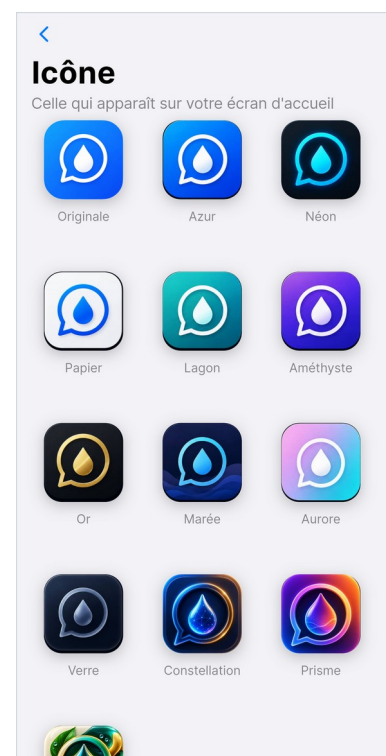
Pairs



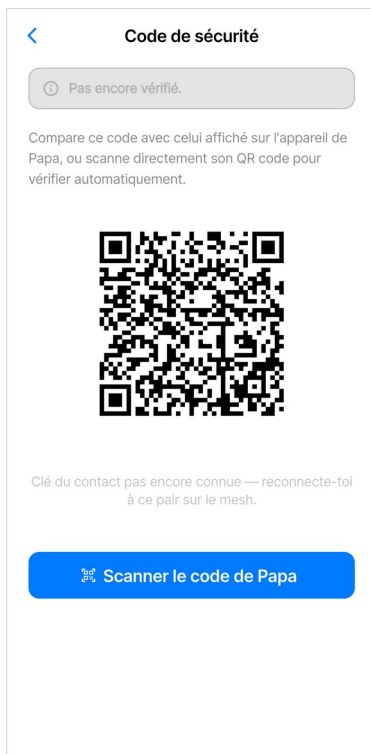
Appels



Réglages



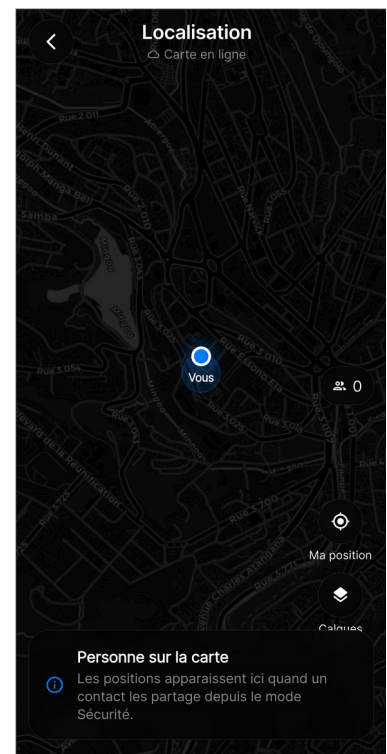
Icônes



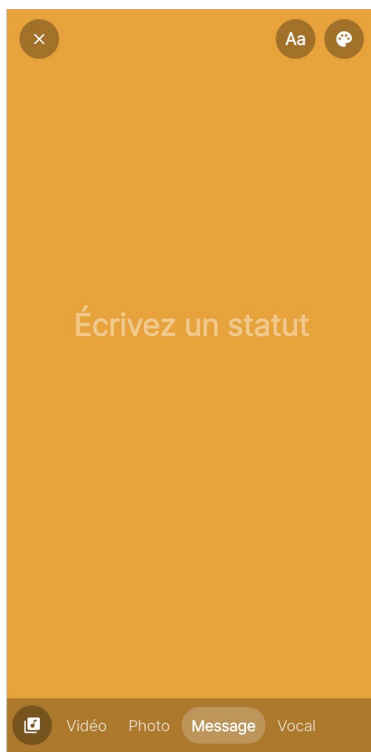
Code de sécurité



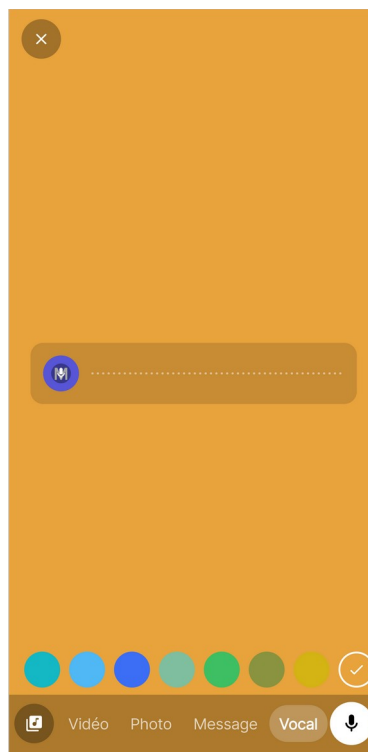
Cartes hors ligne



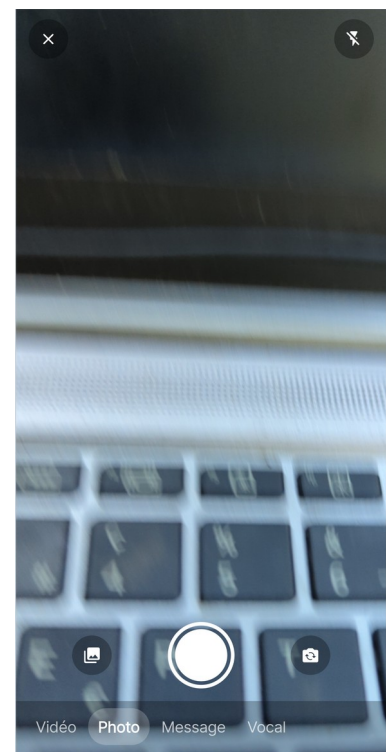
Cartographie



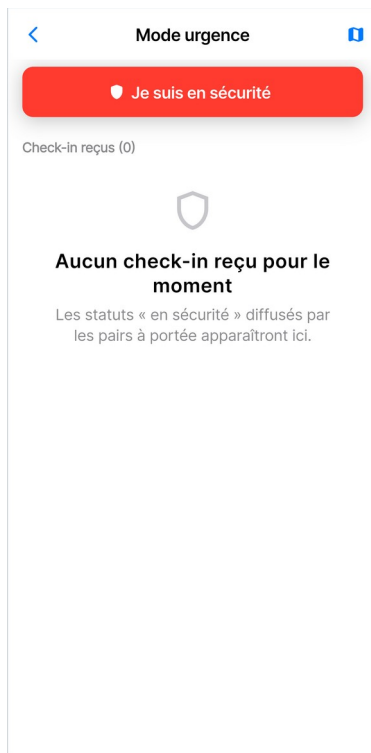
Statut — Message



Statut — Vocal



Statut — Photo



Mode urgence

Annexe G — Scénarios de tests

À COMPLÉTER PAR VOUS — Détaillez les scénarios exécutés, avec leur protocole précis.

Annexe H — Résultats détaillés des tests

À COMPLÉTER PAR VOUS — Reproduisez ici le rapport d'expérimentation *RAPPORT_SESSION_DEUX_PAIRS.md*, ainsi que les sorties brutes du banc d'essai.

Annexe I — Extraits de code

Les cinq extraits ci-dessous ont été choisis parce qu'ils portent chacun une décision de conception, et non parce qu'ils sont représentatifs du volume de code.

Extrait I.1 — Métrique composite de routage (*droplet_mesh_protocol.dart*)

```
double computeLinkMetric(String peerId) {
  final metrics = _linkMetrics[peerId];
  if (metrics == null) return 100.0; // métrique par défaut : mauvais

  // metric =  $\alpha \times \text{latence} + \beta \times (1 - \text{PDR}) + \gamma \times (1 - \text{batterie})$ 
  //  $\alpha = 0,4$   $\beta = 0,4$   $\gamma = 0,2$  – le PDR et la latence dominant.
  const alpha = 0.4, beta = 0.4, gamma = 0.2;

  final normalizedLatency = (metrics.latencyMs / 1000).clamp(0.0, 1.0);
  final pdrPenalty = 1.0 - metrics.packetDeliveryRatio;
  final batteryPenalty = 1.0 - metrics.batteryLevel;

  return alpha * normalizedLatency
    + beta * pdrPenalty
    + gamma * batteryPenalty;
}
```


Ce qu'il faut y lire — une métrique BASSE désigne un bon lien. Les trois pondérations sont le cœur de la décision : latence et fiabilité pèsent également lourd (0,4 chacune), la batterie du pair moitié moins (0,2). Ce dernier terme est ce qui distingue un réseau de téléphones d'un réseau d'équipements fixes — router systématiquement par l'appareil le plus rapide viderait sa batterie et ferait disparaître du maillage le meilleur relais. La ligne « return 100.0 » traite le cas d'un lien inconnu : en l'absence d'information, on suppose le pire.

Extrait I.2 — La correction de la livraison mensongère (mesh_transport_service.dart)

```
// AVANT – le refus du transport était rapporté comme un succès
Future<bool> sendViaRoute(String peerId, Uint8List data) async {
  if (_peers.containsKey(peerId)) {
    await sendToPeer(peerId, data); // Future<void> : ne rend rien
    return true;                  // ← toujours vrai
  }

  // APRÈS – le verdict du transport est propagé
  Future<bool> sendViaRoute(String peerId, Uint8List data,
    {int type = 0x00}) async {
    if (_peers.containsKey(peerId)) {
      return sendToPeer(peerId, data, type: type); // Future<bool>
    }
  }
}
```

Ce qu'il faut y lire — toute la différence tient dans le type de retour. Dans la version du haut, sendToPeer est un Future<void> : il ne peut RIEN dire. La ligne « return true » n'est donc pas une erreur d'inattention, c'est la seule chose que le code pouvait écrire — il n'avait aucune information à propager. Le défaut n'était pas dans cette ligne mais dans la signature de la fonction appelée. C'est l'enseignement principal : un type de retour void sur une opération qui peut échouer est un choix de conception, et ici c'était le mauvais.

Extrait I.3 — Les deux refus du transport Bluetooth (ble_mesh_transport.dart)

```
// BLE ne porte jamais de fichier ni de flux audio.
if (type >= 0x20) {
  debugPrint('[BLE] Refusé : type 0x\${type.toRadixString(16)}');
  return false; // ← rendait « void » auparavant
}
if (data.length > _maxPayloadSize) { // _maxPayloadSize = 512
  debugPrint('[BLE] Refusé : \${data.length} o > limite');
  return false;
}
```

Ce qu'il faut y lire — les deux conditions sont justes et doivent être conservées : un fichier n'a rien à faire sur un lien de 19 octets par écriture. Ce qui a changé tient au mot « false » à la place de « return » nu. Auparavant, ces deux refus produisaient un message de journal que personne ne lisait, et le message disparaissait. Le comportement fonctionnel est identique ; seule la transparence a changé.

Extrait I.4 — Délai exponentiel avec gigue (premium_message_queue.dart)

```
Duration get retryDelay {
  final baseMs = min(pow(2, attempt).toInt() * 100, 30000);
  final jitter = Random().nextInt(max(1, baseMs ~/ 4));
  return Duration(milliseconds: baseMs + jitter);
}
```

Ce qu'il faut y lire — deux mécanismes se superposent. L'exponentielle ($2^n \times 100$ ms) espace les tentatives pour laisser au réseau le temps de se rétablir. La gigue — un délai aléatoire ajouté — traite un problème plus subtil : si dix messages échouent en même temps parce qu'un pair s'est

éloigné, ils réessaieraient tous exactement au même instant et échoueraient de nouveau ensemble. La part aléatoire les disperse. Le plafond à 30 secondes évite qu'après cinq échecs, l'attente ne devienne absurde.

Extrait I.5 — Le correspondant du banc d'essai (test/two_peer_session_test.dart)

```
/// Le pair simulé déchiffre RÉELLEMENT ce qu'il reçoit.
Future<bool> receive(Uint8List frame) async {
  final payload = utf8.decode(frame.sublist(2));
  final map = json.decode(payload) as Map<String, dynamic>;

  if (map['e'] == true) {
    final clear = await CryptoService.decryptBytes(
      _shared!,
      base64Decode(map['c'] as String),
      base64Decode(map['n'] as String),
    );
    if (clear == null) return _reject();
    plaintexts.add(utf8.decode(clear));
  }
  received++;
  return true;
}
```

Ce qu'il faut y lire — c'est ce déchiffrement réel qui donne leur valeur à tous les chiffres du chapitre 17. Un pair simulé qui se contenterait de compter les paquets reçus validerait le transport, pas la chaîne complète : on pourrait obtenir 100 % de livraison avec des messages parfaitement illisibles à l'arrivée. Ici, le pair dérive son secret partagé de son propre côté, déchiffre, et rejette ce qu'il ne comprend pas — exactement comme le ferait un vrai téléphone.

Annexe J — Documentation API

À COMPLÉTER PAR VOUS — Documentez les interfaces publiques des principaux services.

Annexe K — Chronologie du développement

À COMPLÉTER PAR VOUS — Retracer les grandes étapes avec leurs dates.

Annexe L — Business Model Canvas

À COMPLÉTER PAR VOUS — Insérez le canvas synthétisant le chapitre 6.

Annexe M — Maquette / prototype

À COMPLÉTER PAR VOUS — Insérez les maquettes réalisées avant le développement, si elles existent.

Annexe N — Documents techniques produits pendant le stage

À COMPLÉTER PAR VOUS — Listez et joignez les documents produits : rapport d'expérimentation, notes de conception, comptes rendus.

NOTE SUR L'USAGE DE CE DOCUMENT

Ce document a été pré-rempli à partir du code source réel du projet Droplet et des mesures effectivement réalisées. Trois principes ont été suivis dans sa rédaction :

63. Aucun résultat n'a été inventé. Tous les chiffres cités (45 185 lignes, 63 tests, 130 envois, 233 octets d'enveloppe, Thermal Status 4...) proviennent d'une exécution ou d'une lecture réelle du code. Vous pouvez les vérifier.
64. Les sections qui vous sont personnelles — identité, entreprise, dates, missions, contributions — sont signalées en rouge par la mention « À COMPLÉTER PAR VOUS ». Elles ne peuvent pas être écrites à votre place, et le jury verra immédiatement si elles sonnent faux.
65. Les limites du travail sont exposées sans atténuation. Un rapport qui annonce que tout fonctionne parfaitement est moins crédible qu'un rapport qui sait dire ce qui n'a pas pu être validé, et pourquoi.

Avant de rendre ce document :

- Recherchez « À COMPLÉTER » dans Word (Ctrl+F) et traitez chaque occurrence.
- Recherchez « [» pour repérer les champs entre crochets restants.
- Supprimez cette page.
- Mettez à jour la table des matières (Références → Mettre à jour).
- Insérez les figures annoncées dans la liste des figures.
- Relisez le chapitre 13 et retirez tout ce qui ne relève pas de votre travail personnel.